November 4–8, 2025
Hong Kong, China

Association for
Computing Machinery

Advancing Computing as a Science & Profession

Open & AI RAN'25

Proceedings of the 2025

2nd ACM Workshop on Open and AI RAN

Sponsored by:

ACM SIGMOBILE

Part of:

Mobicom'25

Association for
Computing Machinery

Advancing Computing as a Science & Profession

The Association for Computing Machinery
2 Penn Plaza, Suite 701
New York, New York 10121-0701

ISBN: 979-8-4007-1977-6

Additional copies may be ordered prepaid from:
ACM Order Department
PO Box 30777
New York, NY 10087-0777, USA

Phone: +1 800 342-6626 (USA and Canada)
+1 212 626-0500 (Global)
Fax: +1 212 944-1318
Email: acmhelp@acm.org
Hours of Operation: 8:30 am–4:30 pm ET

Organizers

Steering Committee

Dinesh Bharadia	UC San Diego
Yongguang Zhang	Microsoft

Program Committee

TPC Co-chairs

Mahesh Marina	University of Edinburgh, UK
Yue Wang	China Telecom
Dinesh Bharadia	UC San Diego

Publicity and Publication chair

Ish Kumar Jain	Rensselaer Polytechnic Institute

Local Chair

Jun Zhang	The Hong Kong University of Science and Technology
Hongyang Du	The University of Hong Kong

TPC Members

Aditya Gudipati	Arista Networks
Andres Garcia-Saavedra	NEC Laboratories Europe
Antony Franklin	IIT Hyderabad, India
Bheemarjuna Reddy Tamma	IIT Hyderabad, India
Bozidar Radunovic	Microsoft Research
Carla Fabiana Chiasserini	Politecnico di Torino
Dimitrios Koutsonikolas	Northeastern University
Dinesh Bharadia	UC San Diego
Falko Dressler	Technische Universität Berlin
Haitham Al Hassanieh	EPFL
Ish Kumar Jain	Rensselaer Polytechnic Institute (RPI)
Juheon Yi	Microsoft Research Asia
Koteswararao Kondepu	IIT Dharwad, India
Michele Polese	Northeastern University
Qingtian Wang	China Telecom Research Institute
Rahman Doost-Mohammady	Rice University
Salvatore D'Oro	Northeastern University
Shenghui Song	Hong Kong University of Science and Technology

Srinivas Shakkattoi	Texas A&M University
Sumei Sun	Institute for Infocomm Research, A*STAR, Singapore
Syed Zaidi	University of Leeds
Tony Quek	Singapore University of Technology and Design (SUTD)
Xenofon Foukas	Microsoft Research
Zhaowei Tan	University of California, Riverside

Contents

LLM-5GMAC: Performance Optimization in O-RAN Split 7.2 Using LLM-Based MAC-Layer Log Analysis

Osamu Kamatani
Kobe International University
Hyogo, Japan
kamatani@kobe-kiu.ac.jp

Shunsuke Saruwatari
Osaka University
Osaka, Japan
saru@ist.osaka-u.ac.jp

Sushila Seshasayee
University of California San Diego
California, USA
sseshasayee@ucsd.edu

Ali Mamaghani
University of California San Diego
California, USA
amamaghani@ucsd.edu

Dinesh Bharadia
University of California San Diego
California, USA
dineshb@ucsd.edu

ABSTRACT

The adoption of Open RAN and virtualized RAN (vRAN) architectures promises increased interoperability and cloud-native flexibility, but introduces operational challenges including limited visibility in low-MAC layers. We present LLM-5GMAC, a framework using Large Language Models to analyze MAC-Layer logs from Split 7.2 vRANs, transforming technical logs into actionable insights.

Our evaluation reveals that LLM analysis quality depends critically on prompt specificity—while general prompts yield surface-level insights, targeted prompts enable sophisticated failure analysis including autonomous detection of 305ms cascades. This dependency creates operational consistency challenges, highlighting the need for AI-agent-based approaches to ensure reliable network diagnostics independent of operator expertise. We propose a Multi-Agent Framework that addresses these limitations through coordinated prompt generation and validation.

KEYWORDS

Open RAN, vRAN, Split 7.2, MAC-Layer log analysis, Large Language Models (LLM), Performance optimization, Prompt dependency

ACM Reference Format:
Osamu Kamatani, Shunsuke Saruwatari, Sushila Seshasayee, Ali Mamaghani, and Dinesh Bharadia. 2025. LLM-5GMAC: Performance Optimization in O-RAN Split 7.2 Using LLM-Based MAC-Layer Log Analysis. In *2nd ACM Workshop on Open and AI RAN (Open & AI RAN 25), November 4–8, 2025, Hong Kong, China.* ACM, New York, NY, USA, 7 pages. https://doi.org/10.1145/3737900.3770164

Open & AI RAN 25, November 4–8, 2025, Hong Kong, China

ACM ISBN 979-8-4007-1977-6/25/11
https://doi.org/10.1145/3737900.3770164

1 INTRODUCTION

The evolution of mobile network architectures toward openness and virtualization is reshaping Radio Access Networks (RANs). Open RAN, through O-RAN Alliance specifications, promotes disaggregation of baseband functions and standardized interfaces. Split 7.2 divides PHY processing between DU (higher PHY) and RU (lower PHY), while centralizing MAC functions in the DU, creating unique observability challenges where MAC logs must coordinate with distributed PHY across fronthaul [7]. Combined with virtualized RAN (vRAN) deployments, this enables cloud-native infrastructure and multi-vendor support [2].

However, these advantages introduce operational challenges. In Split 7.2-based vRANs, key components like DU and Central Unit (CU) run as containers on shared cloud infrastructure, creating deeply layered dependencies. Failures become difficult to detect and interpret—especially in the time-critical MAC layer operations within the DU component. Log information is fragmented across components, and existing visibility tools are vendor-specific or require deep domain expertise [1, 4].

In this work, we propose **LLM-5GMAC**, a framework leveraging Large Language Models (LLMs) to analyze detailed MAC-layer logs from Split 7.2 vRAN systems. We implement our framework using logs from a testbed built with OpenAirInterface and NVIDIA Aerial [6]. However, our investigation reveals a critical operational challenge: while modern LLMs like GPT-5 [9] demonstrate remarkable capability in network log analysis, their effectiveness is fundamentally dependent on prompt specificity and operator

expertise. This operator-dependent variability creates significant challenges for consistent network operations, where different skill levels lead to varying diagnostic outcomes.

As illustrated in Figure 1, our framework addresses both technical challenges of MAC log interpretation and the operational challenge of prompt dependency.

The contributions of this paper are as follows:

- We identify critical operational challenges in analyzing low-MAC logs within Split 7.2-based vRAN architectures and propose **LLM-5GMAC**, a framework leveraging LLMs while revealing critical dependencies between prompt formulation strategies and analysis quality.
- We demonstrate the feasibility and effectiveness through experimental evaluations using real-world low-MAC logs from an OpenAirInterface + NVIDIA Aerial testbed.
- We discuss how the identified prompt dependency challenge can be addressed through AI-agent-based approaches for consistent network diagnostics in future AI-native O-RAN environments.

2 BACKGROUND AND RELATED WORK

Open RAN, promoted by the O-RAN Alliance, defines a modular RAN architecture enabling multi-vendor interoperability. Split 7.2 is particularly relevant to commercial deployments due to its balance between implementation complexity and fronthaul efficiency. In this split, the Radio Unit (O-RU) and Distributed Unit (O-DU) connect via low-layer split (LLS), where PHY functions are divided between high and low PHY, while MAC functions remain entirely in the DU. This enhances flexibility but places stringent synchronization requirements between units [7].

Parallelly, vRAN shifts from dedicated hardware to cloud-based software components. While enabling rapid deployment and resource sharing, this introduces observability and fault diagnosis challenges. Virtualized components add multiple layers (hardware, virtualization, OS, network stack, RAN function), making anomalies difficult to trace [1, 4].

Conventional diagnostic approaches include vendor provided O1-based KPI monitoring, I/Q signal capture, and RIC/xApp-based policy control using metrics like RSRP, SINR, or BLER. However, these suffer from limited low-MAC visibility, vendor-specific granularity, high resource costs, and need for specialized expertise. In multi-vendor deployments, such heterogeneity exacerbates integrated analysis difficulties. Traditional logging systems lack semantic pattern extraction capabilities.

Recently, Large Language Models (LLMs) have emerged as powerful tools for interpreting complex, unstructured sequences like logs and event traces. Recent work on LLM-based network analysis includes LogLLM [3] for log anomaly detection, LLMcap [11] for PCAP analysis, and [5] for context-aware Wi-Fi roaming. LLMs like GPT-4 and GPT-5 [8, 9] demonstrate remarkable capabilities in processing technical documentation and identifying patterns in complex data structures. However, their application in real-time RAN observability and low-layer MAC log interpretation remains largely unexplored—this paper addresses that gap while examining the critical dependency between prompt formulation strategies and analysis effectiveness.

3 SYSTEM DESIGN: LLM-5GMAC FRAMEWORK

The LLM-5GMAC framework is designed to extract high-level operational insights from MAC-layer logs in O-RAN Split 7.2 vRAN systems. It addresses the gap between fragmented, low-level protocol traces and the need for interpretable, actionable diagnostics. As illustrated in Figure 1, our system is built around two key stages: log acquisition and LLM-based interpretation with strategic prompt formulation. Importantly, our evaluation reveals a critical dependency between prompt strategy and analysis quality that has significant implications for operational deployment.

3.1 Log Acquisition and Preprocessing

The first stage involves collecting detailed logs from the MAC layer of the gNB, particularly focusing on the O-DU side of the Split 7.2 interface. In our testbed, these logs are captured from OpenAirInterface with the NVIDIA Aerial platform [6, 10], where MAC-level debug traces are recorded in real time. Logs include messages such as scheduling decisions, HARQ activity, CQI feedback, PUSCH/PUCCH transmissions, and timing misalignments. Given their size and granularity (up to tens of MB per minute), logs are optionally filtered by severity level (e.g., Info or Debug) and temporally segmented to focus on time windows surrounding observed anomalies.

3.2 LLM-Based Analysis and Prompt Strategy Impact

The second stage uses a Large Language Model, specifically GPT-5 [9], to interpret the MAC log segments. However, our investigation reveals that the effectiveness of this analysis is fundamentally dependent on prompt formulation strategy—a critical operational challenge that significantly impacts diagnostic quality.

As demonstrated in Figure 2, our framework exposes two distinct analysis modes based on prompt specificity:

General Prompting Approach: When operators use general prompts such as "Analyze the wireless link quality from the MAC layer logs...", the LLM produces surface-level

statistical summaries and generic recommendations. While this approach is accessible to operators with limited domain expertise, it often misses critical error patterns and fails to identify specific technical issues.

Targeted Error-Focused Prompting: When operators formulate targeted prompts such as "Analyze the CQI measurement error from the MAC layer logs...", the same LLM demonstrates remarkable analytical capabilities. In our experiments using GPT-5, targeted prompts enabled autonomous identification of complex multi-layer failures, including the 305ms cascade from L2/L1 timing desynchronization through UCI corruption to CQI failure, without requiring explicit timestamp guidance.

This significant variation in analytical outcomes based on prompt formulation presents a fundamental challenge for consistent network operations. The implications include:

- **Operator Skill Dependency**: The same technical problem may receive dramatically different analytical treatment depending on the operator's ability to formulate effective prompts.
- **Inconsistent Diagnostic Quality**: Network operations teams with varying expertise levels will obtain different insights from identical log data, potentially leading to inconsistent troubleshooting approaches.
- **Knowledge Gap Amplification**: Rather than democratizing expert-level analysis, LLM dependency on operator expertise may exacerbate differences between experienced and novice operators.

3.3 Architecture and Limitations

Our current implementation demonstrates both the potential and constraints of LLM-based network diagnostics. While targeted prompts can reveal sophisticated technical insights, such as the precise timing relationships in MAC-layer failures—they require domain expertise to formulate effectively. Additionally, even targeted analysis may miss broader operational context, as evidenced by our discovery that apparent CQI failures were actually part of normal UE disconnection sequences, revealed only through follow-up questioning.

The modular design of LLM-5GMAC enables extension with additional input modalities (e.g., PHY logs, container metrics) and integration into existing observability stacks. However, the critical finding of prompt dependency necessitates future architectural enhancements that can provide consistent analytical quality independent of operator expertise. This challenge points toward the need for AI-agent-based approaches that can automate prompt optimization and eliminate operator skill dependencies.

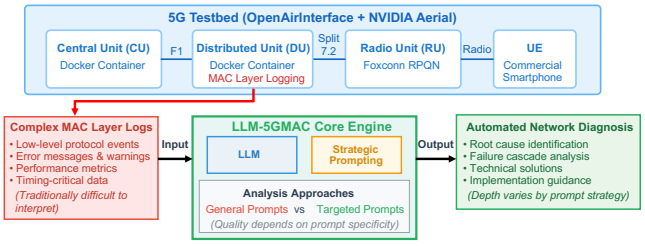


Figure 1: LLM-5GMAC System Overview. The framework processes MAC layer logs using LLM with strategic prompting approaches, revealing critical dependencies between prompt specificity and analysis quality.

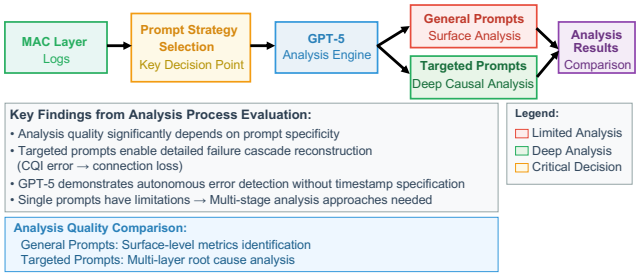


Figure 2: LLM-5GMAC Analysis Process showing prompt strategy impact. General prompts yield surface-level analysis while targeted prompts enable deep root cause identification, including autonomous detection of the 305ms failure cascade.

4 EXPERIMENTAL SETUP AND METHODOLOGY

To evaluate the LLM-5GMAC framework in a realistic setting, we constructed a 5G non-standalone vRAN testbed based on OpenAirInterface (OAI) and NVIDIA Aerial SDK. This testbed emulates an O-RAN Split 7.2 architecture, where the gNB stack is disaggregated into separate Central Unit (CU) and Distributed Unit (DU) components running as Docker containers. The DU and CU communicate via the F1 interface, and a SmartNIC-enabled RU handles radio processing using the Aerial platform. The testbed server is equipped with high-performance CPUs and NVIDIA GPUs to support real-time vRAN workloads.

As shown in Figure 3, the CU container includes the MAC processing logic and is the main observation point for our logging-based analysis. OAI's logging subsystem provides multiple verbosity levels, including Error, Info, Debug, and Trace. For most experiments, we configured the DU to log at the Info level, which offers a suitable trade-off between information richness and data volume (typically 60–70 KB per minute).

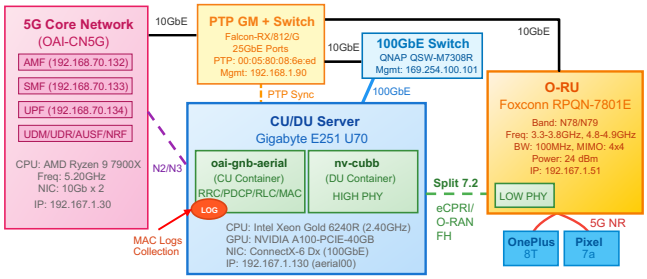


Figure 3: Split 7.2 vRAN testbed architecture. Split 7.2 divides PHY layer between DU (High PHY) and RU (Low PHY), while MAC remains in CU container.

To generate traffic for analysis, we connect a commercial smartphone to the testbed and initiate bi-directional data transfers using the SpeedTest app. The session runs for approximately one minute, during which MAC-layer logs are captured continuously. These logs include events such as PUSCH and PUCCH scheduling, HARQ behavior, CSI feedback, and MAC timer states. After collection, the logs are scanned for performance anomalies such as DL throughput drops, uplink underutilization, or excessive retransmissions.

Log segments surrounding the anomalies are extracted and prepared for LLM input. Figure 2 illustrates the processing pipeline: a task-specific prompt template is combined with the extracted log snippet, forming the input to a large language model such as GPT-5 [9]. The prompt typically requests analysis, anomaly detection, and actionable insights. LLM responses are stored as structured summaries, optionally annotated by human operators for validation.

This setup enables us to assess the reasoning capabilities of LLMs and examine the quality of the outputs in terms of diagnostic accuracy, interpretability, and consistency over multiple trials. In later sections, we specifically investigate how prompt formulation strategies impact analysis quality and operational consistency.

5 EVALUATION AND RESULTS

Our evaluation focuses on a critical discovery: the fundamental dependency between prompt formulation strategies and LLM analysis quality in network diagnostics. Using identical MAC layer logs from our Split 7.2 vRAN testbed, we demonstrate how operator expertise and prompting approach directly impact diagnostic outcomes—a finding with significant implications for operational deployment.

Figure 4 presents a timeline of events extracted from MAC-layer logs during a downlink throughput degradation event. The 305ms chronological sequence shows progression from normal operation through CQI measurement errors to complete UE disconnection. This event timeline serves as input for comparative LLM analysis using different prompting strategies.

5.1 Prompt Dependency Analysis and Technical Discovery

We conducted controlled experiments using identical low-MAC log segments to assess how different prompting strategies affect LLM analysis quality. The results reveal a stark contrast between general and targeted approaches, exposing a fundamental challenge for consistent network operations.

General Prompting Approach: Figure 5 shows results from general prompts typically used by operators with limited domain expertise: "Analyze the wireless link quality from the MAC layer logs..." This approach yields surface-level insights, focusing on standard metrics like RSRP and CQI averages. Critically, this general analysis completely missed the CQI measurement failures present in the logs, demonstrating how accessible prompting approaches may fail to identify specific technical issues.

Targeted Error-Focused Prompting: In contrast, Figure 6 demonstrates the impact of targeted error-focused prompting: "Analyze the CQI measurement error from the MAC layer logs..." This specific approach enabled GPT-5 [9] to autonomously identify complex multi-layer failures and provide detailed technical remediation strategies.

Our targeted prompting experiments revealed a sophisticated failure pattern that exemplifies LLM diagnostic capabilities when properly prompted. Using GPT-5 without explicit timestamp guidance, the system autonomously identified:

- **L2/L1 Timing Desynchronization**: Initial timing mismatch between MAC and PHY layer processing
- **UCI Corruption**: Subsequent corruption of Uplink Control Information due to timing issues
- **CQI Measurement Failure**: Final manifestation as Channel Quality Indicator measurement errors
- **305ms Cascade Timing**: Duration from L2/L1 timing desync to UE disconnection

This multi-layer root cause correlation (timing → UCI → CQI) demonstrates the potential for LLM-based analysis to identify complex technical relationships that would be challenging for traditional rule-based systems.

5.2 Operational Challenges and Limitations

The significant variation in diagnostic quality based on operator prompting skills presents a critical challenge for consistent network operations. Key findings include:

Skill-Dependent Outcomes: Experienced operators who understand specific error patterns can formulate targeted prompts, while less experienced operators default to general queries, receiving dramatically different analytical value from the same system.

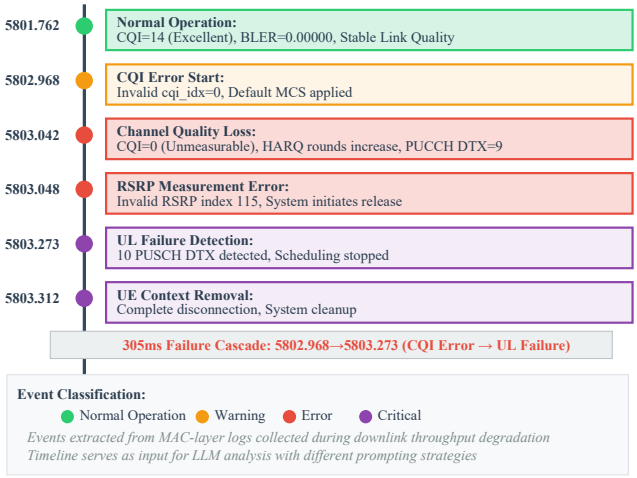


Figure 4: Timeline of MAC-layer events extracted from logs during a 305ms failure cascade. The chronological sequence shows the progression from normal operation (CQI=14) through measurement errors to complete UE disconnection.

Knowledge Gap Amplification: Rather than democratizing expert-level analysis, current LLM dependency on operator expertise may exacerbate differences between experienced and novice operators.

Inconsistent Operational Response: Identical network conditions may receive different levels of analysis and varying mitigation strategies based on who performs the diagnosis.

Further investigation revealed additional complexity beyond single-prompt analysis. Follow-up questioning indicated that the observed CQI failures were likely part of a normal UE disconnection sequence rather than RF quality issues, demonstrating that even sophisticated targeted prompts may miss broader operational context. This finding highlights the fundamental limitation of single-prompt approaches, regardless of operator skill level, and points toward the need for iterative, multi-perspective analysis approaches.

Across multiple test sessions, we observed consistent patterns: while LLM-5GMAC successfully detected various anomalies such as scheduling gaps, PUCCH DTX irregularities, and throughput asymmetries, the quality and depth of insights remained fundamentally dependent on prompt formulation strategy.

6 DISCUSSION

Our evaluation demonstrates that while modern LLMs like GPT-5 possess remarkable analytical capabilities for complex network diagnostics, a critical gap exists between technical capability and practical deployment effectiveness. This gap

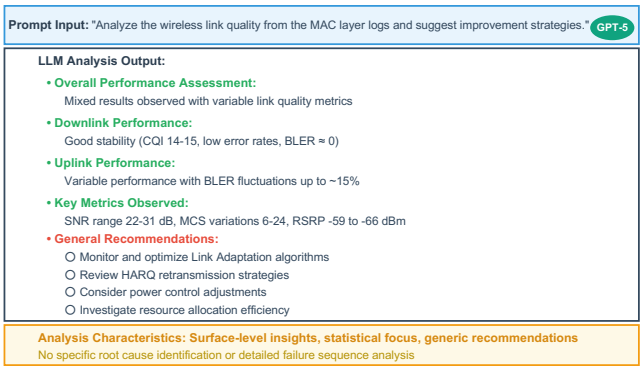


Figure 5: General prompt analysis results showing surface-level performance assessment without specific root cause identification.

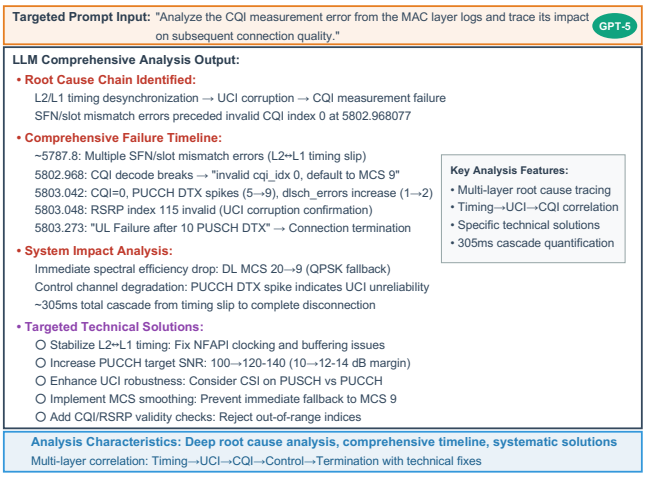


Figure 6: Targeted error analysis revealing multi-layer root cause correlation and 305ms failure cascade autonomous detection.

manifests as prompt dependency—where identical technical problems receive dramatically different analytical treatment based on operator skill and prompting strategy.

The LLM demonstrated sophisticated ability to interpret scheduling behavior, detect subtle anomalies like the 305ms failure cascade, and produce explanations that align with expert reasoning—all without explicit rule definitions or labeled training data. However, accessing these capabilities requires domain expertise to formulate effective prompts, creating a fundamental paradox: the systems designed to democratize expert-level analysis instead amplify the importance of operator expertise.

This dependency on operator expertise creates potential inconsistencies in network management, simultaneously risking to widen the gap between experienced and novice

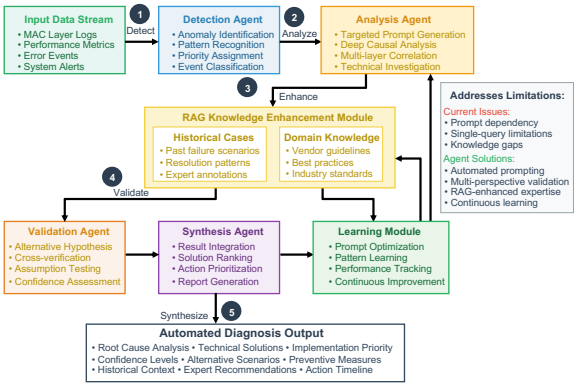


Figure 7: Proposed Multi-Agent Framework for Automated Network Analysis. The system addresses prompt dependency through coordinated agents for detection, automated prompting, validation, and synthesis with RAG support.

operators. Addressing this challenge requires systematic prompt optimization and intelligent intermediation systems independent of operator skill level.

Figure 7 shows our proposed Multi-Agent Framework approach with Detection, Analysis, Validation, and Synthesis agents. This coordinated system eliminates prompt dependency by automatically generating optimal prompts and ensuring consistent diagnostic quality across skill levels, with RAG providing historical context. The modular design principles demonstrated in LLM-5GMAC lay the groundwork for extending beyond MAC logs to encompass PHY-layer data, container metrics, or fronthaul indicators. Integration with RIC/xApp frameworks could enable real-time feedback for closed-loop optimization.

Our implementation uses GPT-5 conducting offline analysis. Future work could explore smaller language models (SLMs) for resource-constrained scenarios, though multi-layer protocol analysis may require larger models' capabilities. While LLM-5GMAC currently serves as an operator assistive tool, the challenges identified point toward autonomous fault management for AI-native O-RAN deployments.

7 CONCLUSION

In this paper, we introduced LLM-5GMAC, a novel framework for interpreting low-MAC layer logs in O-RAN Split 7.2 vRAN environments using Large Language Models. Our approach addresses key operational challenges—such as log fragmentation, observability gaps, and diagnostic complexity—by transforming raw MAC-layer log data into human-readable, actionable insights.

Through experiments conducted on a real 5G testbed using OpenAirInterface and NVIDIA Aerial, we demonstrated that modern LLMs like GPT-5 possess remarkable analytical capabilities for complex network diagnostics, including autonomous identification of sophisticated failure patterns such as the 305ms L2/L1 timing cascade leading to CQI measurement errors. The system operates without vendor-specific instrumentation or hand-crafted rules, making it highly adaptable to multi-vendor and cloud-native deployments.

However, our evaluation revealed a critical operational challenge that represents the primary contribution of this work: **LLM analysis quality depends fundamentally on prompt specificity and operator expertise**. While general prompts yield surface-level insights accessible to novice operators, sophisticated diagnostic capabilities require targeted, domain-specific prompting that demands expert knowledge. This creates a fundamental paradox where systems designed to democratize expertise instead amplify the importance of operator skill, potentially widening rather than closing the knowledge gap in network operations.

This discovery has profound implications for the practical deployment of AI-assisted network diagnostics. The challenge lies not in improving LLM performance—current models already demonstrate sophisticated analytical capabilities—but in developing systematic approaches to eliminate prompt dependency and ensure consistent diagnostic quality independent of operator expertise.

Our findings point toward the need for intelligent intermediation systems, such as the AI-agent-based framework proposed in this work (Figure 7). This Multi-Agent approach with coordinated Detection, Analysis, Validation, and Synthesis agents can automatically generate optimal prompts, perform multi-perspective validation, and provide consistent operational guidance across all skill levels. Such systems would preserve the interpretability and technical depth demonstrated by our LLM-based approach while addressing the critical operational consistency challenges identified.

LLM-5GMAC represents an important step toward understanding the fundamental requirements for automated, interpretable, and scalable log analysis in vRAN environments. By identifying both the remarkable potential and the critical limitations of LLM-based network diagnostics, this study establishes a foundation for developing more robust, operator-independent AI systems essential for future AI-native O-RAN deployments.

The core insight of this work is that achieving reliable AI-assisted network operations requires not just advanced AI capabilities, but intelligent systems that can bridge the gap between AI potential and operational reality. Our proposed Multi-Agent Framework represents a concrete step toward this goal for future AI-native O-RAN deployments.

REFERENCES

[1] Ericsson. 2022. 5G RAN Disaggregation: Architecture and Challenges. *Ericsson Technology Review* (2022). https://www.ericsson.com/en/reports-and-papers/ericsson-technology-review/articles/5g-ran-disaggregation-architecture-and-challenges

[2] GSMA. 2022. *Open RAN Industry White Paper.* Technical Report. GSMA. https://www.gsma.com/futurenetworks/resources/openran/

[3] Wei Guan, Jian Cao, Shiyou Qian, Jianqi Gao, and Chun Ouyang. 2024. LogLLM: Log-based Anomaly Detection Using Large Language Models. arXiv:2411.08561 [cs.LG] https://arxiv.org/abs/2411.08561 Submitted on 13 Nov 2024 (v1), last revised 14 Apr 2025 (v5).

[4] Intel Corporation. 2021. *Virtualized RAN Deployment Guide.* https://www.intel.com/content/www/us/en/communications/virtualized-ran-deployment-guide.html White Paper.

[5] Ju-Hyung Lee, Yanqing Lu, and Klaus Doppler. 2025. On-Device LLM for Context-Aware Wi-Fi Roaming. arXiv:2505.04174 [cs.NI] https://arxiv.org/abs/2505.04174 Submitted on 7 May 2025 (v1), last revised 20 May 2025 (v2).

[6] NVIDIA Corporation. 2023. NVIDIA Aerial SDK: AI-Driven 5G vRAN Platform. https://developer.nvidia.com/aerial-sdk.

[7] O-RAN Alliance. 2023. O-RAN Architecture Description v6.0. https://www.o-ran.org/specifications.

[8] OpenAI. 2023. GPT-4 Technical Report. https://openai.com/research/gpt-4.

[9] OpenAI. 2025. *Introducing GPT-5.* https://openai.com/index/introducing-gpt-5/ OpenAI's official announcement of GPT-5 model.

[10] OpenAirInterface Project. 2024. OpenAirInterface 5G RAN Stack. https://openairinterface.org/.

[11] Lukasz Tulczyjew, Kinan Jarrah, Charles Abondo, Dina Bennett, and Nathanael Weill. 2024. LLMcap: Large Language Model for Unsupervised PCAP Failure Detection. arXiv:2407.06085 [cs.LG] https://arxiv.org/abs/2407.06085 Submitted on 3 Jul 2024.

Comprehensive End-to-End Performance Evaluation of a Multi-Vendor 5G O-RAN-based Testbed

Pedro Rezende
Future Communications Translation Lab, Singapore Institute of Technology
Singapore
phamorimrezende@gmail.com

Shyam Mahato
Future Communications Translation Lab, Singapore Institute of Technology
Singapore
shyam.mahato@singaporetech.edu.sg

Amogh PC
Future Communications Translation Lab, Singapore Institute of Technology
Singapore
amogh.pc@singaporetech.edu.sg

Aris Risdianto
Future Communications Translation Lab, Singapore Institute of Technology
Singapore
ariscahyadi.risdianto@singaporetech.edu.sg

Vignesh Nagamuthu
Future Communications Translation Lab, Singapore Institute of Technology
Singapore
nagamuthu.vignesh@singaporetech.edu.sg

Yiyang Pei
Future Communications Translation Lab, Singapore Institute of Technology
Singapore
yiyang.pei@singaporetech.edu.sg

Sumei Sun
Future Communications Translation Lab, Singapore Institute of Technology
Singapore
sumei.sun@singaporetech.edu.sg

Abstract

Open RAN introduces a paradigm shift in Radio Access Network (RAN) design by leveraging disaggregated and virtualized functions connected through open interfaces. While this approach enhances flexibility and vendor diversity in 5G deployments, it also introduces new performance and interoperability challenges, especially in multi-vendor environments. In this paper, we present a comprehensive performance evaluation of a private 5G O-RAN-based testbed composed of commercial off-the-shelf components from multiple vendors. Leveraging a cutting-edge User Equipment Simulator (UESim), we assess end-to-end (E2E) downlink performance under varying traffic conditions, user loads, and radio signal qualities. Our findings show that under high traffic, application-level downlink throughput can decrease by 50%, and latency can increase by over 3× due to signal degradation. Similarly, increasing the number of user equipments (UEs) significantly impacts latency, even when total data throughput remains constant. These results underscore the importance of optimal O-RU placement and dynamic traffic/load-aware planning to meet Quality of Service (QoS) requirements in O-RAN deployments.

CCS Concepts

• **Networks** → **Network performance analysis**; **Mobile networks**.

Keywords

5G, O-RAN, UESim, Testbed, Network Performance Analysis

ACM Reference Format:
Pedro Rezende, Shyam Mahato, Amogh PC, Aris Risdianto, Vignesh Nagamuthu, Yiyang Pei, and Sumei Sun. 2025. Comprehensive End-to-End Performance Evaluation of a Multi-Vendor 5G O-RAN-based Testbed. In *2nd ACM Workshop on Open and AI RAN (OpenRan '25), November 4–8, 2025, Hong Kong, China.* ACM, Hong Kong, 7 pages. https://doi.org/10.1145/3737900.3770165

1 Introduction

5G networks [1] are an essential communication system utilized by various applications, including high-resolution video streaming, telemedicine, unmanned aerial vehicles, and manufacturing. According to the Ericsson Mobility Report [2] published in November 2024, 5G subscriptions are expected to increase from 2.3 billion in 2024 to 6.3 billion by the end of 2030. Furthermore, total mobile network data traffic is forecast to grow three times, reaching 473 EB per month in 2030, of which 80% of this total will be carried over 5G. To satisfy this ever-growing data demand, Mobile Network Operators (MNOs) need to upgrade and expand their 5G infrastructures, which may require the deployment of further specialized solutions [3]. Consequently, MNOs are exploring new initiatives to reduce their costs, such as Open Radio Access Network (RAN) [4].

Unlike traditional RAN, which are black-box solutions from a limited set of vendors, Open RAN introduces a more flexible and interoperable architecture. It employs vertical and horizontal disaggregation in conjunction with virtualized and software-based components. These components communicate via well-defined open interfaces, fostering

multi-vendor interoperability, increasing market competition, and ultimately driving cost reductions. One of the key initiatives to advance Open RAN is the O-RAN Alliance [5], a consortium of network operators, vendors, researchers, and academic institutions. The O-RAN Alliance's proposed Open RAN architecture aligns with the 3GPP 7.2x functional split, which disaggregates RAN functionalities into three primary components. The Open Radio Unit (O-RU) handles PHY-low functions and Radio Frequency (RF) transmission. The Open Distributed Unit (O-DU) manages Radio Link Control (RLC), Medium Access Control (MAC), and PHY-high functions, while the Open Central Unit (O-CU) oversees Radio Resource Control (RRC), Service Data Adaption Protocol (SDAP), and Packet Data Convergence Protocol (PDCP) functions. Additionally, the RAN Intelligent Controller (RIC) [6] is a key innovation introduced by the O-RAN Alliance. It consists of software-based components designed to intelligently control and orchestrate RAN resources, leveraging AI/ML algorithms for dynamic network optimization.

On the one hand, open interfaces and the modularization of the RAN into several components foster innovation and market competition. On the other hand, it brings more complexity as open interfaces and a diverse set of components from multiple vendors bring performance, interoperability, and security challenges [7]. With respect to the evaluation of the E2E performance of 5G systems, there are several challenges for the following reasons.

- The system should be evaluated through different radio conditions, various numbers of UEs, and traffic profiles to assess real-world scenarios, which implies that testing with only a few UEs, and sometimes even within a shield box, is insufficient to evaluate diverse scenarios.
- It is important to continuously collect Quality of Service (QoS) metrics to assess how the 5G network behaves over time. In particular, gathering the E2E one-way delay can become very complicated due to the need to synchronize the clocks from the client and server hosting the application. This is important as the 5G network can behave differently depending on the direction of traffic.
- It is essential to collect radio conditions in real time, such as Signal-to-Interference-plus-Noise Ratio (SINR), Modulation and Coding Scheme (MCS), and Block Error Rate (BLER), which are important metrics to assess the reliability of the system and the impact on the achievable data rate.

Consequently, this evaluation requires specialized network testing tools. A commercial grade User Equipments Simulator (UESim) [8], which is a primary tool available in Open Testing & Integration Centres (OTIC), can be used to evaluate the E2E 5G performance. Usually, UESim solutions consist of proprietary hardware and software and are capable of emulating several UEs, with different traffic profiles, radio conditions, and mobility models. In addition, they also capture and expose QoS metrics to assess the network performance.

In this context, this paper presents the performance assessment of the 5G O-RAN multi-vendor system residing in our premises named Future Communications Translation Laboratory (FCTLab) [1] using UESim. FCTLab is open to industry and academia for research, application development, and deployment purposes. This work aims to shed light on the performance of our testbed under different traffic conditions, number of UEs, and radio conditions. We have decided to focus on downlink measurements as most of our use cases utilize mainly downlink resources.

Finally, we believe that the results shown in this paper can also be used as reference for the expected achievable performance of new deployments of 5G infrastructure with comparable radio conditions, system stack, and configuration. The remainder of this paper is organized as follows. Section 2 reviews previous studies. Section 3 unveils the technical details of our testbed. Section 4 presents the methodology and system performance evaluation. Finally, Section 5 concludes our paper.

2 Related Work

Several academic institutions have recently deployed Open RAN-based testbeds to study system performance. In [9], the authors present X5G, a private multi-vendor 5G O-RAN-compliant network that leverages NVIDIA Aerial acceleration on a GPU to execute PHY-high operations. They evaluate the system's uplink and downlink throughput using iPerf with up to four UEs. However, their study does not consider end-to-end latency, which is critical for latency-sensitive applications. The FCT O-RAN testbed [10] addresses this gap by deploying a multi-vendor, end-to-end private 5G network that integrates Radisys CU/DU, Foxconn and Benetel RUs, and ENEA/Microsoft 5G core with MEC. Validated through both digital twin simulations and real-world walk tests, showcasing the potential of O-RAN for campus-scale research and innovation.

OpenRAN@Brazil, introduced in [11], is another Open RAN testbed that comprises hardware and software from multiple vendors. The testbed uses Open5GS as an open-source core emulator and is validated using iPerf3 and Ping tools to measure throughput and round-trip time on a single UE under varying radio conditions. Nonetheless, latency

[1] https://www.sit-fctlab.org/home

evaluation is limited to low-traffic scenarios, and only ICMP-based round-trip time is considered, which may not accurately reflect application-layer delays.

In parallel, several works have used UESim for conformance and stress testing in 5G networks. The authors in [12] assess the performance of market-ready O-DU and O-CU implementations using UESim, which emulates both UEs and O-RU over the fronthaul and connects to the core emulator via N2/N3 interfaces. Meanwhile, [13] describes a software-based implementation of the High-PHY layer within the O-DU, validated using UESim to generate user and control-plane procedures.

Compared to those previous works, we differentiate ourselves from other works with these following contributions:

- We employ a cutting-edge UESim, which is used at OTIC centers, not only for conformance testing but also to perform a comprehensive E2E evaluation of a live multi-vendor 5G O-RAN-based testbed. To the best of our knowledge, this is the first research paper to do so.
- We have carried out experiments with up to 15 UEs with varying traffic load, verifying the one-way downlink latency and throughput for each scenario and how it impacts the end-user's experience. In contrast to [9], which considers only throughput for up to four UEs, and [11], which uses one UE and evaluates latency using only ICMP messages.
- We present a full-system performance perspective, correlating physical-layer metrics like SINR, MCS, and BLER with end-user application performance, while [12, 13] focus on validating individual components such as O-DU and O-CU.

3 Testbed Architecture

This section describes the technical architecture of our private 5G O-RAN-based testbed, which comprises the RAN, core, and edge components. The testbed is hosted on our premises, where we hold the spectrum license for radio transmission. It adheres to cloud-native design principles [14], including: (1) containerization for efficient resource utilization and rapid deployment, (2) automated orchestration and scaling using Kubernetes, and (3) monitoring and logging for system observability and fault detection.

The testbed is fully configurable, allowing experimentation with various O-DU and O-CU configurations and enabling access to message exchanges across standard 5G interfaces. Figure 1 illustrates the system's logical architecture. The RAN supports the 7.2x functional split and integrates multi-vendor components. The O-RUs handle PHY-low and RF functions; the O-DU manages the RLC, MAC, and PHY-high layers; and the O-CU oversees the RRC, SDAP, and PDCP layers. For indoor coverage, we deployed three Foxconn RPQN-7800 O-RUs. For outdoor coverage, two Benetel RAN650 O-RUs equipped with Alpha Wireless AW3161 antennas are used, resulting in a total of five deployed cells. The Radisys O-DU and O-CU are hosted on four HPE ProLiant DL110 Gen10 Plus Telco servers. Network synchronization is achieved through LLS-C3 mode, with a Fibrolan Falcon-RX (incorporating the Nokia AYE 475647A GNSS module) serving as the Precision Time Protocol (PTP) Grandmaster.

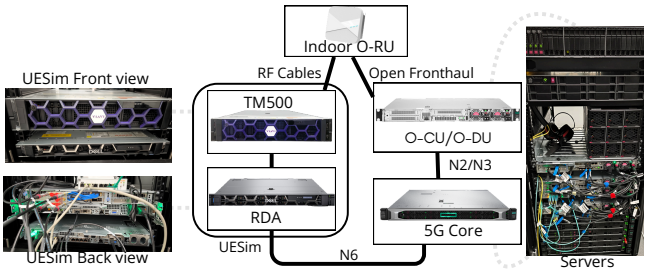


Figure 1: 5G E2E testbed overview.

The 5G Core consists of microservices from Microsoft and ENEA. Microsoft provides the Access and Mobility Management Function (AMF), Session Management Function (SMF), Authentication Server Function (AUSF), Network Repository Function (NRF), Network Slice Selection Function (NSSF), Unstructured Data Storage Function (UDSF), and User Plane Function (UPF). Two UPF instances are deployed: one acts as the Edge UPF for local breakout and processing of latency-sensitive traffic, while the Central UPF connects the testbed to the public Internet. ENEA provides the Policy Control Function (PCF), Unified Data Management (UDM), and Unified Data Repository (UDR). All 5G Core components are deployed across four HPE ProLiant DL360 Gen10 servers.

To emulate user traffic and validate system performance, we employ Viavi's UESim [15, 16], a commercial-grade solution widely adopted in OTIC centers. UESim consists of two main components:

(1) **TeraVM Real Data Applications (RDA):** This node emulates both client- and server-side applications. Each application runs on separate virtual machines and interfaces with distinct physical cards within the RDA. Using a proprietary application called Teraflow, RDA generates traffic for OSI layers 3–7 and connects to the TM500 Network Tester via Point-to-Point Protocol over Ethernet (PPPoE), as well as to the N6 interface of the 5G Core via Ethernet. A graphical user interface enables the configuration of the numbers of UE, UE mobility models, traffic profiles, radio parameters, and session durations.

(2) **TM500 Network Tester:** This hardware module interfaces with the RDA over PPPoE and connects to the O-RU via RF cables. It manages downlink and uplink data exchange between the emulated UEs and the RAN.

4 Validation

4.1 Methodology

The scope of our validation is UESim connected to one of our Foxconn RPQN-7800 O-RUs which is connected to the rest of the 5G testbed. UESim is used to generate the UEs and edge server.

The RAN configuration is illustrated in Table 1. Since both client- and server-side applications run on RDA (as stated in Section 3), it is straightforward to gather one-way latency. For the remainder of this paper, whenever indicated, UESim refers to the entire solution that consists of TM500 Network Tester and RDA. Similarly, client-side applications refer to those running on emulated UEs on RDA, while server-side refers to those running on RDA that are connected via the N6 interface of the 5G core, also known as edge applications.

Table 1: 5G RAN configuration used in our experiments.

Parameter	Value
Radio Unit	Foxconn RPQN-7800
Radio Unit Power	24 dBm per RF port
Radio Unit TRX	4T4R
Frequency Band	FR1, N78
Frequency Range	3400–3450 MHz
Center Frequency	3424.980 MHz
NR-ARFCN	627264
Bandwidth	50 MHz
Subcarrier Spacing	30 kHz
TDD Slot Pattern	DDDSU

Through the usage of Rohde & Schwarz Qualipoc Samsung S22+ [17] network tester, we have observed that the SINR inside our building ranged from 20 to 36 dB. The highest SINRs are generally obtained closer to the O-RU, whereas the lowest SINRs are observed farther from the O-RU and closer to the corners of the building. At present, UESim is unable to import real measured channel conditions into it, and consequently, we had to simulate the same SINR range gathered from Qualipoc in UESim. To do so, we have used the scenario highlighted in Figure 2, applied a channel model available for use in UESIM and the resulting SINR can be seen in the same figure. The path has a total length of 26 meters. The O-RU is in the ceiling, 3 m above the floor, and marked as waypoint B. Furthermore, we had placed waypoint A in Figure 2 to illustrate where the path begins and ends. Waypoint B is far apart 4 meters from waypoint A. Once at waypoint B, the UE is in the closest position to the O-RU, under the O-RU, and 1.5 m away from it. The main parameters configured for UESim are described in Table 2. In particular, to simulate a channel model that resembles our building, we have used the Tapped Delay Line (TDL) model, which is a primary model of the 3GPP standard and has been widely tested and validated by real-world measurements [18]. From the TDL model, we have used the TDL-A profile and a short delay spread of 10 ns, which are typically used for indoor scenarios with light obstruction [19].

Table 2: UESim Configuration Parameters.

Category	Parameter	Value
UE	MIMO DL Layers (#)	4
	Speed	0.1 m/s
Channel	Model	TDL-A
	RMS Delay Spread	10 ns
Traffic Profile	Transport Protocol	UDP
	Single-UE Data Rate	50, 250, 450, 600, 650, 700, 750 Mbps
	Multi-UE Data Rate	300 Mbps

We have carried out single-UE and multi-UE experiments. For both experiments, the scenario shown in Figure 2 is used, with the UE (s) moving at 0.1 m/s. In the single-UE validation, each experiment consists of 7 rounds, where each round starts and ends at waypoint A, and differs by the volume of data transmitted. The downlink UDP data rate the server sends to the UE for each round are 50, 250, 450, 600, 650, 700 and 750 Mbps. In total, the single-UE experiment takes 1820 s to conclude, that is, it takes 260 s to finish each round.

The multi-UE experiment consists of four rounds with a total duration of 1040 s, in which each round takes 260 s to conclude. The server is configured to send 300 Mbps UDP traffic to the UEs. The number of UEs per round is: 1, 5, 10 and 15. The server shares equally the total data rate to be transmitted to the UEs. For example, in the round with 5 UEs, the server sends data at 60 Mbps to each UE.

4.2 Results

This section presents the performance results for the testbed for single-UE and multi-UE experiments.

4.2.1 Single-UE validation. This experiment aims to evaluate the throughput and latency experienced by the UE under different data rates and radio conditions. Waypoints A and B were added to the figures in this section to identify a position

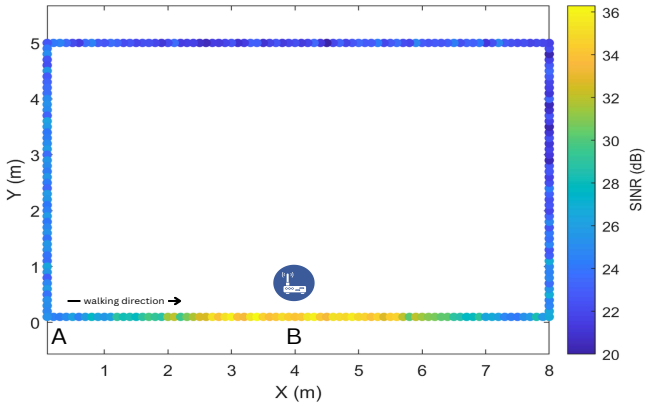


Figure 2: Scenario evaluated along with the SINR heat map obtained through UESim.

along the path traversed by the UE, and vertical green lines were also included to determine when a new round begins.

Figure 3 illustrates the average downlink throughput and average downlink latency per packet over time measured at the client-side application (i.e. UE) for different amounts of data sent by the server-side application. The first green vertical line denotes the instant that the data rate generated by the server starts at 50 Mbps, while the second green line represents the instant the server transmission rises from 50 to 250 Mbps, and so forth. As can be seen in Figure 3, when the server transmits 50 Mbps of data, the average UE's throughput is 46 Mbps and the latency is 7 ms. For the 250 Mbps round, the UE's throughput is on average 236 Mbps and the latency is 7 ms. As the volume of data is considerably lower compared to the cell capacity, which is 711 Mbps, the RAN is able to transport the data with low latency, even during SINR degradation periods, albeit with some throughput reduction. For 50 and 250 Mbps rounds, over 99.6% of the recorded latencies are below 15 ms.

For the rounds where the server generates data at 450 Mbps or higher, the latency grows and the instantaneous throughput decreases considerably once the UE is farther from the O-RU mainly due to (1) the SINR degradation, which reduces the MCS and thus the data block used for transmission, and (2) more data enqueued in the gNB's buffer. In the 450 Mbps round, the average throughput and latency experienced by the UE as well as the proportion of latencies measurements below 15 ms are: 422 Mbps, 20 ms, and 62%; while for the 600 Mbps round, they respectively are: 509 Mbps, 68 ms, and 26%. Regarding the 650 Mbps round, the UE's average throughput is 530 Mbps, whereas the average latency and proportion of latency instances below 15 ms are 164 ms and 23%. For the 700 Mbps round, these values are 546 Mbps, 167 ms, and 20%, respectively.

As seen in the 750 Mbps round, this amount of traffic exceeds the cell downlink capacity, which is 711 Mbps. Consequently, as depicted in Figure 3, the latency of the packets is very high since the gNB's buffer are continuously saturated even during excellent radio conditions, causing all the latencies measurements to be over 150 ms, reaching up to 644 ms during lower SINR values. The average throughput and latency are 544 Mbps and 371 ms. Figures 4, 5, and 6 examine the 750 Mbps round in more detail. At the beginning of the round, the UE is closer to the O-RU, undergoing a higher SINR and MCS, lower Downlink Shared Channel Block Error Rate (DL-SCH BLER) and therefore higher throughput. As the UE moves farther from the O-RU, the SINR decreases, the DL-SCH BLER increases to reach 13%, and thus the MCS is reduced. In particular, the throughput, which once hit 711 Mbps and had a latency of 178 ms with an SINR of 36 dB (at instant 66 s), was reduced by 52%, dropping to 338 Mbps, and the latency increased by 3.5 times, reaching 619 ms, with an SINR of 21 dB at instant 116 s. As indicated in Figure 5, 25 is the maximum MCS achievable in the downlink in our testbed.

4.2.2 Multi-UE validation. Unlike the experiment carried out in Section 4.2.1, the experiment in this section aims to verify how the growth in the number of UEs affects the QoS in terms of throughput and latency under different radio conditions. The experiment consists of four rounds, where a different number of UEs is used in each round, starting from 1, then 5, 10 and finally 15. The same scenario as in Figure 2 is used. For all rounds, the UE (s) start at waypoint A and move until they reach A. For the rounds consisting of more than 1 UE, they start at waypoint A at the same time and traverse together until reaching A again, sharing similar radio conditions. Once the first round concludes at instant 260 s, a new round starts with 5 UEs, and thus the server splits the 300 Mbps to 5 UEs, transmitting about 60 Mbps towards each UE. The figures presenting the experimental results include waypoints A and B to pinpoint a position along the path traversed by the UE (s) and, to facilitate the visualization, the results were consolidated into four different lines with duration ranging from 0 to 260 s.

Figures 7 and 8 illustrate the average downlink throughput and average downlink latency per packet over time for the multiple UEs experiment. As can be observed, for all rounds, the throughput follows a similar behavior, increasing when the UEs are closer to the O-RU and dropping when they are farther from the O-RU. For the round with 1 UE, the average throughput is 284 Mbps, while the average latency per packet and the proportion of latency values below 15 ms are 7 ms and 100%. For the round with 5 UEs, those values are 277 Mbps, 7 ms and 100%. In particular, for the scenario with more than one UE, as in this case, the throughput means that

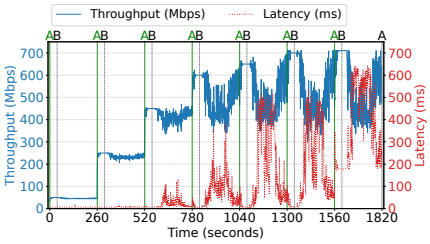


Figure 3: Downlink throughput and one-way latency over time for different data rates (50 to 750 Mbps).

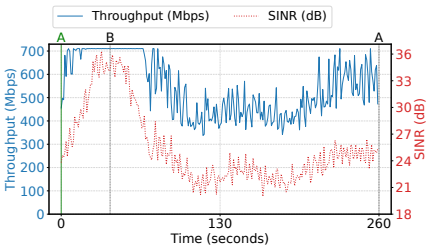


Figure 4: Throughput vs SINR.

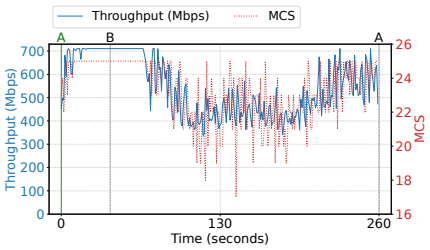


Figure 5: Throughput vs MCS.

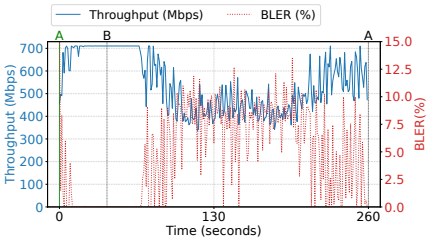


Figure 6: Throughput vs BLER.

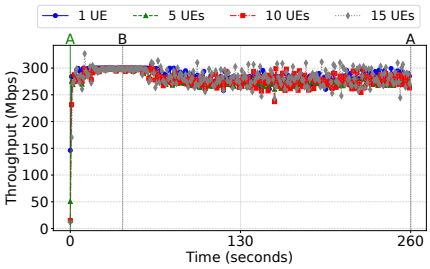


Figure 7: Downlink throughput over time for different number of UEs.

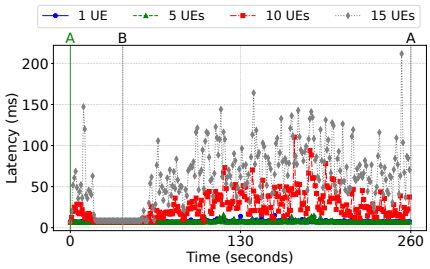


Figure 8: Downlink latency over time for different number of UEs.

on average the sum of the client-side applications throughput is 277 Mbps. Regarding the round with 10 UEs, the average throughput is 278 Mbps, the latency is 26 ms, and the share of latency values below 15 ms is 28%. For the last round with 15 UEs, 16% of the latency values are below 15 ms, while the throughput is 283 Mbps and the latency is 62 ms. As can be seen, the average throughput is relatively stable with the growth of the number of UEs, changing on the basis of the SINR value. However, except when the UEs are positioned near the O-RU, the latency per packet increases substantially as the number of UEs grows, even though the network is still transmitting the same amount of data at a rate of 300 Mbps.

5 Conclusion

This paper presented our private 5G O-RAN-based testbed and its evaluation on varying numbers of UEs, radio conditions, and data loads. We validated the testbed using a cutting-edge UESim with up to 15 UEs. The results show that for the single-UE scenario, a traffic volume of up to 250 Mbps with SINR ranging from 20 to 36 dB slightly affects the throughput and latency experienced by the UE. However, higher volumes considerably influence the QoS offered to users, such as the average latency per packet grew from 7 to 167 ms when the volume of data increased from 50 to 700 Mbps. Moreover, for the multi-UE experiment, the average throughput is fairly stable regardless of the number of UEs. However, the latency increases fourteen times when the number of UEs changes from 5 to 15. Furthermore, while 100% of the packets have a latency below 15 ms for 5 UEs; for 15 UEs this share is equal to 16%. Future works include evaluating the testbed performance with other types of protocols, such as TCP and QUIC, as well as other combinations of O-RU, O-DU and O-CU. In addition, future studies that benchmark between traditional and O-RAN-based 5G networks are needed to identify the eventual performance limitations of multi-vendor 5G O-RAN systems.

6 Acknowledgments

This research is supported by the National Research Foundation, Singapore and Infocomm Media Development Authority under its Communications and Connectivity Bridging Funding Initiative. Any opinions, findings and conclusions or recommendations expressed in this material are those of the author(s) and do not reflect the views of National Research Foundation, Singapore.

References

[1] Akhil Gupta and Rakesh Kumar Jha. A survey of 5G network: Architecture and emerging technologies. *IEEE access*, 3:1206–1232, 2015.

[2] Ericsson Mobility Report. [online] Available: https://www.ericsson.com/4adb7e/assets/local/reports-papers/mobility-report/documents/2024/ericsson-mobility-report-november-2024.pdf, November 2024.

[Accessed 07-07-2025].

[3] William Lehr, Fabian Queder, and Justus Haucap. 5G: A new future for Mobile Network Operators, or not? *Telecommunications Policy*, 45(3):102086, 2021.

[4] Michele Polese, Leonardo Bonati, Salvatore D'oro, Stefano Basagni, and Tommaso Melodia. Understanding O-RAN: Architecture, interfaces, algorithms, security, and research challenges. *IEEE Communications Surveys & Tutorials*, 25(2):1376–1411, 2023.

[5] O-RAN ALLIANCE. [online] Available: https://www.o-ran.org/, 2025. [Accessed 07-07-2025].

[6] Bharath Balasubramanian, E Scott Daniels, Matti Hiltunen, Rittwik Jana, Kaustubh Joshi, Rajarajan Sivaraj, Tuyen X Tran, and Chengwei Wang. RIC: A RAN intelligent controller platform for AI-enabled cellular networks. *IEEE Internet Computing*, 25(2):7–17, 2021.

[7] Gabriele Gemmi, Michele Polese, Pedram Johari, Stefano Maxenti, Michael Seltser, and Tommaso Melodia. Open6G OTIC: A Blueprint for Programmable O-RAN and 3GPP Testing Infrastructure. In *2024 IEEE 100th Vehicular Technology Conference (VTC2024-Fall)*, pages 1–5. IEEE, 2024.

[8] Ali Parichehreh, Umberto Spagnolini, Paolo Marini, and Alberto Fontana. Load-Stress Test of Massive Handovers for LTE Two-Hop Architecture in High-Speed Trains. *IEEE Communications Magazine*, 55(3):170–177, 2017.

[9] Davide Villa, Imran Khan, Florian Kaltenberger, Nicholas Hedberg, Rúben Soares Da Silva, Anupa Kelkar, Chris Dick, Stefano Basagni, Josep M Jornet, Tommaso Melodia, et al. An open, programmable, multi-vendor 5G O-RAN testbed with NVIDIA ARC and OpenAirInterface. In *IEEE INFOCOM 2024-IEEE Conference on Computer Communications Workshops (INFOCOM WKSHPS)*, pages 1–6. IEEE, 2024.

[10] Amogh P C, Nagamuthu Vignesh, Pei Yiyang, Neelakantam Venkatarayalu, Pedro Henrique Amorim Rezende, Shyam Babu Mahato, and Sumei Sun. FCT O-RAN: Design and Deployment of a Multi-Vendor End-to-End Private 5G Testbed. In *IEEE INFOCOM 2025 - IEEE Conference on Computer Communications Workshops (INFOCOM WKSHPS)*, pages 1–6, 2025.

[11] Weskley Mauricio, Francisco Hugo Costa Neto, Maykon Silva, Rodrigo Kenji Yaly Aoki, Eduardo Ebeling de Almeida, Eduardo Melão, Sérgio Barros, Daniel Lazkani Feferman, and Fuad M Abinader Jr. Performance Analysis of Outdoor Open RAN Deployments in Long-Haul 5G Networks using the OpenRAN@ Brasil Testbed. In *Proceedings of the 30th Annual International Conference on Mobile Computing and Networking*, pages 1850–1857, 2024.

[12] Tuan V Ngo, Mao V Ngo, Binbin Chen, Gabriele Gemmi, Eduardo Baena, Michele Polese, Tommaso Melodia, William Chien, and Tony Quek. Consistent and Repeatable Testing of O-RAN Distributed Unit (O-DU) across Continents. In *2024 IEEE 100th Vehicular Technology Conference (VTC2024-Fall)*, pages 1–5. IEEE, 2024.

[13] Seung Nam Choi and Nam Il Kim. Software implementation of O-RAN fronthaul interface. In *2023 28th Asia Pacific Conference on Communications (APCC)*, pages 53–56. IEEE, 2023.

[14] Dennis Gannon, Roger Barga, and Neel Sundaresan. Cloud-native applications. *IEEE Cloud Computing*, 4(5):16–21, 2017.

[15] TM500 Network Tester. [online] Available: http://viavisolutions.com/en-us/products/tm500-network-tester, 2025. [Accessed 07-07-2025].

[16] TeraVM. [online] Available: https://www.viavisolutions.com/en-us/products/teravm, 2025. [Accessed 07-07-2025].

[17] QualiPoc. [online] Available: https://www.rohde-schwarz.com, November 2024. [Accessed 07-07-2025].

[18] Juan Diego Belesaca, Andres Vazquez-Rodas, Luis F. Urquiza-Aguiar, and J. David Vega-Sánchez. An in-depth assessment of the physical layer performance in the proposed b5g framework. *Ad Hoc Networks*, 164:103609, 2024.

[19] K 3GPP. Study on channel model for frequencies from 0.5 to 100 GHz. *3rd Generation Partnership Project (3GPP), Technical Specification (TS) 38.901*, 2018.

Distributed AI Platform for the 6G RAN

Ganesh Ananthanarayanan, Matthew Balkwill, Xenofon Foukas, Zhihua Lai, Bozidar Radunovic, Connor Settle, Yongguang Zhang
{ga,Matthew.Balkwill,xefouk,zhihualai,bozidar,connorsettle,ygz}@microsoft.com
Microsoft Research

Abstract

Cellular Radio Access Networks (RANs) are rapidly evolving towards 6G, driven by the need to reduce costs and introduce new revenue streams for operators and enterprises. In this context, AI emerges as a key enabler in solving complex RAN problems spanning both the management and application domains. Unfortunately, and despite the undeniable promise of AI, several practical challenges still remain, hindering the widespread adoption of AI applications in the RAN space. In this work, we attempt to shed light to these challenges and argue that existing approaches in addressing them are inadequate for realizing the vision of a truly AI-native 6G network. We propose a distributed AI platform architecture, tailored to the needs of an AI-native RAN.

CCS Concepts

• **Networks** → **Network architectures**; **Wireless access points, base stations and infrastructure**; **Mobile networks**.

Keywords

Open RAN, Programmable RAN, distributed AI RAN, AI RAN platform

ACM Reference Format:
Ganesh Ananthanarayanan, Matthew Balkwill, Xenofon Foukas, Zhihua Lai, Bozidar Radunovic, Connor Settle, Yongguang Zhang. 2025. Distributed AI Platform for the 6G RAN. In *2nd ACM Workshop on Open and AI RAN (OpenRan '25), November 4–8, 2025, Hong Kong, China.* ACM, New York, NY, USA, 7 pages. https://doi.org/10.1145/3737900.3770167

1 Introduction

In 5G, RAN is undergoing a paradigm shift, disaggregating monolithic base stations into separate components (CU, DU, and RU) and containerizing them on commodity hardware/-software platforms across edges and cloud (Fig. 1). This enables open interfaces for RIC-based applications, accelerating innovation and facilitating new capacity-boosting technologies like Massive MIMO, mmWaves, and densification. 6G will further address long-standing problems (e.g., radio resource management, mobility, energy savings) and introduce transformative uses (e.g., joint communication-sensing, security, slicing).

The AI revolution, as catalyzed by the current generative AI phenomenon, has motivated the telecom industry to see AI as an ideal fit for RAN problems like signal classification, traffic prediction, and optimal scheduling. The next-gen mobile networks, including 6G, will be "AI-native". AI-RAN Alliance [1], a telecom industry's AI initiative, has defined three promising domains of AI use cases in the RAN:

(1) AI-for-RAN – use AI to optimize RAN performance. For example, higher frequencies, cell density, and larger antenna arrays will increase the search space in scheduling with AI-based solutions [4]. Infrastructure management like predictive maintenance of RAN sites, vRAN troubleshooting [25], and energy savings [12, 14, 22] will all benefit from AI.

(2) AI-and-RAN – share compute (CPU/GPU) between RAN and AI apps. The vRAN is largely underutilized (< 50%) [20] due to overprovisioning and traffic inbalance [10], but AI can be leveraged to achieve sharing without violating the RAN's realtimeness [10, 11, 21].

(3) AI-on-RAN – use RAN infrastructure for AI apps. For example, localization from channel data for tracking/autonomy [26], context-aware security [5], and latency-critical video analytics [27] will benefit from interfacing with RAN.

Despite these promises, challenges persist. First, we need to collect data from vastly and geo-distributed RAN sites for accurate AI models. Second, AI-RAN use cases vary in compute, latency, and privacy needs, complicating edge-cloud orchestration. The goal of this research is to design a distributed AI platform for 6G RAN. We highlight the issues, arguing static approaches fall short for AI-native networks. We propose a distributed AI architecture to address these problems.

2 Background

2.1 Canonical application examples

To help with highlighting the need for a distributed AI RAN platform, we define two examples (Fig. 2), which we use as a reference for the remainder of this work.

RAN slicing scheduler – A scheduler allocates radio resources across network slices (inter-slice scheduling) and across users of the same slice (intra-slice scheduling). Both

OpenRan '25, Hong Kong, China

ACM ISBN 979-8-4007-1977-6/2025/11
https://doi.org/10.1145/3737900.3770167

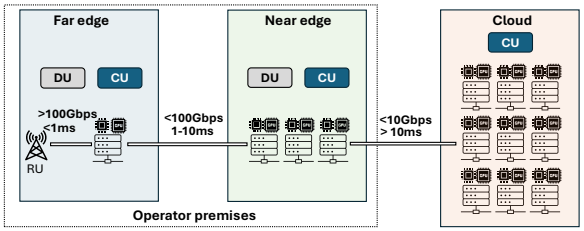


Figure 1: High-level 5G RAN overview. The RAN infrastructure capabilities can vary depending on the location.

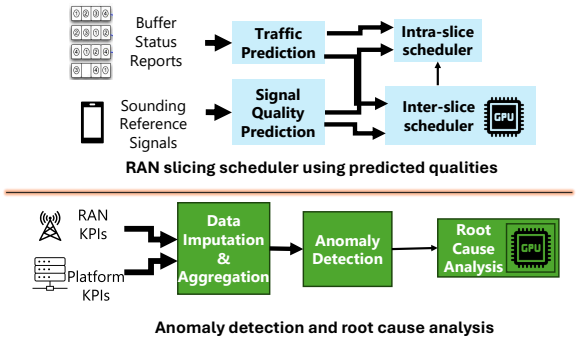


Figure 2: Graphs of AI/ML models for RAN applications

schedulers use the traffic demand information provided by the phones through buffer status reports, to try and predict the traffic load for the upcoming period [3, 9]. They also use physical layer sounding reference signals to predict the users' signal quality. The inter-slice scheduler makes scheduling decisions at coarse granularities (e.g., seconds) and feeds its decisions to the real-time intra-slice scheduler. This example illustrates both AI-on-RAN (predicting load using buffer status reports) as well as AI-for-RAN (scheduling decisions).

Anomaly detection and root cause analysis – A service management and orchestration framework tries to detect RAN-related anomalies and identify their root cause for potential mitigations towards AI-for-RAN. Since the anomalies can originate both from the RAN and the platform, the application collects data from both sources [25]. It also applies imputation and aggregation techniques, to deal with noisy/missing data and to reduce their volume. It then processes the collected statistics, to detect if an anomaly is present, and if so, it attempts to localize it and detect its root cause.

From these examples, we see that a typical AI RAN application, rather than being a monolith, may consist of a sequence of blocks. Each block can do its own data pre-processing and can include an ML model. Those blocks are chained together into a graph, where the output of one model becomes the input to the next. Typically the traffic volume between components reduces from left to right, as more data gets processed and aggregated.

2.2 Distributed edge infrastructure

The RAN infrastructure is typically deployed across a distribution of far and near edges, extending to the cloud (Fig.1). As we move from the far edge to the cloud, more compute resources become available for the AI RAN applications. However, choosing the cloud for deployment (e.g., due to the presence of GPUs) is not always the right choice. The abundance of resources comes at the expense of reduced bandwidth and higher latency. This can be crucial factors for applications such as the example anomaly detector, which requires large volumes of input data, or the RAN slicing scheduler, which is latency-sensitive in its allocation decisions.

3 Challenges to build AI RAN applications

We now discuss the challenges of deploying AI RAN applications, that make the case for a distributed AI RAN platform.

3.1 Data collection

Different applications require different feature sets, that might have heterogeneous characteristics in terms of type, time granularity, etc. For example, the radio resource scheduling application might require real-time data from the RAN network functions, while the anomaly detection might need to combine aggregate data from both the RAN and the platform in a time windows of seconds. Similarly, the anomaly detection application might rely on the inter-arrival time between IQ samples, while the scheduler might require the actual raw IQ samples carrying the sounding reference signals.

The heterogeneity in the input features of AI applications is what makes the data collection process challenging. Exposing raw data from all possible data sources is not a viable option, as it would lead to a huge volume of data that one would have to process, store and transmit. For example, capturing all the raw IQ samples from the physical layer of the RAN translates into several gigabits of traffic per second, even for a single base station with four antennas. Similarly, capturing all the CPU scheduling events of a server, in order to detect if a platform anomaly due to CPU interference is present, would result in the collection of millions of events per second.

The current standard practice to bypass this roadblock is to expose a set of coarse-grained data sources, that are applicable to a wide range of use cases. These data sources are exposed through static APIs specified by standardization bodies like 3GPP and O-RAN. For instance, O-RAN defines the E2 interface for the collection of pre-defined RAN KPIs, and the O2 interface for the collection of platform data.Any change to a data source requires the standardization body consensus, which can be a very long process. As such, AI RAN applications end up being developed as an afterthought, subject to the available data sources, rather than in a truly AI-native way, where the data collection is application-driven.

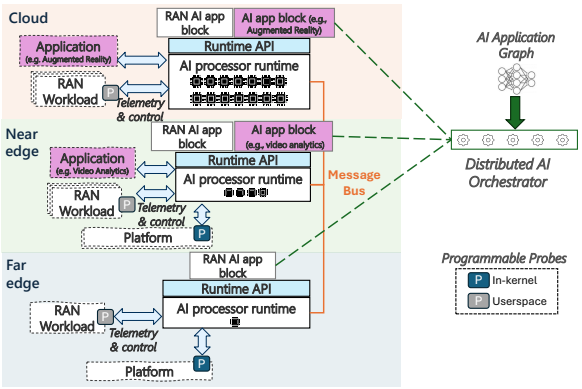


Figure 3: Distributed AI platform architecture.

3.2 RAN AI application orchestration

The disaggregated nature of the RAN means that its network functions might be deployed across the edges and the cloud. This raises a fundamental question as to what should be the location in which the blocks of AI applications should reside. Answering this question is far from straightforward. It involves matching the applications' requirements to the capabilities of the infrastructure, which can vary in terms of compute resources, network bandwidth, latency (Fig. 1) and privacy.

The many constraints and trade-offs make the deployment of AI RAN applications a major challenge. First, a developer needs to understand the underlying constraints which may differ from one deployment to another. Second, they need to carefully choose what data to collect and where to place the application blocks, to maintain high accuracy, while respecting the constraints. Third, deployed applications will naturally have competing requests for the same resources, meaning that it falls on the developer to design their application with placement flexibility in mind. Finally, AI RAN applications have to coexist with other AI applications that also execute on the edge, such as video analytics and self-driving cars [27]. In the absence of a structured framework, AI RAN applications have to be developed with ad hoc programming interfaces and manually distributed by developers.

4 Distributed AI platform for RAN

We present our vision for distributed AI-native RAN platform, with the architecture illustrated in Fig. 3. It builds on top of three components; i) programmable probes for the flexible collection of data, ii) AI processor runtimes for the deployment of AI applications across the distributed compute fabric, and iii) an orchestrator for the coordination of the platform.

Programmable probes for dynamic data collection – To allow developers to define optimal feature sets for their AI applications, we propose the use of dynamic probes for the platform (i.e., OS kernel) and the userspace (i.e., RAN network functions) to programmatically collect data. Through the probes, a developer could write small pieces of code to access raw events and data structures and summarize them in a custom way. This would expose the right features for training and inference, while minimizing the volume of generated data. For example, a developer could leverage a probe to access the raw IQ samples of the base station to feed them directly to the application, like in the example of the RAN slicing scheduler. Alternatively, they could pre-process them and export a derived KPI, like in the case of the anomaly detection application. The same approach could also be used to capture platform data (e.g., capture the incoming TCP packets and calculate the average inter-packet delay).

While this approach provides flexibility in tailoring the data collection process to the AI application's requirements, it also introduces safety concerns (e.g., illegal memory accesses). For this, we propose to use probes based on the eBPF technology, which has recently caught the attention of the telco industry [8, 23]. eBPF allows the injection of code to instrumentation points, subject to a static verification process that guarantees the safety of the injected code. While eBPF originates in the Linux kernel, recent advances [8] have expanded its use to userspace RAN applications.

AI processor runtime – Considering the constraints and trade-offs that characterize the various locations of the infrastructure, an AI-native platform should allow the seamless deployment of AI applications to the most suitable location, without the developer having to worry about providing location-specific flavors. In the proposed architecture, this is achieved through the *AI processor runtime*. The runtime can be deployed at any location where AI applications are expected to run and introduces an API for its interactions with the applications. The runtime should provide the following functionalities:

– *Data ingestion and control:* It should enable the ingestion of data captured through the local programmable probes and their exposure to applications. It should also enable the issuing of API calls towards the RAN functions and the platform for closed loop control.

– *Data exchange:* It should allow the exchange of data streams among the AI application blocks deployed in different locations through a common message bus. Similarly, it should enable the issuing of RPCs for applying control decisions.

– *Execution environment:* It should implement the process of running inference or training tasks, by abstracting the underlying compute resources (e.g., CPUs and GPUs) and exposing them to both RAN and non-RAN AI applications.

– *Life-cycle management:* It should provide a standard interface to deploy, update, and remove AI applications, as well as to monitor their performance and resource utilization.

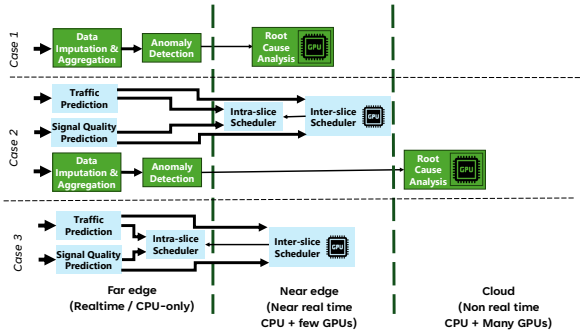


Figure 4: Distributed deployment scenarios for orchestrated AI applications.

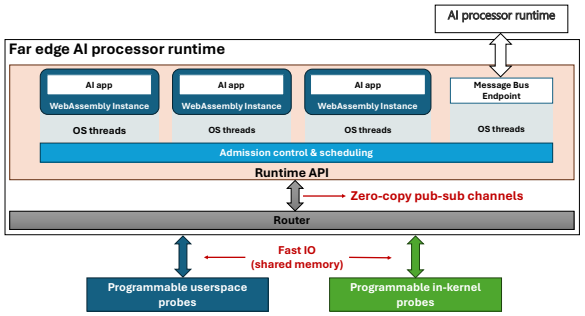


Figure 5: Architecture of far edge AI processor runtime.

As long as it provides the functionalities described above, we eschew making an implementation prescription because we believe that the framework should be extensible to include existing and future runtime environments and messaging technologies, thus not restraining AI developers. As an example, one could consider using Docker containers combined with a WebAssembly (WASM) runtime as a highly portable and sandboxed execution environment, while the message bus implementation could be based on REST or gRPC calls.

Orchestrator – The distributed AI framework is overseen by an orchestrator (Fig 3) that is responsible for the placement of the AI applications across the AI processor runtimes. The orchestrator allows for dynamic addition or removal of applications, and also handles dynamic changes to the available resources (compute and network). It also allows to plug in diverse policies that trade off between compute at the different edges and the network load between the edges.

Applications are exposed to the orchestrator in the form of blocks in a graph. The blocks are characterized compute, memory, network and latency requirements, as well as other constraints (e.g., the locations where the block is not allowed to run due to privacy concerns). The orchestrator takes into account both the developer requirements and the capabilities of the infrastructure and places the blocks to the AI processor runtimes that maximize the overall utility of the platform. Unique to AI workloads are *inference parameters* that influence resource demand [27], and hence is a resource allocation knob for the orchestrator. Examples of inference parameters would be data sampling rates, or AI models with varying accuracy for anomaly detection. We propose the AI RAN applications to expose their parameters to the orchestrator, and design the orchestrator to dynamically change them.

We use the example applications of Fig. 2 to explain the operation of the orchestrator. We consider a far-edge location without a GPU and a limited amount of CPUs, a near-edge location with a single GPU, and a cloud location with many GPUs. Under this setup, we consider the three cases of Fig. 4. In the first case, we install the anomaly detection application. Given that the root cause analysis model requires a GPU, the orchestrator places it at the near edge and the rest of the applications' blocks at the far edge, minimizing the amount of outgoing traffic. In the second case, we install the RAN slicing scheduler application in addition to the anomaly detector. The latency-sensitive inter-slice scheduler requires a GPU, however the one available at the near edge is already reserved. Therefore, the orchestrator migrates the root cause analysis model to the cloud, since it is not latency sensitive, and deploys the inter-slice scheduler to the near edge. The latency-tolerant intra-slice scheduler is also deployed at the near edge, since there is not enough CPU capacity at the (preferable) far edge location. Finally, in the third case, we illustrate how the orchestrator handles the completion of the anomaly detector and reschedules the radio resource scheduler. With the anomaly detector removed, CPU resources are freed up at the far edge, so the orchestrator migrates the intra-slice scheduler there, reducing the traffic sent to the near edge.

5 An efficient far edge AI processor runtime

We advocate that a highly optimized far edge runtime is required to host real-time AI applications (e.g. the RAN slicing scheduler), as has been demonstrated in recent research works (e.g., [13]), and as envisaged in the O-RAN concept of dApps [6]. As such, it needs to have sub-millisecond reaction times. Furthermore, the far edge is resource constrained, with only a small fraction of CPUs available for AI applications, and with a small or no GPU present. Our proposed design is illustrated in Fig. 5 and has the following characteristics:

Tight integration with the probes – The AI processor runtime communicates with the probes through fast IO channels using shared memory and a zero-copy mechanism. This allows the passing of data between the probes and the AI applications with very low compute and latency overhead.

Efficient, flexible and secure execution environment – We have experimented with the use of WASM as an execution environment and observed its benefits. It enables the sandboxing of applications with a very small overhead, while maintaining near-native runtime performance. The interaction with the

rest of the system for the acceleration of inference or the data collection can take place through API calls exposed by the WASM Interface (WASI). For example, wasi-nn could be used for the exposure of the CPUs and the GPUs (when available).

Admission control & real-time resource allocation – All AI applications deployed on the processor runtime need to request a fraction of the CPU time or the GPU area. An admission control process ensures that this request can be met, while also serving other applications. Once deployed, an appropriate schedule is imposed by the runtime (for example, for the CPU we leverage the Linux deadline scheduler).

6 Implementation & Evaluation

To demonstrate the feasibility and benefits of our proposed architecture, we implemented a reference far edge AI processor runtime, based on the design outlined in Section 5, and also prototyped the distributed AI orchestrator.

6.1 Far edge AI processor runtime

Our reference AI processor runtime is written in ~5K lines of C/C++ code and is publicly available at [16]. The runtime provides both a C++ and a Python API for the development of applications, to cater the needs of developers requiring either high performance (C++) or quick integration with popular AI frameworks and libraries (Python). It should be noted that the current reference implementation does not support WASM for deploying applications. The applications can instead be loaded in the form of shared libraries (.so), dynamically linked at runtime through a REST-based API. The extension to WASM applications is left as future work.

For the programmable in-kernel probes, we leveraged the popular eBPF technology that is part of the Linux kernel [7], while for the userspace probes, we leveraged the jbpf userspace eBPF framework [8, 15], which has been designed with lightweight RAN instrumentation in mind. The integration of the probes with the AI processor runtime is done using zero-copy shared memory channels that are part of the jbpf framework.

To evaluate the efficiency of our far edge AI processor runtime under realistic conditions, we instrumented the popular open source RAN stack of srsRAN [24] with hooks that allow us to capture several important events across all the layers of the RAN (e.g., fronthaul packets, MAC scheduling events, PDCP packets, etc.), and to send back control decisions [17]. We deployed our custom stack and the AI processor runtime on an enterprise scale private 5G testbed [2], and we measured the latency of delivering data between the programmable probes of srsRAN and the AI processor runtime. As illustrated in the violin plots of Figure 6, in a well tuned system, the communication latency overhead in both the monitoring and control directions remains below 16μs, demonstrating the feasibility of our proposed AI processor runtime design.

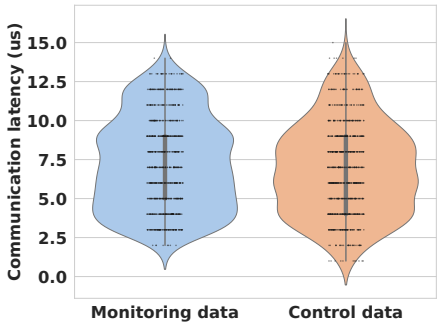


Figure 6: Communication latency overhead for monitoring and control data to/from the AI processor runtime.

6.2 Distributed AI orchestrator

To demonstrate the benefits of distributed AI orchestration, we developed a proof of concept orchestrator. The orchestrator is capable of dynamically orchestrating container-based applications across the edge and the cloud, taking into account their GPU and network requirements.

For our evaluation, we consider a setup composed of a far edge location with a single HPE DL110 Linux server equipped with an Nvidia L4 GPU, with 24GB of memory, and an Azure cloud VM equipped with an Nvidia A100 GPU with 40GB of memory. As sample workloads we use the SpotLight [25] AI-based RAN anomlay detection and the visual SLAM of the Nvidia Isaac ROS framework [19]. The two applications were chosen as representative examples of AI-for-RAN and AI-on-RAN scenarios, and they both require a GPU.

By profiling the applications, we observe that SpotLight requires approximately 4% of the edge GPU and 500Kbps of network bandwidth for performing anomaly detection for a single 5G cell, when using the dataset provided by [25]. In the case of the vSLAM, a single instance requires 8GB of GPU memory and ~40Mbps of network bandwidth for an RGB-D camera configured with a resolution of 1280×720 at 15 FPS.

As illustrated in Figure 7, we deploy SpotLight across 10 cells (case 1). The AI orchestrator decides to orchestrate all the SpotLight instances at the edge location, considering that the resources of the L4 GPU are adequate (utilization of ~40%) and this deployment choice requires shipping no data to the cloud. Then, we request to deploy three instances of the visual SLAM application. The remaining edge GPU capacity is not adequate to accommodate the new workload, which requires 24GB of memory. As such, the AI orchestrator migrates the SpotLight applications to the cloud and deploys the visual SLAM applications to the edge (case 2). The reason behind this deployment choice of the orchestrator is that it maximizes the utilization of the edge GPU (100% utilization), while minimizing the amount of data that has to be shipped to the cloud. An alternative sub-optimal deployment option would be

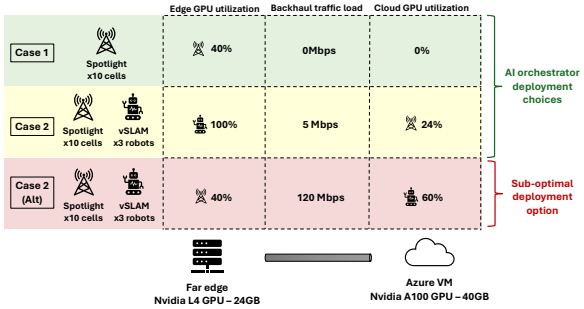


Figure 7: AI orchestrator deployment choices vs sub-optimal option.

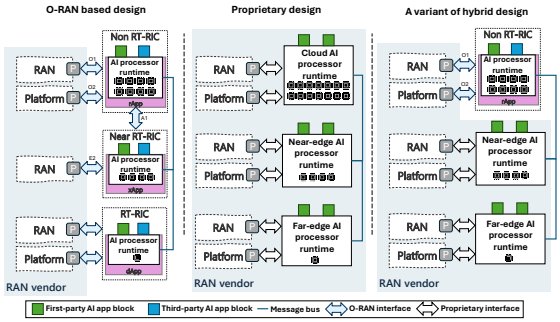


Figure 8: Deployment options for integration of distributed AI platform with the RAN.

to deploy the visual SLAM to the cloud and keep SpotLight to the edge, however, this would lead to a requirement of shipping 120Mbps worth of data to the cloud (Case 2 - Alt). Along the lines of the discussion of Section 4, the orchestrator could also be tailored to take into account other optimization parameters, such as the latency or the deployment cost.

7 Open vs Closed architecture and interfaces for integration with RAN

Despite the progress within the O-RAN community, many fundamental tensions remain around control interfaces. Several major vendors do not support the idea of a near-RT RIC [18], expressing concerns regarding the network stability and security. If RAN vendors allow developers to exert fine-grained control on the RAN behavior, these may clash with the proprietary algorithms vendors have implemented. Also, some vendors argue that exposing sensitive RAN data may affect the security of the network and the data privacy. Vendors are also reluctant to open up some of their interfaces, as this may reduce their competitive advantage. Our view is that the distributed AI platform proposed in this work should be a building block that can be customized appropriately for different use cases. We briefly discuss some examples of deployment options (Fig. 4).

O-RAN based design – One extreme is an O-RAN based design (Fig. 8, left), allowing any first or third-party vendor to deploy AI applications across the RAN infrastructure. One could leverage the O-RAN RICs and integrate the AI processor runtimes in them as apps (i.e., rApps, xApps, dApps). For the local data collection and control operations, the AI processor runtimes could leverage the open RIC interfaces (e.g., E2, O1), augmented with programmable probes and exposed to the AI runtimes through appropriate adapters. The communication between the AI processor runtimes could be facilitated by a message bus, which could be standardized and realized as an overlay network on top of the RIC fabric. For the far-edge, and considering that there is currently no real-time RIC specification, one could leverage our far-edge AI processor runtime design as a blueprint.

Proprietary design – Another extreme is a design, fully controlled by the RAN vendor (Fig. 8, middle). In this case, the group that is in charge of the RAN product development could also be in charge of developing and managing the AI platform and of defining proprietary interfaces for data collection and control. A different group could deploy and manage their own AI RAN applications, without having to constantly go through the RAN product development group, to ask them to introduce new features or expose more data. By decoupling the responsibilities, the vendor can accelerate its innovation, while still maintaining a full control over the RAN behavior.

Hybrid design – It is also possible to create a hybrid (Fig. 8, right), that would be a mix of proprietary AI processor runtimes and runtimes running on top of standard O-RAN RICs. AI applications could be composed of a mixture of first-party and third-party AI blocks, where the proprietary data and control interfaces of the RAN functions can only be accessed by the first-party AI blocks, whereas third-party blocks can only access APIs exposed by the standard interfaces. This would allow the RAN vendor to maintain control over the RAN behavior, while still allowing third-party developers to innovate.

8 Conclusions

To realize the 6G vision, we argued about the benefits of introducing AI in the RAN, from the management and infrastructure up to the application layer. Based on these observations, we outlined the challenges for deploying AI solutions, stemming from the requirements of the applications and the characteristics of the RAN infrastructure. Motivated by these challenges, we proposed a distributed AI platform architecture. Its goal is to alleviate the common painpoints of deploying AI applications in the RAN without being prescriptive about the implementation details. By identifying the future requirements and by initiating a discussion on the architecture of a 6G AI platform will help both the standards and the vendors to create opportunities for introducing AI solutions in 6G.

References

[1] AI-RAN Alliance. Aug 2024. *Integrating AI/ML in Open-RAN: Overcoming Challenges and Seizing Opportunities*. Technical Report. AI-RAN Alliance.

[2] Paramvir Bahl, Matthew Balkwill, Xenofon Foukas, Anuj Kalia, Daehyeok Kim, Manikanta Kotaru, Zhihua Lai, Sanjeev Mehrotra, Bozidar Radunovic, Stefan Saroiu, et al. 2023. Accelerating Open RAN Research Through an Enterprise-scale 5G Testbed. In *Proceedings of the 29th Annual International Conference on Mobile Computing and Networking*. 1–3.

[3] Arjun Balasingam et al. 2024. Application-Level Service Assurance with 5G RAN Slicing. In *USENIX NSDI*.

[4] Ioannis A Bartsiokas et al. 2022. ML-based radio resource management in 5G and beyond networks: A survey. *IEEE Access* 10 (2022).

[5] Arsenia Chorti et al. 2022. Context-aware security for 6G wireless: The role of physical layer security. *IEEE Communications Standards Magazine* 6, 1 (2022), 102–108.

[6] Salvatore D'Oro et al. 2022. dApps: Distributed applications for real-time inference and control in O-RAN. *IEEE Communications* (2022).

[7] eBPF. [n. d.]. Dynamically program the kernel for efficient networking, observability, tracing, and security. https://ebpf.io/.

[8] Xenofon Foukas et al. 2023. Taking 5G RAN analytics and control to a new level. In *ACM MobiCom*. 1–16.

[9] Xenofon Foukas, Mahesh K Marina, and Kimon Kontovasilis. 2017. Orion: RAN slicing for a flexible and cost-effective multi-service mobile network architecture. In *Proceedings of the 23rd annual international conference on mobile computing and networking*. 127–140.

[10] Xenofon Foukas and Bozidar Radunovic. 2021. Concordia: Teaching the 5G vRAN to share compute. In *ACM SIGCOMM*.

[11] Xenofon Foukas and Bozidar Radunovic. 2025. The future of the industrial AI edge is cellular. In *Proceedings of the 26th International Workshop on Mobile Computing Systems and Applications*. 61–66.

[12] Anuj Kalia et al. 2025. Towards Energy Efficient 5G vRAN Servers. In *USENIX Symposium on Networked Systems Design and Implementation (NSDI)*.

[13] Woo-Hyun Ko, Ushasi Ghosh, Ujwal Dinesha, Raini Wu, Srinivas Shakkottai, and Dinesh Bharadia. 2024. {EdgeRIC}: Empowering real-time intelligent optimization and control in {NextG} cellular networks. In *21st USENIX Symposium on Networked Systems Design and Implementation (NSDI 24)*. 1315–1330.

[14] Lopamudra Kundu et al. 2024. Towards Energy Efficient RAN: From Industry Standards to Trending Practice. *arXiv:2402.11993* (2024).

[15] Microsoft. [n. d.]. Userspace instrumentation and control framework for deploying control and monitoring functions in a secure manner. https://github.com/microsoft/jbpf.

[16] Microsoft. 2025. jrt-controller. https://github.com/microsoft/jrt-controller.

[17] Microsoft. 2025. jrtc-apps. https://github.com/microsoft/jrtc-apps.

[18] Iain Morris. 2024. AT&T, all in with Ericsson, seems to have shut the door to xApps. https://bit.ly/3TZzYP0.

[19] Nvidia. [n. d.]. NVIDIA Isaac ROS. https://developer.nvidia.com/isaac/ros.

[20] Nvidia. 2023. Building Software-Defined, High-Performance, and Efficient vRAN. http://bit.ly/4mjBiIF.

[21] Leonardo Lo Schiavo et al. 2024. YinYangRAN: Resource Multiplexing in GPU-Accelerated Virtualized RANs. In *IEEE INFOCOM*. 1–10.

[22] Rajkarn Singh et al. 2021. Energy-efficient orchestration of metro-scale 5G radio access networks. In *IEEE INFOCOM 2021-IEEE Conference on Computer Communications*. IEEE, 1–10.

[23] David Soldani et al. 2023. ebpf: A new approach to cloud-native observability, networking and security for current (5g) and future mobile networks (6g and beyond). *IEEE Access* 11 (2023), 57174–57202.

[24] srsRAN. [n. d.]. srsRAN Project – Open Source RAN. https://www.srslte.com/.

[25] Chuanhao Sun et al. 2024. SpotLight: Accurate, Explainable and Efficient Anomaly Detection for Open RAN. In *ACM MobiCom*.

[26] Stylianos E Trevlakis et al. 2023. Localization as a key enabler of 6G wireless systems: A comprehensive survey and an outlook. *IEEE Open Journal of the Communications Society* (2023).

[27] Yiwen Zhang et al. 2024. Vulcan: Automatic Query Planning for Live ML Analytics. In *USENIX NSDI*.

Sim2Field: End-to-End Development of AI RANs for 6G

Russell Ford, Pranav Madadi, Daoud Burghal
Samsung Research America
Mountain View, CA, USA
{russell.ford,p.madadi,d.burghal}@samsung.com

Mandar Kulkarni, Xiaochuan Ma, Guanbo Chen, Yan Xin
Samsung Research America
Berkeley Heights, NJ, USA
{m.kulkarni2,shawn.ma,guanbo.h,yan.xin}@samsung.com

Hao Chen, Yeqing Hu, Chance Tarver, Panagiotis Skrimponis, Yu Zhang, Jianzhong Zhang
Samsung Research America
Plano, TX, USA
{hao.chen1,yeqing.h,c.tarver,notis.sk, y3.zhang,jianzhong.z}@samsung.com

Shubham Khunteta, Yeswanth G. Reddy, Ashok K. R. Chavva, Mahantesh Kothiwale
Samsung Research India
Bangalore, India
{sk.khunteta,guddeti.r,ashok.chavva,mahantesh.k}@samsung.com

Davide Villa
Northeastern University
Boston, MA, USA
villa.d@northeastern.edu

Abstract

Following state-of-the-art research results, which showed the potential for significant performance gains by applying AI/ML techniques in the cellular Radio Access Network (RAN), the wireless industry is now broadly pushing for the adoption of AI in 5G and future 6G technology. Despite this enthusiasm, AI-based wireless systems still remain largely untested in the field. Common simulation methods for generating datasets for AI model training suffer from "reality gap" and, as a result, the performance of these simulation-trained models may not carry over to practical cellular systems. Additionally, the cost and complexity of developing high-performance proof-of-concept implementations present major hurdles for evaluating AI wireless systems in the field. In this work, we introduce a methodology which aims to address the challenges of bringing AI to real networks. We discuss how detailed Digital Twin simulations may be employed for training site-specific AI Physical (PHY) layer functions. We further present a powerful testbed for AI-RAN research and demonstrate how it enables rapid prototyping, field testing and data collection. Finally, we evaluate an AI channel estimation algorithm over-the-air with a commercial UE, demonstrating that real-world throughput gains of up to 40% are achievable by incorporating AI in the physical layer.

Keywords

6G, AI RAN, AI channel estimation, Digital Twins

OpenRan '25, November 4-8, 2025, Hong Kong, China

ACM ISBN 979-8-4007-1977-6/2025/11
https://doi.org/10.1145/3737900.3770168

1 Introduction

In recent years, the application of Artificial Intelligence (AI) and machine learning to the wireless networking domain has seen a surge of interest in the telecom industry, thanks to the potential for large performance gains. Beyond enhancing current 5G, AI is expected to revolutionize future 6G technology, particularly at the wireless Physical (PHY) layer, where proposed AI-based Channel Estimation (CE), detection, decoding and Channel State Information (CSI) compression methods, among others, have shown that impressive gains in spectral efficiency and throughput may be possible [22]. While these results are indeed promising, they generally rely on simulation for generating datasets for AI training and evaluation. Thus far, there have been few prototype implementations or field trials demonstrating the true effectiveness of AI at the PHY layer under real-world conditions. Therefore, it may be argued that many of the challenges of

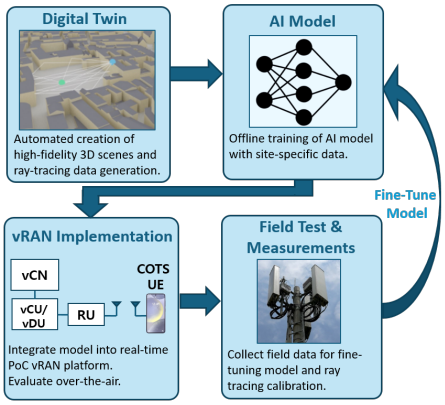


Figure 1: Platform for rapid AI wireless innovation.

making these advanced algorithms perform well in the field have yet to be thoroughly investigated on a practical level.

To accelerate the development of novel AI algorithms and ensure their reliability in real operator networks, a new development approach and experimental platform is required. In this work, we propose such a system, which we hope will serve as a framework for other researchers to rapidly innovate new 5G/6G technologies and address the many challenges of making AI RAN a commercial reality. The proposed framework is illustrated in Fig 1, which also describes the high-level structure of the paper.

To summarize the key contributions of this work,

(1) In Section 2, we introduce a "Simulation-to-Field" (abbreviated Sim2Field) methodology, which aims to address the practical challenges of AI RAN development through site-specific, Digital Twin-based training. Insights are provided on how channel data may be synthesized to address the reality gap of standard channel models and improve data diversity.
(2) Methods are proposed for adding RF impairments to the generated channels, which is found to improve estimation accuracy by over 1 dB.
(3) In Section 3, we explain how the Sim2Field approach is applied toward the development of an AI uplink channel estimation algorithm. While the exemplary ResNet-based algorithm is not, by itself, a novel contribution, practical system integration aspects are presented in detail, which prior literature often neglects to address.
(4) In Section 4, we present an end-to-end prototype system based on the NVIDIA ARC-OTA platform. The ARC testbed enabled a real-time software implementation of the algorithm to be quickly brought up and tested Over-the-Air (OTA) using Commercial Off-the-Shelf (COTS) hardware.
(5) Lastly, in Section 5, we demonstrate the algorithm performance experimentally using a real-time channel emulator, as well as OTA in an indoor lab environment. Our AI-CE implementation achieves over a 40% throughput gain in both OTA tests and channel emulator tests with mobility.

Please note that additional results and discussion are provided in the arXiv version [16].

1.1 Related Work

A large body of work presently exists on machine learning and, in particular, deep learning methods for OFDM channel estimation [3, 4, 12, 19]. However, these studies are based on simulation results, and rarely discuss the practical hurdles at hand – for instance, how to obtain accurate label data for training or compensate for RF impairments. In [17], impairments such as phase noise and frequency offset are considered. However, purely theoretical models are employed, as opposed to the field data-driven approach of our work. In [6], the authors propose a method for field data collection and fine tuning of a neural receiver model. Although tested with field data, OTA evaluation of the system with commercial devices is not conducted. Several 5G testbeds have been developed by the wireless research community, notably POWDER [5], COSMOS [20], Colosseum [11], O-RAN Gym [10], Open AI Cellular [15], and X5G [21]. These systems are either limited in computing power, are not compliant with the 3GPP NR standard, or are designed mainly for evaluating higher-layer components, for example, O-RAN xApps, in contrast to baseband PHY components. Several works have developed prototypes and conducted OTA experiments to validate AI PHY algorithms but are simplistic and lack the real-time, NR-compliant capabilities to test with COTS UEs [4, 14].

2 Sim2Field Methodology

In wireless research today, AI models are commonly trained on datasets generated by stochastic channel models, which are popular thanks to their ease of implementation and simulation speed. These models are designed to characterize a general class of propagation environments, such as the Urban Microcell (UMi) and Rural Macrocell (RMa) scenarios defined by 3GPP [2]. As such, they are not regarded as faithful representations of any specific real-world environment. The resulting "reality gap" thus poses a critical problem for developing AI models, as models pre-trained on simulated data may not generalize well and lose performance when deployed in the field.

While field measurements would provide the greatest accuracy in capturing the channel distribution, the cost and effort involved in collecting extensive measurements makes it infeasible for operators to conduct at scale. Ray Tracing (RT) simulation, on the other hand, presents a happy medium,

as it represents the channel with a high degree of realism and without the need for costly measurements. Though known to be computationally-intensive, tools such as NVIDIA Sionna [7] and Aerial Omniverse Digital Twin [13] offer GPU acceleration of RT computation, allowing large datasets to be generated quickly.

Still, the Channel Impulse Responses (CIRs) generated by RT only reflect the propagation paths through the environment, as well as the Transmitter/Receiver (TX/RX) antenna patterns and Doppler effect, which may be explicitly modeled. However, the received I/Q samples also include impairments from the TX and RX components themselves. As a main contribution of this work, we explain how we have modeled these effects in the channel generation procedure to ensure the training data distribution is similar to the expected real distribution at the input of the baseband system.

The proposed procedure for high-fidelity RT channel simulation of real-world environments, combined with receiver impairment modeling, lends to the development of a Digital Twin of the cellular network, as now described.

2.1 Ray Tracing Scene Generation

Cellular Digital Twins are *surrogate* models, which replicate the exact buildings, terrain, foliage and other blockages, as well as antennas and RF components, for cell sites in the operator network. To this end, we first generate 3D scenes for ray tracing from detailed map data, using tools such as Sionna and Blender [1] to automate most of the scene generation process. The objects imported with the scene are then assigned materials based on their expected composition, such as concrete, glass and metal for buildings and earth for the ground plane.

The scene is then imported into the Sionna RT simulator and the NR gNBs and User Equipment (UE) devices are defined, along with their respective TX power and antenna array parameters, i.e. number of elements, polarization and radiation pattern. The gNB Radio Unit (RU) locations and other parameters may be obtained from site engineering data for the operator network. UEs are randomly dropped in pre-defined regions within the scene. Ray shooting and bouncing is performed to compute paths between the TX and RX, taking into account scattering and diffraction at the given carrier frequency. Doppler shift is applied on the paths, which is a function of the UE velocities. The Channel Impulse Response (CIR) is then computed from these paths, which may be transformed to a Channel Frequency Response (CFR) $H(f)$ for the Orthogonal Frequency-Division Multiplexing (OFDM) channel at the subcarrier spacing defined for the specified NR numerology.

Domain Randomization and RT Calibration. Regardless of how detailed one constructs a Digital Twin, inevitably some residual reality gap is expected with respect to the real environment. Techniques such as domain randomization, which creates diversity in the training data distribution by introducing random variations to the original dataset, have helped to overcome this dilemma in the robotics field and may prove useful for training AI wireless algorithms, as well.

One approach used in this work is to randomize the locations of mobile devices and introduce scene objects of varying dimension and location, for example, by placing random blockages to represent variation in foliage, vehicles, etc. The material properties of different surfaces may also be varied.

2.2 Receiver Impairment Modeling

The RF front-end imparts numerous non-idealities to the received signal, including Timing Offset (TO), Carrier Frequency Offset (CFO), Power Amplifier (PA) non-linearity, Phase Noise and Local Oscillator (LO) leakage. Additionally, the receiver will not have a flat frequency response, especially over the wide 100 MHz carrier bandwidth of 5G systems. Automatic Gain Control (AGC) also dynamically scales the signal in response to the time-varying received power. It is thus necessary to capture these effects in the channel generation procedure to improve the robustness of the trained AI model. How the key impairments are represented and applied for training data generation is summarized in Fig. 2 and described in the following.

Timing and Carrier Frequency Offset. TO and CFO are often present in the received channel, the former due to imperfect synchronization, with the latter attributed to LO mismatch. While these impairments are explicitly compensated for in the baseband processing, residual time and frequency offset may still remain. To model these effects, measurements are obtained using the NR testbed system, described later in Section 4, for the uplink channel between the COTS RU and UE. Channel estimates are computed from the Demodulation Reference Symbols (DMRS) of the Physical Uplink Shared Channel (PUSCH) using a variant of the well-known Minimum Mean-Squared Error (MMSE) algorithm, described in [8]. TO manifests in the frequency domain as a phase shift, which may be estimated directly from the CFRs by finding the average phase difference between adjacent pilot subcarriers $k, k+1$, expressed as

$$TO^i = \frac{1}{N_{ant}(N_p - 1)} \sum_{j=0}^{N_{ant}-1} \sum_{k=0}^{N_p-2} H_{est}^{i,j}(k) H_{est}^{i,j}(k+1)^* \quad (1)$$

for symbol index i within the set of measured CFRs, where N_{ant} and N_p denote the number of antenna ports and pilot symbols, respectively. Here, averaging over antenna ports j

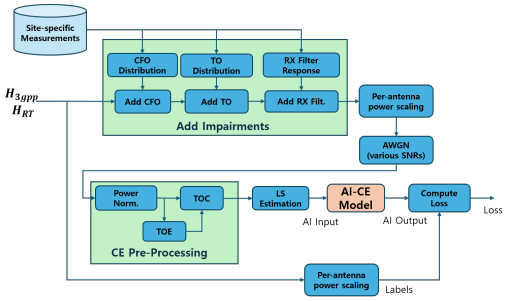


Figure 2: Sim2Field training data generation procedure.

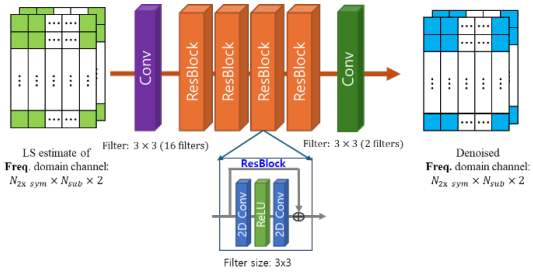


Figure 3: Example ResNet CE model architecture.

improves estimation accuracy. Similarly, CFO may be estimated as

$$CFO^i = \frac{1}{N_{ant}N_p} \sum_{j=0}^{N_{ant}-1} \sum_{k=0}^{N_p-1} H_{est}^{i_1,j}(k) H_{est}^{i_2,j}(k)^* \quad (2)$$

for DMRS symbols i_1, i_2 within the same TDD slot. As shown in Fig. 2, the distribution of TO and CFO, estimated from measurement data, is then randomly sampled and applied to the set of RT channels H_{RT}, or may similarly be applied to channel responses generated from stochastic models, denoted H_{3GPP}.

RX Filter Response Scaling. Typically, the hardware vendor will have characterized the complex filter response of the RU, making it straightforward to incorporate into the channel generation procedure. Otherwise, it may be characterized by averaging the measured frequency-domain channel gain $H_{est}^{i,j}(k)$ over many symbols i. In this work, this was accomplished by collecting measurements in an anechoic chamber for a fixed RU and UE position, since the wireless channel is mostly stationary. A magnitude response with passband ripple of about ±0.5 dB was observed for the Foxconn 7800E RU used in our testbed. The set of RT channels H_{RT} is then scaled by the estimated filter response.

Additive Noise. The Signal-to-Interference-plus-Noise Ratio (SINR) of the received signal can naturally vary with different path loss and inter-cell interference, on top of the receiver noise floor. The final step is to add Gaussian noise for a range of SINRs, over which the receiver operation is expected.

Modeling of other impairments and effects, such as LO leakage and per-antenna power scaling, are discussed in the arXiv version [16]. An ablation study was conducted to assess the impact of training for the aforementioned RF effects on AI-CE performance, the full results of which are also available [16]. The addition of each impairment model individually was found through simulation to improve estimation accuracy, in terms of Normalized Mean-Squared Error (NMSE) w.r.t. the ground truth channel, by between 0.3-0.8 dB.

3 AI Channel Estimation

Channel estimation is a key PHY-layer function for which data-driven, AI techniques may provide an improvement over their conventional counterparts. Previous work has shown that deep learning methods such as denoising CNNs, borrowed from the image processing domain, may offer gains compared to methods like MMSE estimation, which is widely used in 5G systems today [18].

Although the main contribution of this paper is not to introduce a novel AI-CE algorithm, in this section we present an exemplary AI model for uplink (PUSCH) CE, which is designed to be integrated in the real-time, NR-compliant platform. We base the model on the now established ResNet architecture thanks to its robust performance for image denoising problems [12]. Illustrated in Fig. 3, the model input is a 3D tensor with dimensions $2N_{sym} \times N_p \times N_{ant}$, where N_{sym} is the maximum number of OFDM symbols containing DMRS, N_p is the maximum number of pilot subcarriers (1638 for a 100 MHz system) and N_{ant} is the number of RX antennas of the RU. The first dimension corresponds to the CNN channels, for which there are two channels per DMRS symbol, since the complex channel gains are split into real and imaginary components. The input tensor contains the frequency-domain Least Squares (LS) estimates, which must be computed prior to AI estimation, as shown in Fig. 2. The output is then the de-noised version of the LS estimates. An architecture with 4 ResBlocks is chosen to provide a balance between complexity and performance.

4 Real-Time vRAN Testbed

Compared to traditional wireless signal processing algorithms and systems, for which the practical performance is well-established, the early stage of AI wireless technology makes it imperative for prototypes to be built for field evaluation. Unfortunately, due to the demands of 5G and future 6G RANs for high throughput and low latency, and the complexity of today's cellular standards, it is challenging to develop prototype systems that can approximate the capabilities of commercial base stations. Until recently, this has

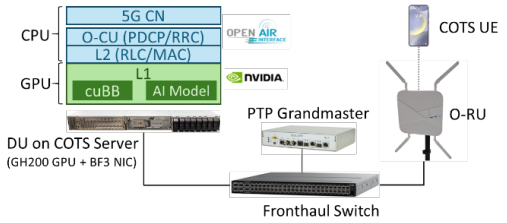


Figure 4: Aerial ARC-OTA testbed.

usually driven the need for high-speed FPGA or ASIC implementations, which are costly and involve long development cycles.

We adopt the Aerial RAN CoLab Over-The-Air (ARC-OTA) system, which is a software-defined virtual RAN platform developed by NVIDIA and the OpenAirInterface (OAI) Software Alliance (OSA). The testbed, shown in Fig. 4, combines powerful GPU servers for the base station Data Unit (DU) and commercial RUs for OTA transmission. OAI provides the Layer 2 MAC/RLC and Layer 3 functions, as well as the 5G core network elements. The L1 PHY functions leverage the NVIDIA Aerial CUDA-Accelerated RAN, referred to as cuBB, for efficient in-line GPU processing. Importantly, the CUDA/C++ software implementation of the L1 shortens the time for custom algorithms, like AI channel estimation, to be developed and tested over-the-air. Further details of the ARC-OTA platform are presented in [9].

Real-Time AI-CE Implementation in cuBB

Another contribution of this work is the development of a framework for incorporating AI models into into the real-time L1 processing of the DU. Extensive modifications were made to the cuBB PUSCH implementation to integrate the AI model introduced in the earlier section. The NVIDIA TensorRT toolchain is used to convert the model, originally developed in Python/PyTorch, to an execution format that is compatible with CUDA/C++. Specifically, a TensorRT *engine* is generated from the PyTorch model, which may then be integrated into the cuBB pipeline, as shown in Fig. 5. Additional details of the AI-CE module implementation are provided in the arXiv version [16].

5 Experimental Setup and Results

In this section, we conduct experiments using the ARC-OTA system toward validating whether the large gains found in previous studies, for example [12], are reasonable to expect in practical systems. In the sequel, we compare the performance of the proposed AI-CE algorithm, trained using the Sim2Field methodology, with respect to conventional MMSE. The baseline MMSE algorithm implemented in the Aerial cuBB PHY is based on the Power-Delay Profile (PDP) approximation scheme discussed in [8]. Salient parameters

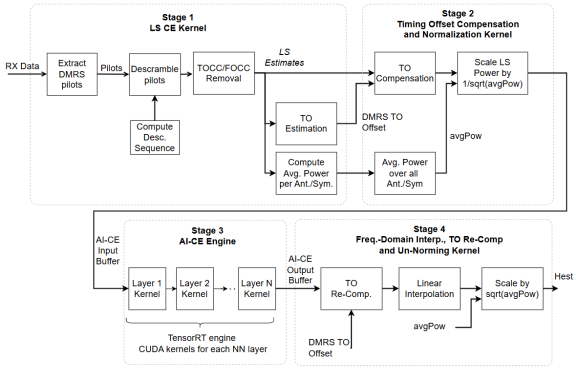


Figure 5: AI-CE cuBB (CUDA) implementation.

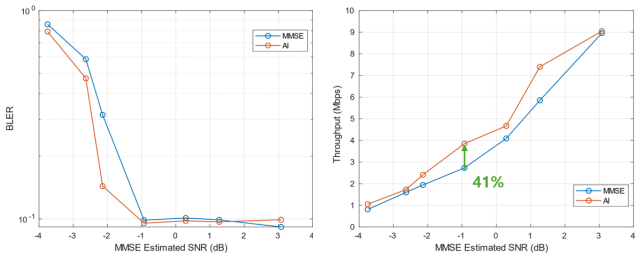
pertaining to the emulator and OTA experiments are given in Table 1.

5.1 Channel Emulator Experiments

In this experiment, the performance of AI-CE is evaluated using a Keysight Propsim channel emulator. A custom AI-CE model is trained with 20k samples of statistical channel data from the well-known 3GPP Clustered Delay Line (CDL) channel model, specifically CDL-C and CDL-D from [2], with delay spread (DS) ranging from 10 to 100ns. The channel emulator is configured with the CDL-B channel with 300ns DS. The UE moves at a constant speed of 30 kmph along a circular path around a single RU with fixed position and antenna orientation, maintaining a constant distance of 200m from the RU. The UE velocity is thus kept constant w.r.t. the RU. In terms of physical setup, the RU is cabled directly to

Table 1: Testbed Experimental Parameters

	Parameter	Value
General	Carrier Frequency	3.75 GHz (band n78)
	Bandwidth	100 MHz
	Duplexing	TDD, DDDSU pattern
	NR Numerology	$\mu = 1$ (30 kHz SCS)
	RU/UE Max. TX Power	24 dBm
	RU Antenna	4 vertical dipoles
	Num. DMRS symbols/slot	3
	Max. PRB allocation	273
	Max. UL Layers	1
	Target BLER	10%
Emulation	Channel Model	38.901 CDL-B, 300 ns DS
	UE Speed	30 k/h
	RU/UE Antenna Spacing	0.5λ
	Target SINR	-4 to 4 dB
OTA	RU-UE Distance	15 m
	Target SINR	5 dB



(a) BLER % vs. SNR (b) Throughput vs. SNR

Figure 6: Results for emulator experiments.

the input/output ports of the channel emulator. Additionally, a Samsung S24 smartphone is modified such that the device's antenna ports are hard-wired to cables with SMA connectors, allowing it to also be directly connected to the emulator.

The testbed implementation enables end-to-end metrics, like UL Block Error Rate (BLER) and MAC-layer throughput, to be obtained. The mean of the observed throughput and BLER are plotted in Fig. 6 over a range of SNRs. The SNR is swept by changing the UE TX power, which is controlled by setting the target SINR in the DU configuration. Closed-loop power control commands from the DU trigger the UE to adjust its TX power to meet the target SINR. Sixty seconds of data are collected for each SNR point.

As seen in the figure, appreciable gains in BLER and throughput of AI-CE w.r.t. MMSE are found over the range of SINRs between -2 to 2 dB, with up to a 41% gain around -1 dB. Above 2-3 dB SINR, the performance of the two algorithms tends to converge. This suggests that AI-based channel estimation could be especially beneficial for cell edge users. The result also gives evidence that an AI model trained with statistical channel data from one distribution (CDL-C/D, in this case) can still generalize and perform well on the unseen CDL-B channel distribution.

5.2 Over-the-Air Experiments

Further experiments are conducted to assess the performance of AI-CE over-the-air in an indoor lab space. A 3D model of the lab, shown in Fig. 7 is created in Blender and imported into Sionna RT for channel data generation, again using the approach from Sec. 2 to train a site-specific model. During OTA testing, it was found that interference and movement

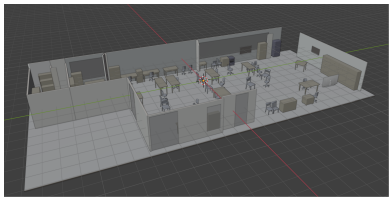


Figure 7: Blender model of lab for ray tracing.

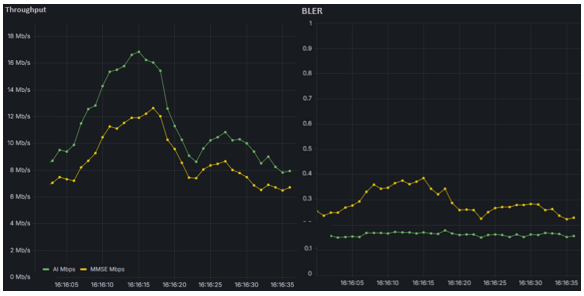


Figure 8: OTA tput. and BLER of AI vs. MMSE.

Table 2: Results for AI-CE trained with 3GPP vs. RT data.

Model	BLER (%)	Tput. (Mbps)
AI-3GPP	18	19
AI-RT (Non-site specific)	16	20
AI-RT (Site-specific)	16	22

of persons through the lab resulted in large variations in the channel, making it difficult to collect repeatable results to compare each algorithm. Modifications were made in the implementation to allow for multiple PUSCH processing chains to be instantiated for the same connected UE, so to compare the real-time performance of AI-CE and the baseline in parallel with the same received I/Q data. The screenshot of the live dashboard in Fig. 8 shows that AI consistently outperforms MMSE by up to a 40% margin in throughput. In this test, the UE is placed in a non-Line-of-Sight location 15m away from the RU and the target SINR is set to 5 dB.

In a separate experiment, an AI-CE model trained on 3GPP channel data is evaluated, along with a model trained on a combination of 3GPP and RT data from other scenes. Table 2 shows the comparison of these two models with the previous AI model trained on site-specific RT data, for the same OTA test setup. It is found that training on non-site specific RT data provides an additional gain over the 3GPP data-trained model, with a larger improvement coming from site-specific training.

6 Conclusions

These early results encouragingly suggest that real-world gains are indeed possible for AI-based channel estimation and other PHY algorithms. To our knowledge, this work is one of the first to comprehensively address the many of the real-world aspects of AI PHY training. Still, before AI may be adopted in production systems, further innovation and testing is undoubtedly needed to ensure reliable performance for a broad range of network devices and environments.

References

[1] [n. d.]. Blender (software). https://www.blender.org/

[2] 3GPP TR 38.901, V18.0.0. 2024. Study on channel model for frequencies from 0.5 to 100 GHz. (Apr. 2024).

[3] A. Feriani and D. Wu and S. Liu and G. Dudek. 2023. CeBed: A Benchmark for Deep Data-Driven OFDM Channel Estimation. *arXiv preprint arXiv:2306.13761* (Jun. 2023).

[4] A. K. Gizzini and M. Chafii. 2024. RNN based channel estimation in doubly selective environments. *IEEE Trans. Mach. Learn. Commun. Netw.* 2 (2024).

[5] J. Breen et al. 2021. POWDER: Platform for Open Wireless Data-driven Experimental Research. In *Proceedings of the IEEE Wireless Communications and Networking Conference (WCNC)*.

[6] J. Corgan et al. 2024. How Critical is Site-Specific RAN Optimization? 5G Open-RAN Uplink Air Interface Performance Test and Optimization from Macro-Cell CIR Data. In *Proc. 2024 IEEE Vehicular Technology Conference*.

[7] J. Hoydis et al. 2023. Sionna: An Open-Source Library for Next-Generation Physical Layer Research. *arXiv preprint arXiv:2203.11854* (Feb. 2023).

[8] K.C. Hung and D.W. Lin. 2010. Pilot-Based LMMSE Channel Estimation for OFDM Systems With Power–Delay Profile Approximation. *IEEE Transactions on Vehicular Technology* 59, 1 (2010).

[9] A. Kelkar and C. Dick. [n. d.]. Introducing NVIDIA Aerial Research Cloud for Innovations in 5G and 6G. https://developer.nvidia.com/blog/introducing-aerial-research-cloud-for-innovations-in-5g-and-6g

[10] L. Bonati and M. Polese and S. D'Oro and S. Basagni and T. Melodia. 2022. OpenRAN Gym: AI/ML development, data collection, and testing for O-RAN on PAWR platforms. In *Proc. IEEE Wireless Communications and Networking Conference*.

[11] L. Bonati et al. 2021. Colosseum: Large-Scale Wireless Experimentation Through Hardware-in-the-Loop Network Emulation. In *Proc. IEEE Intl. Symp. on Dynamic Spectrum Access Networks*.

[12] L. Li, H. Chen, H. Chang, and L. Liu. 2020. Deep residual learning meets OFDM channel estimation. *IEEE Communications Letters* 9, 5 (May 2020), 615–618.

[13] NVIDIA Corporation. 2024. Aerial Omniverse Digital Twin. https://developer.nvidia.com/blog/introducing-aerial-research-cloud-for-innovations-in-5g-and-6g

[14] P. Jiang and T. Wang and B.H. and X. Gao and J. Zhang and C.K. Wen. 2021. AI-Aided Online Adaptive OFDM Receiver: Design and Experimental Results. *IEEE Transactions on Wireless Communications* (Nov. 2021).

[15] P.S. Upadhyaya and N. Tripathi and J. Gaeddert and J.H. Reed. 2023. Open AI Cellular (OAIC): An Open Source 5G O-RAN Testbed for Design and Testing of AI-Based RAN Management Algorithms. *IEEE Network* 37 (2023). Issue 5.

[16] R. Ford et al. 2025. Sim2Field: End-to-End Development of AI RANs for 6G. *arXiv preprint: arXiv:2509.23528* (Sept. 2025).

[17] R. Verdecia-Peña and R. Oliveira and J. I. Alonso. 2024. Enhancing mmWave Channel Estimation: A Practical Experimentation Approach With Modeled Physical Layer Impairments Incorporated in Deep Learning Training. *IEEE Open Journal of the Communications Society* (Jul. 2024).

[18] P. Saikrishna, A.K.R. Chavva, M. Beniwal, and A. Goyal. 2021. Deep Learning Based Channel Estimation with Flexible Delay and Doppler Networks for 5G NR. In *Proceedings of the IEEE Wireless Communications and Networking Conference*.

[19] M. Soltani, V. Pourahmadi, A. Mirzaei, and H. Sheikhzadeh. 2019. Deep learning-based channel estimation. *IEEE Communications Letters* 23, 4 (Feb. 2019), 652–655.

[20] T. Chen et al. 2023. Open-access millimeter-wave software-defined radios in the PAWR COSMOS testbed: Design, deployment, and experimentation. *Computer Networks* 234 (2023).

[21] Villa et al. 2024. An Open, Programmable, Multi-Vendor 5G O-RAN Testbed with NVIDIA ARC and OpenAirInterface. In *Proc. IEEE Conference on Computer Communications Workshops (INFOCOM WKSHPS)*.

[22] C. Zhang, P. Patras, and H. Haddad. 2019. Deep Learning in Mobile and Wireless Networking: A Survey. *IEEE Communications Surveys & Tutorials* 3 (Mar. 2019).

Atomicity in O-RAN

Sixu Tan, Zhaowei Tan
{stan073,ztan}@ucr.edu
University of California, Riverside

Abstract

The Open Radio Access Network (O-RAN) promotes a disaggregated, multi-vendor RAN architecture, offering increased flexibility and innovation. However, this distribution of network functions across components like the Centralized Unit (CU) and Distributed Unit (DU) introduces a critical challenge: ensuring the atomicity of procedures that span open interfaces such as F1, NG, and E2. A failure to maintain transactional atomicity can lead to severe operational issues, including state desynchronization between components and critical resource leakage, ultimately compromising network and user service stability. This paper formally defines the O-RAN atomicity problem and provides the first empirical evidence of its existence in real-world O-RAN testbeds. Through a proof-of-concept analysis, we identify several classes of atomicity violations and propose two possible countermeasures for future exploration. Our results and analysis establish a foundation for developing robust mechanisms to guarantee transactional integrity in next-generation disaggregated RAN.

CCS Concepts

• **Networks** → **Mobile networks**; **Wireless access points, base stations and infrastructure**; **Error detection and error correction**.

Keywords

Open Radio Access Network (O-RAN), 5G Testing and Verification

ACM Reference Format:
Sixu Tan, Zhaowei Tan . 2025. Atomicity in O-RAN. In *2nd ACM Workshop on Open and AI RAN (OpenRan '25), November 4–8, 2025, Hong Kong, China.* ACM, New York, NY, USA, 7 pages. https://doi.org/10.1145/3737900.3770170

1 Introduction

The fifth-generation (5G) of wireless networks is designed to support a diverse array of applications, from enhanced Mobile Broadband (eMBB) for high-speed data access to Ultra-Reliable Low-Latency Communications (URLLC) for mission-critical services like autonomous driving and remote surgery, and massive Machine-Type Communications (mMTC) for the Internet of Things (IoT) [1]. To realize these goals, the network infrastructure, particularly the Radio Access Network (RAN), has been evolving beyond traditional, monolithic architectures. The Open RAN (O-RAN), proposed by the O-RAN Alliance, represents a fundamental shift in network design, promising a more flexible, intelligent, and cost-effective infrastructure by leveraging principles of openness, virtualization, and intelligence [2].

At the heart of the O-RAN architecture lies the principle of disaggregation. Traditionally, RAN has been delivered as a vertically integrated "black box" from a single vendor. O-RAN dismantles this model by breaking down the RAN into distinct, virtualized components: the O-RAN Centralized Unit (O-CU), the O-RAN Distributed Unit (O-DU), and the O-RAN Radio Unit (O-RU) [2], as shown in Figure 1. These components, together with other parts in the 5G system, are interconnected via standardized, open interfaces such as the F1 interface (between O-CU and O-DU), the Open Fronthaul interface (OFH) (between O-DU and O-RU) [2], and the NG interface (between O-CU and 5G Core Network (5GC)). This functional split allows network operators to source components from multiple vendors, fostering competition and innovation. Furthermore, O-RAN introduces the RAN Intelligent Controller (RIC), a platform for third-party applications (xApps/rApps) to manage and optimize radio resources, further disaggregating the RAN control logic from the data plane [3].

However, this profound disaggregation, while beneficial, introduces significant challenges due to its distributed systems nature. Core 5G procedures, which were once executed as monolithic, atomic transactions within a single vendor's RAN, are now partitioned into a sequence of elementary operations distributed across components from multiple vendors [2]. Consider a standard UE handover procedure: in a traditional RAN, the entire process, from measurement reporting and decision-making to resource allocation at the target cell and command execution, is handled internally by a single RAN. In an O-RAN environment, this same procedure is fragmented: an xApp on the Near-RT RIC might make the handover decision, which is then communicated to the O-CU over the E2 interface. The O-CU must then coordinate with

OpenRan '25, November 4-8, 2025, Hong Kong, China

ACM ISBN 979-8-4007-1977-6/2025/11
https://doi.org/10.1145/3737900.3770170

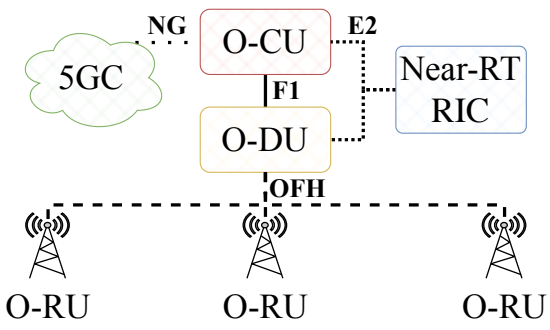


Figure 1: O-RAN Architecture

the source and target O-DUs over the F1 interface, which in turn configure their respective O-RUs.

A failure at any point in this distributed chain can lead to catastrophic inconsistencies. For instance, if the O-CU successfully instructs the UE to switch but then fails before confirming resource allocation with the target O-DU, the UE's connection may be dropped entirely, leaving it in a ghost state with no serving cell. This failure model example exposes a critical problem in the O-RAN architecture: the procedure atomicity across multiple O-RAN components needs to be considered and protected, otherwise it will be a fatal point of failure.

Although the O-RAN ecosystem has standardized and enforced several schemes on interface conformance and interoperability testing [4–6] to ensure a reliable deployment, they are insufficient for detecting atomicity violations. Conformance testing validates individual message exchanges against the specification but does not verify the integrity of an entire end-to-end procedure across a chain of components under fault conditions. Interoperability tests confirm that components can work together on the expected path, but they do not systematically inject faults mid-procedure to uncover latent state inconsistencies or resource leaks that arise from partial execution. The challenge of procedure atomicity is no longer just about standard compliance, but about ensuring transactional integrity in a distributed, multi-vendor system.

This paper formally defines and investigates the atomicity problem in disaggregated O-RAN architectures. We first provide a systematic analysis of the types of atomicity violations that can manifest during fundamental 5G procedures, exploring their impact on network reliability and user experience. We then propose and discuss potential solutions to address procedural atomicity violations in the O-RAN context, laying the foundation for building truly resilient and robust O-RAN systems.

2 Atomicity in Distributed O-RAN Systems

2.1 Definition

In the field of distributed systems, a transaction is a sequence of operations performed as a single logical unit of work. The defining characteristic of a transaction is atomicity, which guarantees that all operations within the sequence are completed successfully as a group; if any operation fails, the entire transaction fails, and the system is rolled back to the state it was in before the transaction began. This "all-or-nothing" property is essential for maintaining data integrity and consistency across distributed resources.

Applying this concept to the O-RAN architecture, an O-RAN transaction can be formally defined as:

> A sequence of control and configuration operations $O_1, O_2, \ldots, O_n$ distributed across a set of O-RAN components $C_1, C_2, \ldots, C_m$ and their interconnecting interfaces $I_1, I_2, \ldots, I_k$, which must be executed as a single, indivisible, "all-or-nothing" unit to transition the network from one consistent state to another.

The O-RAN atomicity problem arises from the fact that while both O-RAN and 3GPP specifications define discrete procedures on individual interfaces, they lack a standardized mechanism to bind these procedures into a single, atomic transaction with a deterministic outcome (commit or rollback) across all involved components. For example:

- The O-RAN Alliance specifies the **E2 Application Protocol (E2AP)** for communication between the RIC and an E2 Node, defining elementary procedures such as `RIC CONTROL` and `RIC SUBSCRIPTION`.
- 3GPP defines the **F1 Application Protocol (F1AP)** with procedures like `UE CONTEXT SETUP`.
- 3GPP also defines the **NG Application Protocol (NGAP)** with procedures like `PATH SWITCH`.

Each of these procedures operates in isolation, with its own message identifiers, timers, and failure handling logic. If one step in a multi-step, multi-interface sequence fails, there is no standardized protocol for automatically propagating that failure state to the other components and triggering a coordinated rollback of the changes already made. Instead, the system may rely on a patchwork of independent timeouts and vendor-specific error-handling implementations, a situation that directly contradicts O-RAN's core goal of multi-vendor interoperability.

Consider a handover procedure initiated by the RIC:

(1) An xApp sends an `E2 CONTROL` message, which has its own E2AP procedure identifier.

(2) The receiving O-CU initiates an `F1AP UE CONTEXT SETUP` towards the target O-DU, with a separate F1AP identifier.
(3) The O-CU then initiates an `NGAP PATH SWITCH` towards the 5G Core, which has yet another NGAP-specific identifier.

If the NGAP procedure fails, the 5GC and the O-CU are aware of this specific failure. However, no standard mechanism exists for the O-CU to automatically correlate this NGAP failure back to the originating E2 transaction or to command the target O-DU to roll back the F1AP context setup. The system is left in a fragmented, inconsistent state, which is a direct result of the inability to enforce atomicity across protocol and vendor boundaries.

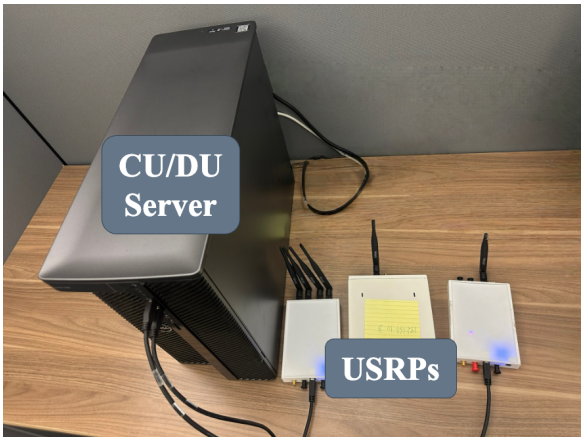


Figure 2: Testbed Setup

2.2 Consequences of Violation

Violations of atomicity in O-RAN transactions can result in several consequences. These consequences are not merely theoretical; they represent tangible operational problems that can degrade network performance, waste resources, and impact the end-user experience. In our study, these consequences can be classified into two primary categories:

State Desynchronization This occurs when different components in the distributed system hold conflicting views of the network's state. Because a transaction failed in the mid of an execution, some components have updated their state while others have not, or have reverted to a previous state. For example, in a failed handover, the O-CU might believe a User Equipment (UE) context has been successfully transferred to a target O-DU, while the target O-DU itself has no record of this context due to a failure in the F1AP procedure. This desynchronization can lead to subsequent protocol errors, incorrect resource management decisions, and an inability to provide service to the affected UE. The system becomes unpredictable because its components are operating on inconsistent assumptions.

Resource Leakage This is a direct consequence of partially executed transactions that allocate network resources but fail before the procedure can be completed or properly cleaned up. In the absence of an atomic rollback, these allocated resources are marked as "in-use" but are not associated with any active session or service. Examples include:

- Radio bearers and C-RNTIs (Cell Radio Network Temporary Identifiers) reserved on an O-DU during a handover that never completes.
- GTP-U (GPRS Tunnelling Protocol - User Plane) tunnels established on an O-CU-UP that are never used because the corresponding path switch at the 5G Core failed.
- PRB (Physical Resource Block) quotas reserved for a network slice on an O-DU scheduler that were never confirmed at the O-CU level.

The leaked resources will affect network capacity, eventually leading to resource exhaustion and an inability to serve new users or sessions, even if the network appears to be lightly loaded.

3 Empirical Analysis of Atomicity Issues

We now show how the atomicity violation manifests in a real-world testbed. As a proof-of-concept study, we assume full access to the source code is granted, so that we can better observe the execution of the implementation.

3.1 Testbed Setup

Our experimental testbed was designed to represent a typical research environment for O-RAN development. The setup consisted of two prominent open-source 5G O-RAN implementations, OpenAirInterface5G (OAI5G) [7] and srsRAN [8], deployed on a workstation equipped with an Intel Xeon W5-3435X processor and 64GB of DDR5 RAM, running Ubuntu 22.04 LTS. Each software stack was connected to its corresponding 5G core network implementation, and USRP B210s were utilized for radio frequency transmission and reception, as shown in Figure 2.

3.2 Instrumentation

To capture the internal procedural states, we performed targeted instrumentation on the source code of the O-CU and O-DU components. The objective was not to alter the program logic but to insert logging hooks at critical function entry and exit points that correspond to distinct stages of a distributed transaction. This allowed us to record when a message was

received, when a specific handler was invoked, and when a corresponding response was generated. The instrumentation points were strategically chosen to provide maximum insight into F1AP procedure handling, as summarized in Table 1. For example, by instrumenting f1ap_handle_message in both the CU and DU, we could precisely log the arrival time of any F1AP message. Further instrumentation within specific message handlers, such as f1ap_du_handle_ue_context_setup_req, allowed us to track the exact moment a component began processing a request, while hooks in generator functions like f1ap_cu_generate_ue_context_setup_request() captured the timestamp of message creation. Note that while we exemplify the instrumentation points on the OAI5G stack in Table 1, the counterparts in srsRAN and other stacks all have similar patterns that are easy to identify.

This instrumentation framework effectively bypasses the "black box" challenge for the purpose of this study, enabling a detailed, evidence-based analysis of procedural timing and state consistency across the disaggregated RAN components.

3.3 Key Findings

Our experiments involved triggering a high volume of traffic of standard 3GPP procedures, including UE Attach, UE Context Release, and PDU Session Modification, within the instrumented testbeds. During the execution of these procedures, our source-code instrumentation captured high-fidelity logs, recording the entry and exit points of key functions across the O-CU and O-DU along with high-resolution timestamps. This logged data was then subjected to an offline trace analysis. We correlated the event sequences from the distributed components, comparing them against the procedural message flows mandated by 3GPP specifications. An atomicity violation was flagged when the observed trace deviated from the specified transactional sequence, such as a component prematurely signaling the completion of a procedure before receiving a required confirmation from its peer. The findings from this process, summarized as four categories in Table 2, highlight the prevalence of such deviations.

Our tests confirmed the Procedural State Inconsistency and Invalid State Acceptance flaws in srsRAN, which lead directly to state desynchronization and resource leakage, respectively. More critically, we identified two instances of a Transactional Race Condition within the srsRAN handover preparation procedure. Under specific timing conditions where the target DU was heavily loaded, it would act upon a resource allocation command before the CU had finalized the transaction, creating a window where a subsequent handover cancellation could result in orphaned resources at the DU.

The OAI stack, while correctly implementing the context management procedures, was not entirely free of issues. We observed one instance of Redundant State Messaging during a PDU Session Modification procedure. A slight delay in the F1AP response from the DU caused the CU's timer to expire, triggering a re-transmission of the modification request. The DU, having already processed the initial request, processed the second one as well, leading to a redundant but harmless confirmation message. While this did not cause a failure in our scenario, such redundant operations can introduce unnecessary processing load and indicate a lack of idempotent design in the message handlers.

These empirical results validate that atomicity flaws, ranging from critical to minor, exist in practice and necessitate the development of more rigorous verification and enforcement frameworks.

3.4 Case Study

We now show two cases from our findings and the detailed analysis. These findings highlight how seemingly minor implementation shortcuts can violate the transactional integrity of distributed RAN procedures, leading to severe consequences listed in § 2.2.

3.4.1 State Desynchronization in UE Context Release. The UE Context Release procedure is a fundamental operation for managing UE lifecycle, ensuring that a UE's context is cleanly removed from the RAN when its connection is terminated. The 3GPP specification defines a strict, synchronous sequence of operations spanning the 5GC, O-CU, and O-DU. The procedure is initiated when the AMF sends a UEContextReleaseCommand message over the NG interface to the gNB. Critically, though not explicitly defined, the O-CU shall forward this command to the relevant O-DU via an F1AP UEContextReleaseCommand. The transaction is only considered complete from the RAN's perspective when the O-DU has de-allocated all UE-specific resources and responded with an F1-AP UEContextReleaseComplete message. Only upon receiving this explicit confirmation from the O-DU should the O-CU send the final UEContextReleaseComplete message back to the AMF over the NG interface. This dependency chain is essential for maintaining a consistent state view across all three entities.

Our results, however, uncovered a critical flaw in this logic. The O-CU component breaks the required F1-level transaction dependency. Upon receiving the initial UEContextReleaseCommand from the AMF, the O-CU initiates a fixed one-second timer. It sends the UEContextReleaseComplete message to the AMF as soon as this timer expires, regardless of the state of the F1 transaction with the O-DU. This behavior fundamentally breaks the atomicity of the procedure.

The analysis reveals a clear failure scenario. If the F1 message to the O-DU is lost or if the O-DU is slow to respond due to high load, the O-CU will still prematurely confirm

Table 1: Example of Source Code Instrumentation Points in OAI5G

Component	Instrumented Function/File		Exported State/Event
O-CU / O-DU	`f1ap_handle_message()` in `f1ap_cu_task.c` / `f1ap_du_task.c`	in	Timestamp of any incoming F1AP message receipt.
O-CU	`f1ap_cu_generate_...()` in `f1ap_cu_ue_context_management.c`	in	Timestamp of outgoing message generation (e.g., UE Context Setup Request).
O-DU	`f1ap_du_handle_...()` in `f1ap_du_ue_context_management.c`	in	Timestamp of specific request processing start (e.g., UE Context Setup).

Table 2: Summary of Detected Atomicity Violations

Violation Category	Triggering Procedure	OAI	srsRAN
Procedural State Inconsistency	UE Context Release		✓
Invalid State Acceptance	UE Context Modification		✓
Transactional Race Condition	Handover Preparation		✓
Redundant State Messaging	PDU Session Modification	✓	

the release to the AMF. At this moment, the state of the system becomes desynchronized. The AMF and the O-CU now consider the UE context to be fully released and its associated identifiers (e.g., AMF UE NGAP ID) to be invalid. The O-DU, however, having never completed the procedure, still maintains the full UE context, including the RRC configuration, security context, and any allocated radio bearers. This inconsistency leads to an unstable system where the CU and DU have conflicting views of the UE's state.

3.4.2 Resource Leakage in UE Context Modification. The second failure consequence, resource leakage, was observed during the UE Context Modification procedure, an operation used to modify parameters of an established UE session, such as security keys or QoS configurations. A foundational principle of robust system design is that any state modification must be preceded by a validation of the target entity's existence. An operation on a non-existent entity should be immediately rejected with an appropriate error.

Our testing demonstrated that the testbed fails to perform this crucial validation step. The system would accept and process UEContextModificationRequest messages that referenced non-existent UE identifiers. Instead of responding with a UEContextModificationFailure indicating "Unknown local UE NGAP ID", the implementation proceeded as if the UE were valid silently. This behavior creates a direct path to resource leakage.

We have to note that, when the O-CU or O-DU receives a request for a non-existent UE, its internal logic, lacking proper validation, may proceed to allocate memory and initialize data structures for this phantom UE context. For example, it might allocate a new object to store the purported security key updates or QoS flow modifications. Because the UE identifier is invalid, this newly allocated memory is immediately orphaned; it is not associated with any legitimate, active UE session. Consequently, this orphaned context will never be subject to a future UE Context Release procedure, as such a procedure would only be triggered for a real UE. Over time, repeated instances of this failed transaction will cause a gradual and unbounded consumption of memory and processing resources within the RAN components. This slow leakage degrades system performance and can eventually lead to service exhaustion and crashes.

4 Restoring Atomicity in O-RAN

In a real-world O-RAN deployment where all components are proprietary, source code instrumentation is not always feasible, which breaks our assumption in § 3. Ensuring transactional integrity under such conditions requires a systematic approach that spans both pre-deployment validation and live network operation. In this section, we outline the primary challenges in this domain and propose two possible approaches to address the atomicity problem.

4.1 Challenges in Analyzing and Enforcing O-RAN Atomicity

Addressing atomicity violations in O-RAN is complicated by two fundamental challenges: the difficulty in observing internal component states and the lack of mechanisms for enforcing corrective actions.

First, verifying the internal states of RAN components is inherently difficult. In a multi-vendor ecosystem, functional components such as the O-CU and O-DU are typically treated as black boxes. While their external behavior is governed by standardized interfaces (F1, NG, E2), their internal implementation, including the specific state machines and logic that manage procedures, remains proprietary. Most atomicity violations, such as the state desynchronization observed in our case study, originate as inconsistencies within these internal states. An external observer monitoring interface traffic can only infer the state of a procedure, but it cannot directly confirm that the internal logic of a component has transitioned correctly. This lack of visibility means that violations can remain latent, only becoming apparent after they have caused a service-impacting failure.

Second, even when an atomicity violation is detected, enforcing transactional integrity is a significant challenge. The current O-RAN specifications do not include protocols for distributed transaction management, such as a two-phase commit [9], which are standard in other distributed systems. If a central controller, like the RIC, detects that a procedure has failed midway, it has no standardized mechanism to command all involved components to atomically roll back their state changes. The controller could attempt to send a sequence of standard messages to reverse the operation, but there is no guarantee that these compensatory actions will succeed, especially if a component has entered an unknown or failed internal state. This enforcement gap makes it difficult to recover gracefully from partial failures, often requiring a full component reset to restore a consistent state.

4.2 Potential Solution Directions

To overcome these challenges, we propose two potential approaches: proactive offline testing and reactive online enforcement.

4.2.1 Testing-wide. One potential solution lies in the domain of enhanced pre-deployment testing and verification. This approach moves beyond basic conformance testing to employ more sophisticated techniques designed to uncover distributed logic flaws. Using model-based testing, a formal model of a complete, multi-interface procedure (e.g., an entire handover transaction) can be created. This model serves as a "golden reference" to automatically generate complex test sequences that probe for race conditions, handle simulated message loss, and validate behavior under unexpected message ordering. By systematically targeting these corner cases in a controlled lab environment before deployment, this testing-centric approach can proactively identify and eliminate the types of implementation flaws that lead to atomicity violations in live networks.

4.2.2 Monitoring-wide. A complementary solution is the development of an online monitoring and enforcement framework, likely realized within the RIC. This approach involves deploying a dedicated xApp or rApp that acts as a runtime transaction monitor. By subscribing to relevant messages across the E2, F1, and NG interfaces, this application could track the state of critical procedures for active UEs in real-time. For instance, it could correlate a UEContextReleaseCommand on NG with the corresponding messages on F1. If the monitor detects a sequence that violates the specified protocol logic (such as a premature confirmation to the AMF). it can take immediate action. Initially, this action could be to raise an alarm for the network operator. In a more advanced implementation, the RIC could leverage its control capabilities via the E2 interface to attempt a corrective action, such as forcing the non-compliant component to release the stale UE context, thereby actively enforcing atomicity in the live network.

5 Conclusion and Future Work

The disaggregation of the Radio Access Network, a central tenet of the O-RAN architecture, introduces significant challenges to maintaining procedural integrity across a multi-vendor, distributed environment. This paper has formally defined the atomicity problem in O-RAN, demonstrating that failures to ensure transactional guarantees for procedures spanning the F1, NG, and E2 interfaces can lead to critical failure modes such as state desynchronization and resource leakage. Through a detailed case study and a proof-of-concept analysis of two prominent open-source implementations, we have provided empirical evidence that these atomicity violations are not merely theoretical but exist in practice. Our findings, which identified flaws ranging from incorrect dependency handling to race conditions, underscore the urgent need for a systematic approach to ensuring transactional integrity.

We proposed two possible solutions, including proactive, pre-deployment verification and reactive, online monitoring and enforcement. This work serves as a foundational step in highlighting and categorizing a critical but often overlooked aspect of O-RAN operational stability.

Looking ahead, our future work will proceed along the two primary solutions and synthesize a framework. The ultimate goal is to create a tool that vendors and operators can use to ensure the transactional integrity of their components in real-world deployments.

Acknowledgment

We would like to sincerely thank the anonymous reviewers for their valuable feedback and advice to help substantially improve the quality of this paper. This work is supported in part by NSF CNS-2403050.

References

[1] Benish Sharfeen Khan, Sobia Jangsher, Ashfaq Ahmed, and Arafat Al-Dweik. Urllc and embb in 5g industrial iot: A survey. *IEEE Open Journal of the Communications Society*, 3:1134–1163, 2022.

[2] Michele Polese, Leonardo Bonati, Salvatore D'oro, Stefano Basagni, and Tommaso Melodia. Understanding o-ran: Architecture, interfaces, algorithms, security, and research challenges. *IEEE Communications Surveys & Tutorials*, 25(2):1376–1411, 2023.

[3] Hoejoo Lee, Jiwon Cha, Daeken Kwon, Myeonggi Jeong, and Intaik Park. Hosting ai/ml workflows on o-ran ric platform. In *2020 IEEE Globecom Workshops (GC Wkshps*, pages 1–6. IEEE, 2020.

[4] Spirent. Comprehensive Open RAN Testing. https://www.spirent.com/campaign/how-to-test-open-ran.

[5] Open RAN Test | VIAVI Solutions Inc. . https://www.viavisolutions.com/en-us/products/open-ran-test.

[6] O-ran alliance global plugfest. https://www.o-ran.org/plugfest.

[7] OpenAirInterface Software Alliance. OpenAirInterface5G. https://openairinterface.org/.

[8] srsRAN. srsRAN - Open Source RAN. https://www.srslte.com/.

[9] Sape Mullender. *Distributed systems*. ACM, 1990.

Log Pruning for Efficient Q&A in 5G Networks

Ali Mamaghani[1], Qingyuan Zheng[1], Ushasi Ghosh[1], Vicram Rajagopalan[2], Srinivas Shakkottai[2], and Dinesh Bharadia[1]
[1]University of California San Diego, CA, USA, [2]Texas A&M University, TX, USA

Abstract

Large Language Models (LLMs) excel across many tasks but struggle in specialized domains like system logs, often leading to inaccuracies. Retrieval-Augmented Generation (RAG) mitigates this by grounding responses in external knowledge, yet conventional RAG faces scalability issues with massive structured datasets due to the high computational cost of embeddings and vector operations. We introduce a novel RAG architecture tailored for large-scale structured data. By combining intelligent context pruning with signal processing techniques that exploit inherent log structures, our design enables rapid, efficient retrieval with reduced computational overhead while maintaining accuracy. We validate this approach on a real-world, high-throughput 5G codebase generating millions of log entries per minute, alongside a comprehensive Q&A dataset on these logs. Results show substantial gains in both speed and accuracy over traditional RAG, making this solution well-suited for real-time analytics in large-scale operational environments.

CCS Concepts

• **Networks → Network measurement**; • **Software and its engineering → Software creation and management**; • **Information systems → Data management systems**.

Keywords

5G, Log Processing, LLM, RAG, Pruning

ACM Reference Format:

Ali Mamaghani[1], Qingyuan Zheng[1], Ushasi Ghosh[1], Vicram Rajagopalan[2], Srinivas Shakkottai[2], and Dinesh Bharadia[1]. 2025. Log Pruning for Efficient Q&A in 5G Networks. In *Open & AI RAN 2025, November 4–8, 2025, Hong Kong, China.* ACM, New York, NY, USA, 7 pages. https://doi.org/10.1145/3737900.3770171

1 Introduction

The advent of 5G networks ushers in unprecedented scale, dynamic configurations, virtualization, and multi-access edge computing, generating vast real-time streams of operational log data. With massive IoT, eMBB, and URLLC driving exponential growth, these logs—often reaching gigabytes per minute per network slice or edge site—are critical for ensuring stringent SLAs and enabling root cause analysis in highly distributed environments. Yet, manually navigating such heterogeneous datasets is infeasible, and conventional automated pipelines for parsing, compression, and anomaly detection often fail to provide deep, contextual insights into real-time system state and behavior.

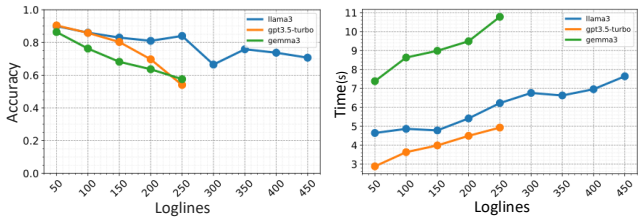
Crucially, network logs implicitly capture the system's operational state machine over time. The ability to issue natural language question-and-answer (Q&A) queries directly on these streams represents a paradigm shift, enabling timely, granular insights into performance, faults, and state transitions that are otherwise buried in fragmented architectures.

Advances in Large Language Models (LLMs), with their exceptional natural language understanding, now make it feasible to build automated Q&A agents that interpret complex log patterns and bridge human intent with the vast landscape of 5G system logs. While LLMs hold great promise for log analysis, directly feeding raw logs is impractical due to limited context windows (thousands of tokens vs. gigabytes per minute in 5G). Retrieval-Augmented Generation (RAG) addresses this by selecting only relevant fragments, but applying it to massive, continuous log streams remains slow and compute-intensive. As shown in Figure 1, expanding input context not only degrades accuracy but also increases response latency, underscoring the need for effective context pruning. Prior efforts in pruning, parsing, and compression help reduce input size, yet fall short at 5G scale. Compression, for instance, removes redundancy but without semantic filtering still passes irrelevant or noisy entries from heterogeneous 5G elements. Consequently, existing methods mitigate some challenges but fail to deliver the efficiency and generalizability needed for real-time, large-scale log reasoning in dynamic 5G networks

We address the critical challenge of enabling real-time, interactive question answering over massive, high-throughput 5G log streams. We propose a novel real-time Retrieval-Augmented Generation (RAG) pipeline designed to overcome the latency bottlenecks of conventional RAG. Our key contribution is a high-speed context retrieval mechanism that

OpenRan '25, November 4-8, 2025, Hong Kong, China

ACM ISBN 979-8-4007-1977-6/25/11
https://doi.org/10.1145/3737900.3770171



(a) Accuracy vs number of log lines (b) Generation time vs number of log lines

Figure 1: System accuracy and generation time

operates directly on incoming logs. Unlike traditional approaches that index large historical corpora, our method processes logs as they arrive (e.g., mapping strings to numeric time-series) and rapidly correlates them with query-derived patterns. This correlation bypasses slow index lookups, acting as an out-of-the-loop accelerator that quickly extracts relevant segments for the LLM, enabling genuine real-time interrogation at scale.

Log data, however, introduces unique challenges. Entries are dynamic and time-sensitive, dominated by timestamps, numerical values, and domain-specific abbreviations—features with low semantic richness that hinder direct interpretation. LLMs must perform semantic inference on such inputs, but limited context windows and fragmented data increase hallucinations and reasoning errors. Moreover, conventional text-based RAG pipelines are poorly suited for evolving log streams where rapid, low-latency synthesis is essential. Without efficient retrieval and context generation, RAG cannot meet the demands of 5G monitoring, fault diagnosis, and anomaly detection.

To address these challenges, we develop a log-aware RAG retrieval pipeline for real-time 5G log analysis, and implement it on an O-RAN–compliant testbed that enables controlled scenario creation under both split 8 and split 7.2 architectures. Building on this testbed, we further construct a large-scale 5G log Q&A dataset spanning diverse operational scenarios, which we use to rigorously evaluate and benchmark our approach.

2 Related Work

Retrieval-Augmented Generation (RAG). RAG has emerged as a powerful paradigm for enhancing the performance of LLMs by incorporating external knowledge without modifying the model parameters [4, 8, 11]. In a typical RAG pipeline, a dense retriever first selects relevant top-k contexts from an external corpus [7, 10], which are then fed into an LLM to generate accurate and grounded responses. This mechanism enables LLMs to handle long-tail knowledge and provide up-to-date information [17]. However, the limited context windows of large language models require sophisticated context pruning or selection mechanisms to identify the most relevant information from potentially vast retrieval results.

Context Pruning. Context pruning approaches improve RAG scalability by reducing the sizes of contexts to be used for generation. Many techniques identify and remove the least relevant portions of the context [1, 5, 9, 12, 13, 16]. Related techniques summarize or rephrase the context to reduce its size [14, 15]. Many context selection techniques exist, but applying these to the unique structure and temporal nature of log data is not straightforward. As seen in a comparison of various state-of-the-art context selection methods [1], most methods achieve a compression ratio of 4x or less, which is insufficient given the the massive scale of system log data.

Log-Based Question-Answering. Specifically for the log domain, LogQA [3] introduces a framework for log-based question answering where a retriever model is trained using supervised learning to select relevant log snippets given a query. Although effective within a specific domain, this approach suffers from a key limitation: it requires system-specific training data. This means a new retriever model must be trained for each type of system log, limiting its generality and requiring significant effort to adapt to new environments or log formats.

3 Testbed and Dataset

This section details the experimental setup of a 5G Standalone network testbed built with open-source software and commercial hardware. It explains how this testbed is configured to collect highly granular, DEBUG-level logs from every layer of the gNB protocol stack. Finally, it outlines the process of using GPT-3.5 and human validation to convert these raw log files into a clean, high-quality question-and-answer dataset for evaluation.

3.1 Experimental setup

Our experimental setup is a full end-to-end 5G standalone (SA) testbed with two complementary architectures (Figure 2). On the left, we deploy a split-8 configuration: the 5G Core (5GC) and gNB run on an Amarisoft UE emulator connected to a USRP X410 SDR as the Radio Unit (RU), establishing over-the-air links to both emulated and COTS UEs (Quectel 5G modem, OnePlus 8T, Pixel 8). On the right, we implement a split-7.2 architecture with CU, DU, and 5GC built from the srsRAN codebase on a high-performance server. The split-7.2 RU (Foxconn RPQN7801E) is synchronized via a PTP grandmaster clock and integrated with NVIDIA ARC-OTA edge server. In both setups, DEBUG-level logging and packet captures are enabled across the gNB stack, enabling fine-grained cross-layer analysis with heterogeneous UEs.

3.2 Log Generation

During log generation, we developed automated scripts to manage scenario configuration and execution in a unified

Case	Scenario	Log Level	Runtime	Log Size
1	1UE: UE Disconnection and New UE Registered	Global INFO; DEBUG Across PDCP/RLC/MAC/F1	120s	0.96 GB
2	1UE: UE SNR Drop under Adverse Channel Conditions		90s	0.93 GB
3	1UE: Mobility (COTS UE in Fast Motion)		90s	1.07 GB
4	1UE: Congested Network (Load Exceeds Capacity)		120s	1.91 GB
5	2UE: Higher DL than UL (both UEs)	Global DEBUG	90s	1.61 GB
6	2UE: UE1 UL, UE2 DL (UE2 Higher Rate)		90s	2.76 GB
7	4UE: Three Normal, One Very High Rate		90s	1.36 GB
8	4UE: UE1/UE2 Higher UL; UE3/UE4 Higher DL		90s	1.41 GB

Table 1: Scenario types, modes, and corresponding log sizes

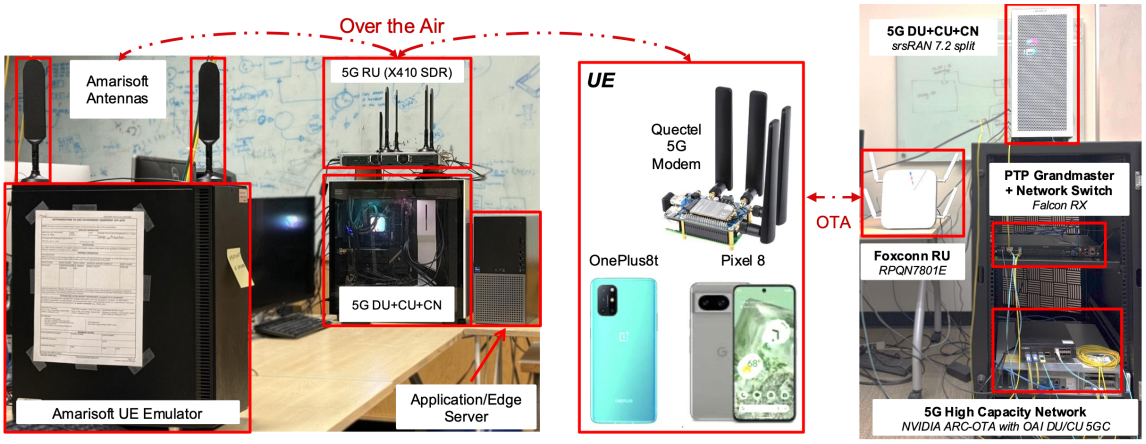


Figure 2: Experimental 5G Standalone (SA) network testbed.

manner. Each script automatically records the execution timestamp of every command. For different scenarios (shown in Table 1), traffic was generated using `iperf` under both ZMQ and Over-the-Air (OTA) modes, covering both TCP and UDP transmissions. By configuring multi-user cases (2UE/4UE) with symmetric or asymmetric uplink/downlink rates, varying channel conditions (e.g., SNR degradation, mobility), and network congestion, we collected logs ranging from 0.9 GB to 2.8 GB. As shown in Table 1, even a short 2-minute run generates logs at the gigabyte scale. Without pruning, such massive data would easily overwhelm the context window of LLMs, leading to inefficient or even inaccurate question answering. The scale and diversity of these experimental logs ensure a broad coverage of representative 5G operating conditions, while the precise execution timestamps provide reliable ground truth to validate answers in log-based Q&A tasks.

3.3 Dataset generation

We created a high-quality instruction-based dataset from raw log files using a combination of AI generation and human oversight. Each log line was provided as a prompt to a large language model (e.g., GPT-3.5), which generated a relevant question–answer pair. This automated process produced a large volume of Q&A samples, which were then carefully reviewed by human validators, with incorrect or irrelevant pairs discarded. The result is a clean, human-verified dataset that accurately reflects the source logs and is well-suited for evaluating our design.

4 System Design

The system architecture is shown in Fig. 1, which mainly consists of three modules: pruning, inference and reasoning.

The pipeline consists of three key stages: preprocessing, pruning, and inference.

4.1 Preliminaries

Log Preprocessing. The preprocessing phase standardizes heterogeneous log data into a uniform, query-friendly format. Given the semi-structured nature of logs, raw data originating from diverse sources, including flat key–value records and deeply nested JSON, poses significant challenges for log vectorization. To address this, we enforce a consistent structure by separating each log into two components: a

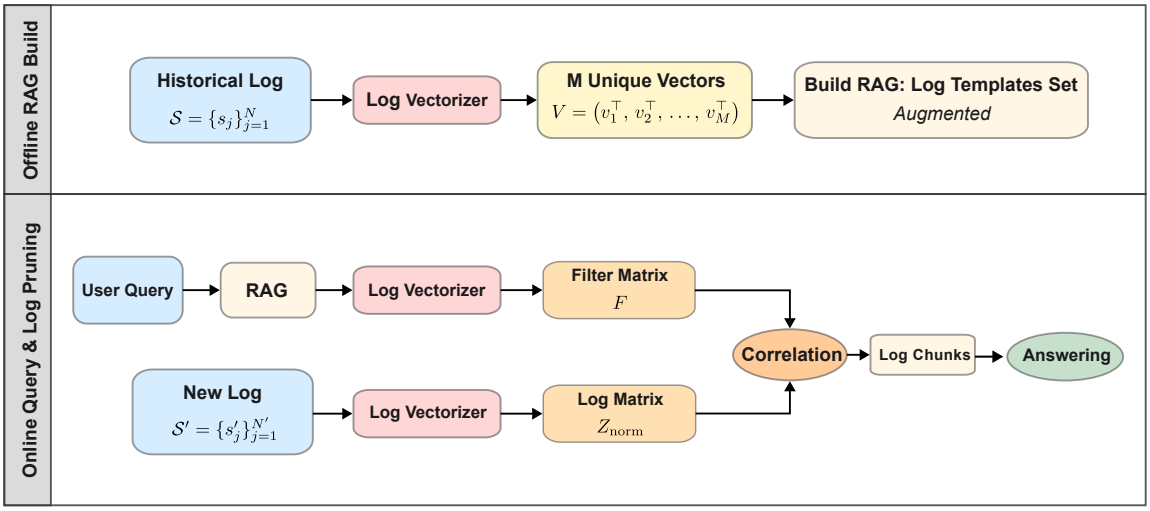


Figure 3: Log Pruning System Overview.

timestamp that serves as a temporal anchor and an information block that consolidates key diagnostic content. This normalization facilitates template detection, supports efficient indexing, and enables accurate semantic similarity matching.

Log Vectorization. During the offline stage, we build an n-gram vocabulary from the historical corpus $\mathcal{S}$. Each log is converted into sparse unigram–bigram features, forming a global vocabulary of size M. The vocabulary is embedded into a matrix $V \in \mathbb{R}^{M \times d}$ and normalized row-wise, yielding V_{norm}. This representation preserves recurring character patterns while filtering out dynamic fields such as timestamps and identifiers.

New logs $\mathcal{S}'$ are cleaned and mapped into vectors $Z_{\text{norm}} \in \mathbb{R}^{N' \times M}$. Cosine similarity between Z_{norm} and V_{norm} produces a score matrix S, from which each log s'_j is assigned to its nearest template:

$$\text{idx}_j = \arg\max_{1 \le i \le M} S_{j,i}.$$

The resulting ID sequence idx transforms the log-stream into template-level representations, enabling efficient indexing and retrieval.

RAG Augmentation. We use extracted log templates as the RAG basis and enrich them with domain knowledge to improve retrieval quality and semantic alignment. Specifically, we augment each log template with explanatory context derived from protocol specifications and implementation details. Linking each template to relevant context improves interpretability, match accuracy, and disambiguates abbreviations in logs. For example, in following log template:

```
[RLC ] [D] du=<:NUM:> ue=<:NUM:> <:*:> UL: RX
SDU. payload_len=<:NUM:> dc=data p=<:NUM:> si=full
sn=<:NUM:>
```

"si" denotes the segmentation-offset state in the 5G RLC layer, which generic models cannot infer without domain grounding. Augmentation ensures that such terms are correctly interpreted during retrieval. Finally, we encode the augmented templates using the "BAAI/bge-large-en-v1.5" sentence embedding model and indexes them with FAISS [2, 6].

4.2 Offline RAG Build

The offline RAG build stage establishes the foundation for efficient retrieval by transforming heterogeneous historical logs into a compact template set. As shown in Figure 3, the process starts with the historical corpus S, which is normalized to ensure a uniform structure across diverse formats.

Each normalized log is processed by the *Log Vectorizer*, which extracts character-level n-gram features and masks dynamic fields (e.g., timestamps, identifiers), producing sparse structural representations that capture recurring patterns while filtering incidental variability.

The resulting vectors are aggregated and deduplicated into M unique templates, denoted V. This template library serves as the semantic dictionary for the RAG system and is further expanded with LLM-based augmentation to improve alignment with user queries.

4.3 Online Query and Log Pruning

At runtime, we have two parallel streams, one is RAG Retrieval, well another stream is new log processing.

Pruning Method	Model	Accuracy (%)	Compression (%)	Generation Time (s)
No Pruning	GPT-3.5	54.0	0.0	2.84
	GPT-4o-mini	91.0	0.0	7.57
	LLAMA-3	70.7	0.0	7.64
	GEMMA-3	57.5	0.0	10.78
8-Template Pruning	GPT-3.5	77.2	82.5	0.91
	GPT-4o-mini	76.3	83.8	2.57
	LLAMA-3	71.2	82.0	5.37
	GEMMA-3	70.2	82.5	8.74
32-Template Pruning	GPT-3.5	85.3	60.2	0.96
	GPT-4o-mini	89.3	62.9	3.32
	LLAMA-3	85.9	60.3	6.09
	GEMMA-3	73.7	60.2	9.44

Table 2: Comparison of pruning methods and their impact on accuracy, compression, and generation time

RAG Retrieval. In this stage, the user query is compared against the augmented template set $\widetilde{T}$. The top-k most similar templates are selected, with their indices denoted by $\mathcal{I}$, and the corresponding entries from the original set T are retrieved. Each selected template is then encoded into a unigram–bigram vector representation, and the resulting collection is stacked into the *Filter Matrix* $F \in \mathbb{R}^{k\times d}$. Acting as a high-precision gatekeeper, F filters out irrelevant log structures and forwards only the most semantically aligned templates to downstream classification. The functionality of this module is illustrated in Example 1.

Example 1: RLC Retransmission Query. For a query "Why frequent uplink retx?", top-k templates often include patterns like `[RLC] PDU, ..., RETX, ... , sn=. . .` and `STATUS report update`. Their rows in F will emphasize bigrams tied to `RETX`, `STATUS`, and `sn`, steering subsequent pruning toward those structures.

New Log Processing. In parallel, we process the stream of new logs, filtering them with respect to timestamp to ensure temporal relevance to the query. These logs are then vectorized to form the Log Matrix, where each vector encodes a single log entry. We describe the vectorization process in the *Log Vectorization* section.

Log Selection. To perform local log selection, we first determine the indices of the expected templates within the new log chunks and then compute the corresponding relevance scores:

$$M \;=\; Z_{\text{norm}}\, F^{\top} \;\in\; \mathbb{R}^{N'\times k}$$

Then, using the following expression, we identify the rows that are highly consistent with our extracted templates:

$$Index \;=\; \Big\{\, i \;\Big|\; \max_{1\le j\le k} M_{i,j} > Threshold \Big\}.$$

Using the new log lines as $\mathcal{S}'$ in the **Log Vectorization** section, we can have the final retrieved lines as: $\text{MatchedLines} = \{\, s_i \mid i \in Index\}$. We demonstrate the operation of log selection using Example 2.

Example 2: Timestamps + Query. For a query scoped to $[t_0, t_1]$, only logs in this window are vectorized. If F is dominated by RLC-reassembly patterns, then $\max_j M_{i,j}$ spikes precisely on lines containing `RX SDU`, `sn`, `si`, etc., while EL1/PHY noise is pruned out. This yields a compact, query-aligned context for question answering.

5 Experiments

To rigorously evaluate the effectiveness of our system, we conducted a series of experiments for a variety of LLMs. We performed a quantitative assessment of our pruning strategy, examining its compression ability, generation time, and accuracy in direct question-and-answer tasks.

5.1 Benchmark Results

The pruning module is a core innovation of our system, designed to optimize downstream retrieval and inference by judiciously selecting only the most relevant log templates and their associated entries. To demonstrate the efficacy of this approach, we performed an experiment comparing the accuracy achieved by our system—which feeds a precisely pruned log context, even approaching the maximum token size—to the accuracy obtained when directly presenting raw, unpruned logs to an LLM. We constructed a dataset containing large log entries, with realistic parameter distributions and timestamp metadata. A set of 198 queries were generated, each associated with a ground truth mapping to the correct log templates required for accurate answers. For each query, we measured:

Answer Accuracy is the proportion of questions where the LLM's output correctly matches the expected answer. To evaluate this, we use a sophisticated LLM, such as **DeepSeek-R1**, as a judge. This judge model is given the question and the generated answer, then evaluates its correctness based on a predefined rubric. This method is highly scalable and provides more nuanced feedback than traditional methods, as the judge can explain its reasoning for the score it gives.

Compression is the ratio of the pruned tokens from the input of LLM. This metric measures the efficiency of pruning in the input data that the model needs to process to generate an answer. A higher compression can decrease both computational cost and processing time.

Generation Time is the total time it takes for the LLM to produce a complete answer after receiving the input. This includes the time spent processing the input and generating the output tokens. It is a critical measure of the system's latency and overall speed.

Table 2 demonstrates that pruning can be an effective technique to improve both accuracy and generation time for language models. Pruning reduces the number of tokens an LLM must process, which leads to a faster response. For models like GPT3.5 and LLAMA-3, pruning also significantly boosts accuracy, suggesting that it helps the models focus on the most relevant information without getting distracted by noise in the original input. However, the effect of pruning can vary by model. For a highly capable model like GPT-4o-mini, an aggressive pruning method (e.g., 8 templates) may harm performance. In contrast, a more moderate approach (32 templates) can restore its accuracy while still cutting costs. The trade-off between the degree of compression and accuracy is evident, with 32 templates offering a more balanced outcome for most models compared to the 8-template approach.

5.2 Q&A examples

The following examples illustrate how our pruning methodology improves accuracy by removing irrelevant content. Expanding the template set further enhances retrieval, ensuring logs relevant to the query are captured. We define the corresponding log entry as the specific entry containing the answer, selected from a large and diverse log set. Example 1 highlights how pruning preserves factual integrity, Example 2 shows how larger templates improve coverage, and Example 3 demonstrates how pruning enables accurate responses to high-level domain-specific questions.

As future work, we plan to expand to larger and more diverse log datasets, supporting more complex queries. This will enable reasoning-based tasks beyond simple retrieval, such as multi-step analyses across heterogeneous log events. Such capabilities are especially important in 5G, where performance issues often stem from cross-layer interactions. Advancing in this direction will support more comprehensive diagnostics, including anomaly detection, stability assessment, and root cause analysis.

Eg. 1: Answer with no/32 pruning (GPT3.5-turbo)

Corresponding log entry: 2025-05-11T20:41:57.747348 [RLC] [D] du=0 ue=2 DRB1 UL: Generating status PDU. rx_next=123544 rx_highest_status=123544 stop_sn=123544
Question: In the log with timestamp 2025-05-11T20:41:57.747348, what is the value of "rx_next" for DRB1 UL?
Ground Truth Answer: 123544
Generated answer with no template pruning: Based on the provided log, in the log with timestamp 2025-05-11T20:41:57.747348, the value of "rx_next" for DRB1 UL is **123547**.
Generated answer with 32-template pruning: Based on the provided log, the value of "rx_next" for DRB1 UL at timestamp 2025-05-11T20:41:57.747348 is **123544**.

Eg. 2: Answer with 8/32 pruning (GPT3.5-turbo)

Corresponding log entry: 2025-05-11T20:41:57.747345 [RLC] [I] du=0 ue=2 DRB1 UL: RX PDU. pdu_len=669 dc=data p=0 si=first sn=123544
Question: At timestamp 2025-05-11T20:41:57.747345, what is the length of the received Protocol Data Unit (PDU) in UL for DRB1?
Ground Truth Answer: 669 bytes
Generated answer with 8-template pruning: Based on the provided log, at timestamp 2025-05-11T20:41:57.747345, the length of the received Protocol Data Unit (PDU) in UL for DRB1 is **1504**.
Generated answer with 32-template pruning: Based on the provided log at timestamp 2025-05-11T20:41:57.747345, the length of the received Protocol Data Unit (PDU) in UL for DRB1 is **669 bytes**.

Eg. 3: Answer with 32-template pruning (GPT3.5-turbo)

Question: I set the bitrate to 120 Mbps, but why does UE bitrate drop to 43 Mbps?
Answer: The drop in UE bitrate from 120 Mbps to 43 Mbps could be due to several factors observed in the log:
Poll Retransmit Timer: The log shows multiple instances of the poll retransmit timer being started (e.g., lines 1, 97, 100, 612). This indicates potential retransmissions, which can reduce effective throughput.
Grant Length Variability: The grant lengths for PDUs vary significantly (e.g., lines 2, 5, 9, 13), suggesting inconsistent resource allocation, which can impact throughput.
Queued SDUs: The number of queued SDUs remains high (e.g., lines 3, 20, 27), indicating potential congestion or bottleneck issues.
These factors collectively contribute to congested network.

References

[1] Natalia Chirkova, Timothee Formal, Vassilina Nikoulina, and Stéphane Clinchant. 2025. Provence: Efficient and Robust Context Pruning for Retrieval-Augmented Generation. arXiv:2501.16214 doi:10.48550/arXiv.2501.16214 arXiv preprint arXiv:2501.16214.

[2] Matthijs Douze, Alexandr Guzhva, Chengqi Deng, Jeff Johnson, Gergely Szilvasy, Pierre-Emmanuel Mazaré, Maria Lomeli, Lucas Hosseini, and Hervé Jégou. 2024. The Faiss library. (2024). arXiv:2401.08281 [cs.LG]

[3] Shouheng Huang, Yao Liu, Christopher Fung, Jun Qi, Hongxu Yang, and Zhi Luan. 2023. LogQA: Question Answering in Unstructured Logs. arXiv:2303.11715 doi:10.48550/arXiv.2303.11715

[4] Gautier Izacard and Edouard Grave. 2021. Leveraging Passage Retrieval with Generative Models for Open Domain Question Answering. In *Proceedings of the 16th Conference of the European Chapter of the Association for Computational Linguistics (EACL)*. 874–880. https://aclanthology.org/2021.eacl-main.74

[5] Huiqiang Jiang, Qianhui Wu, Chin-Yew Lin, Yuqing Yang, and Lili Qiu. 2023. LLMLingua: Compressing Prompts for Accelerated Inference of Large Language Models. In *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing*, Houda Bouamor, Juan Pino, and Kalika Bali (Eds.). Association for Computational Linguistics, Singapore, 13358–13376. doi:10.18653/v1/2023.emnlp-main.825

[6] Jeff Johnson, Matthijs Douze, and Hervé Jégou. 2019. Billion-scale similarity search with GPUs. *IEEE Transactions on Big Data* 7, 3 (2019), 535–547.

[7] Vladimir Karpukhin, Barlas Oguz, Sewon Min, Patrick Lewis, Ledell Wu, Sergey Edunov, Danqi Chen, and Wen-tau Yih. 2020. Dense Passage Retrieval for Open-Domain Question Answering. In *Proceedings of the 2020 Conference on Empirical Methods in Natural Language Processing (EMNLP)*. Association for Computational Linguistics, 6769–6781. https://aclanthology.org/2020.emnlp-main.550

[8] Patrick Lewis, Ethan Perez, Aleksandra Piktus, Fabio Petroni, Vladimir Karpukhin, Naman Goyal, Heinrich Küttler, Mike Lewis, Wen-tau Yih, Tim Rocktäschel, and Sebastian Riedel. 2020. Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks. In *Advances in Neural Information Processing Systems (NeurIPS)*, Vol. 33. 9459–9474. https://proceedings.neurips.cc/paper_files/paper/2020/file/6b493230205f780e1bc26945df7481e5-Paper.pdf

[9] Yucheng Li, Bo Dong, Frank Guerin, and Chenghua Lin. 2023. Compressing Context to Enhance Inference Efficiency of Large Language Models. In *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing*, Houda Bouamor, Juan Pino, and Kalika Bali (Eds.). Association for Computational Linguistics, Singapore, 6342–6353. doi:10.18653/v1/2023.emnlp-main.391

[10] Shuwen-Cheng Lin, Akari Asai, Mingwei Li, Barlas Oguz, Jimmy Lin, Yashar Mehdad, Wen-tau Yih, and Xinying Chen. 2023. How to Train Your Dragon: Diverse Augmentation Towards Generalizable Dense Retrieval. In *Findings of the Association for Computational Linguistics: EMNLP 2023*. Association for Computational Linguistics, 1693–1712. https://aclanthology.org/2023.findings-emnlp.109

[11] Alex Mallen, Akari Asai, Victor Zhong, Rajarshi Das, Daniel Khashabi, and Hannaneh Hajishirzi. 2023. When Not to Trust Language Models: Investigating Effectiveness of Parametric and Non-Parametric Memories. In *Proceedings of the 61st Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, Anna Rogers, Jordan Boyd-Graber, and Naoaki Okazaki (Eds.). Association for Computational Linguistics, Toronto, Canada, 9802–9822. doi:10.18653/v1/2023.acl-long.546

[12] Zhuoshi Pan, Qianhui Wu, Huiqiang Jiang, Menglin Xia, Xufang Luo, Jue Zhang, Qingwei Lin, Victor Rühle, Yuqing Yang, Chin-Yew Lin, H. Vicky Zhao, Lili Qiu, and Dongmei Zhang. 2024. LLMLingua-2: Data Distillation for Efficient and Faithful Task-Agnostic Prompt Compression. In *Findings of the Association for Computational Linguistics: ACL 2024*, Lun-Wei Ku, Andre Martins, and Vivek Srikumar (Eds.). Association for Computational Linguistics, Bangkok, Thailand, 963–981. doi:10.18653/v1/2024.findings-acl.57

[13] Zhiruo Wang, Jun Araki, Zhengbao Jiang, Md Rizwan Parvez, and Graham Neubig. 2023. Learning to Filter Context for Retrieval-Augmented Generation. arXiv:2311.08377 [cs.CL] https://arxiv.org/abs/2311.08377

[14] Fangyuan Xu, Weijia Shi, and Eunsol Choi. 2024. RECOMP: Improving Retrieval-Augmented LMs with Context Compression and Selective Augmentation. In *The Twelfth International Conference on Learning Representations*. https://openreview.net/forum?id=mlJLVigNHp

[15] Chanwoong Yoon, Taewhoo Lee, Hyeon Hwang, Minbyul Jeong, and Jaewoo Kang. 2024. CompAct: Compressing Retrieved Documents Actively for Question Answering. In *Proceedings of the 2024 Conference on Empirical Methods in Natural Language Processing*. 21424–21439.

[16] Ori Yoran, Tomer Wolfson, Ori Ram, and Jonathan Berant. 2024. Making Retrieval-Augmented Language Models Robust to Irrelevant Context. In *The Twelfth International Conference on Learning Representations*. https://openreview.net/forum?id=ZS4m74kZpH

[17] Yujia Yu, Wei Ping, Zihan Liu, Bowen Wang, and Jun You. 2024. RankRAG: Unifying Context Ranking with Retrieval-Augmented Generation in LLMs. (2024). Manuscript in preparation.

Design of a 32 Channel 5G NR MIMO Base Station

K. Sai Praneeth, Jeeva Keshav Sattianarayanin, Sai Prasanth Kotturi, Rohit Singh, Aravind Gundlapalle, Siddharth Singh, Radha Krishna Ganti*
{praneethk,jeevakeshavs,ksaiprasanth7,rohitsingh}@smail.iitm.ac.in,{aravind,siddharth}@5gtbiitm.in,rganti@ee.iitm.ac.in
Indian Institute of Technology Madras
Chennai, Tamil Nadu, India

Abstract

The growth of 5G, paired with techniques like Massive MIMO, has resulted in high data rates with reduced latency and error rates. Such end-to-end systems are complex to build, since they involve multiple blocks/components to be integrated on hardware, each adhering to respective standards. This paper presents the design, hardware implementation, and evaluation of a complete 3GPP-compliant 5G New Radio (NR) stack, which includes a 32-channel scalable radio unit. Our 5G NR stand-alone base station architecture, deployed in real time, can connect to multiple commercially available mobile phones simultaneously. The deployment demonstrates downlink data rates of 1.23 Gbps in a lab setup and 820 Mbps in a field, for a 4 layer MIMO configuration. By developing such a 5G stack on re-configurable hardware like FPGAs, this work aims to bridge the gap between theory, simulation, and field deployment and provide insights into system-level design for future 5G and beyond systems.

Keywords

MIMO, 5G-NR, 3GPP, Testbed, Physical Layer, FPGA

1 Introduction

The evolution of cellular communications from 4G LTE to 5G NR [1] has introduced many capabilities that enable enhanced Mobile Broadband (eMBB), Ultra-Reliable Low-Latency Communication (URLLC), and massive Machine-Type Communication (mMTC) as core use cases. Achieving these use cases requires not only advancements in the physical layer, but also the implementation of a flexible and scalable Radio Access Network (RAN) architecture [2].

Notable 5G implementations leverage software-defined stacks such as OpenAirInterface (OAI)[3] and srsRAN[4], typically deployed on Universal Software Radio Peripheral (USRP) platforms. General-purpose signal processing frameworks such as GNU Radio [5] are also widely used to prototype and evaluate physical layer algorithms. For example, [6] benchmarked the downlink performance of the OAI codebase on a USRP X310, evaluating data rates under RF simulation, cabled RF setups, and over-the-air (OTA) transmissions using VERT2450 antennas. Similarly, [7] investigated mutual downlink interference between 4G and 5G cells using SDRs with open-source cellular software, while [8] addressed the challenges of real-time scheduling in industrial SDR-based systems. A sub-6 GHz 5G testbed based on srsRAN was developed in [9] using a USRP and a 5G UE, where uplink and downlink throughput, downlink reference signal received power (RSRP), and latency were optimized through modulation tuning and MIMO configuration.

This work complements the existing 5G development frameworks such as OAI and srsRAN by providing a newer end-to-end 5G system implementation. This FPGA-based implementation of an end-to-end 5G NR base station provides hardware scalability and opportunities for real-time field deployment.

The key contributions of this work are threefold: [1]

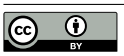
(1) A comprehensive FPGA-based end-to-end 5G NR MIMO hardware design and implementation, presented from the perspective of the RAN.
(2) A scalable multi-antenna radio unit architecture with detailed hardware design specifications, capable of supporting up to 32 RF channels while meeting stringent timing constraints.
(3) A system-level evaluation covering critical radio aspects such as antenna calibration, clocking architecture, different test scenarios, and performance results under realistic channel conditions.

Some key aspects of the 5G communication protocol are summarized as follows. The transfer of control and user data from the base station (BS) to the user equipment (UE) is referred to as downlink (DL), while the reverse direction is termed uplink (UL). In a 5G network, the protocol stack begins with the core network, which generates both control and user data. This information is processed sequentially by the Baseband Unit (BBU) and the Radio Unit (RU). Collectively, the BBU and RU form the gNodeB (gNB), which represents the entire radio access network (RAN) excluding the core. At the physical layer (L1), the transmitted bits and control information from higher layers are converted into waveforms during transmission, and the inverse operation is performed during reception. This is achieved using Orthogonal Frequency Division Multiplexing (OFDM) modulation and demodulation, as described in [10]. The resulting waveform is transmitted and received over the air (OTA) through the respective DL and UL channels.

The 5G NR RAN stack comprises the following layers:

(1) Physical (L1) layer - 3GPP compliant NR waveform generation with scalable multi-layer (MIMO) configuration.

*All authors contributed equally to this work



OpenRan '25, November 4-8, 2025, Hong Kong, China

ACM ISBN 979-8-4007-1977-6/2025/11
https://doi.org/10.1145/3737900.3770173

[1] This end-to-end system has been developed as a part of a project funded by the Department of Telecommunications (DOT), India, from 2018-2022 and has been enhanced since. The end-to-end system has been developed jointly by IIT Madras, IIT Kanpur, Sameer, IIT Bombay, and CEWiT. In this paper, we provide the general description and details of the Radio and the components of the stack/hardware that have been developed by IIT Madras, with a focus on the Radio.

Table 1: Features of the 5G NR Base station

Parameter	Value	Remarks
TDD Structure	D,D,D,S,U, U,D,D,D,D D,D,D,S,U, U,D,D,D,D	Time Division Duplexing with slots for DL (D), UL (U) or transition (S) slots
SCS/ Numerology (μ)	30 KHz/ $\mu = 1$	Sub-carrier spacing (SCS) given by SCS = 2^{μ}.15 KHz
BandWidth/ Frequency	100MHz/ 3.3 to 3.8 GHz	273 Resource Blocks (N_{RB})/ N78 Band (FR1)
Frame and Slot Duration	10ms and 0.5ms	20 slots/frame 14 OFDM symbols (N_s)/slot
M	BPSK, M-QAM	Supported Modulation Schemes. $M \in \{4, 16, 64, 256\}$
Splits	6, 7.2B	FAPI [11] and ORAN [12]
DL Channels	PBCH, PDSCH, PDCCH	Broadcast Channel Data Channel Control Channel
DL Reference Signals	CSI-RS, DMRS, PTRS	CSI Estimation Channel Estimation Phase Estimation
UL Channels	PRACH PUSCH, PUCCH	Initial Access Data Channel, Control Channel
UL Reference Signals	DMRS, SRS	Channel Estimation Channel Quality Estimation
Radio Specifications	• Height: 740 mm, Width: 468 mm, Depth: 253 mm • Weight: 31 Kg • Enclosure category: IP65 • Health monitoring: Voltage, current and temperature • Calibration support • Lightning protection: Line to ground: 15KV, Line to line 7.5KV • Nominal power supply : −48V	
Other features	**1**. Support for 8-layer ($L = 8$) MIMO with real -time beamforming coefficients for precoder, **2**. CSI feedback based adaptive modulation **3**. GPS synchronization **4**. Supports flexible TDD structure.	

(2) MAC (L2) layer - Scheduler for dynamic resource allocation with Hybrid Automatic Repeat Request (HARQ) management for connected UEs
(3) RLC/PDCP/RRC (L3) layer - Higher layer protocols ensuring reliable data transfer and seamless UE connectivity.

The gNB interfaces with the 5G core network via the NG-RAN interface. All the RAN layers mentioned above are implemented in the system described in this paper with a focus on the physical layer. Implementation-specific choices and features of L1 are summarized in Table 1.

2 5G Radio Access Network (5G RAN)

The 5G RAN provides the wireless interface between the UE and the 5G core. In our system design, the 5G RAN is composed of two main components - 1) The Base Band Unit (BBU) and 2) The Radio Unit (RU), as shown in Fig. 2.

2.1 Base Band Unit (BBU)

The BBU consists of the L2/L3 protocol stack, the L1 stack [13], and the associated hardware to execute them. In the ORAN terminology,

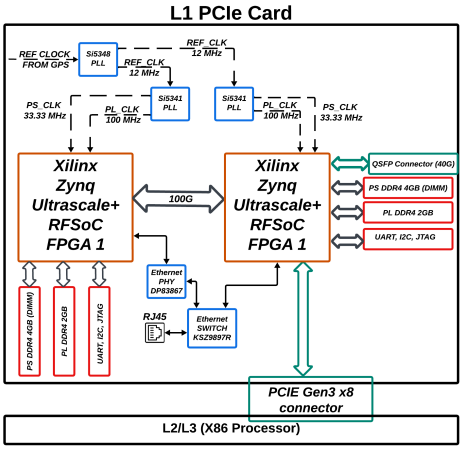


Figure 1: Hardware block design of custom L1 PCIe card for High PHY processing

the BBU incorporates the CU and DU (Centralized and Distributed Unit) functions. The hardware depiction of BBU is shown in Fig. 1. The L2/L3 layers implement the PDCP, RLC, RRC, and MAC protocols. The interface between MAC and PHY is realized through the FAPI protocol [11] over Peripheral Component Interconnect Express (PCIe), corresponding to the MAC–PHY functional split (split-6) in split-stack architectures. In our implementation, L2/L3 is executed on an Intel Xeon Gold x86 processor, while the L1 stack is implemented on FPGAs.

2.1.1 Synchronization and Hardware Architecture: To ensure synchronization between the L2/L3 and L1 layers, we introduce a 12-byte control packet, referred to as the slot indication, which signals the start of each slot. This packet is derived from the 1 Pulse Per Second (1 PPS) reference provided by the GPS, thereby ensuring slot-level alignment across layers. The L1 BBU stack executes on a custom PCIe FPGA card, which is a standard full-height full-width PCIe form-factor card with an $x8$ lane configuration, compatible with standard server platforms. This card is built with two Xilinx Zynq Ultrascale+ RFSoC FPGAs - one dedicated to the transmit chain processing modules and the other to the receiver chain. The choice of RFSoC is primarily motivated by the requirement of high speed SD-FEC blocks, which are available as hard IPs. These FPGAs are interconnected via a 100G Ethernet link. The requirement of two FPGAs was a logical decision based on the resource budget for the entire system.

The BBU and RU are interfaced using a 40G Ethernet sub-system over an optical link. The physical layer stack, fully developed in Verilog, is deployed across both FPGAs. For system observability and debugging, logging mechanisms are implemented at both the FAPI interface (between L2 and the FPGA) and within the FPGA fabric. On-chip Integrated Logic Analyzer (ILA) probes are utilized to monitor critical signals such as module timing, module states, and resource grid (RG)[2] driven packet counts to facilitate real-time verification of signal integrity across the Tx and Rx chains.

[2]A resource grid (RG) corresponds to an $N_{RB} \times N_s \times N_L$ matrix (e.g., 273 RBs × 14 OFDM symbols × 8 layers in our system), constituting a slot.

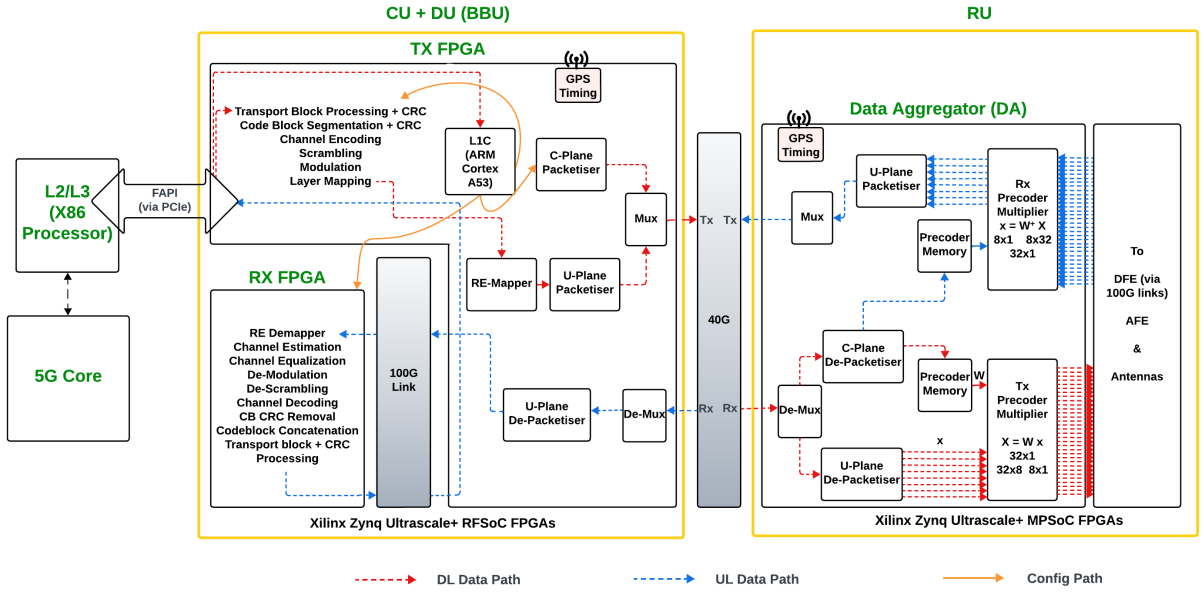


Figure 2: Data and Control flow in the BBU and the RU: L2/L3 processing, L1 processing RTL blocks, L1 Controller and GPS sychronisation modules

2.1.2 Data and Control Processing: In our system, the entire L1 physical layer is implemented in Register Transfer Language (RTL) as a modular design comprising multiple blocks with specific functionality as shown in Fig. 2. Each module is designed to accept one or more inputs per slot: (i) user data from preceding modules, and (ii) module-specific configuration parameters (configs). The user data processing is performed on the FPGAs, and the configuration packet generation is handled by the L1 Controller as defined below:

Transmitter Chain: The transmitter chain maps data bits received from L2/L3 (via the FAPI protocol) to IQ samples in accordance with [14, 15, 16, 17]. Processing begins with appending CRC bits for the transport blocks (TB), segmenting them into smaller code blocks (CBs), further appending CRC bits for the CB, and concludes with IQ mapping to the resource grid (RG). Subsequent stages of the Tx chain are executed in the RU. The BBU and RU communicate over a 40G fronthaul link, which transports IQ samples as user-plane (U-plane) data and precoder coefficients as control-plane (C-plane) data. The C and U-plane framers/packetisers attach relevant header information during DL, while the de-framers/depacketisers remove these headers during UL. These operations comply with ORAN FrontHaul (FH) 7.2 CAT-B intra-PHY split specifications[12]. The ORAN FH functionality is implemented on the TX FPGA.

Receiver Chain: The receiver chain, implemented on the RX FPGA, performs inverse signal processing operations on the received RG to reconstruct the data bits transmitted by the user. The chain incorporates key algorithms such as channel estimation, Minimum Mean Squared Error (MMSE) equalization, Signal-to-Interference-Noise Ratio (SINR) estimation, Forward Error Correction (FEC), CRC validation, etc. The decoded bits are then forwarded to the higher layers.

L1 Controller (L1C): The L1 Controller is responsible for generating module-specific configuration packets for the RTL modules in the transmitter and receiver chains. The L1 Controller receives the unprocessed control information from L2/L3 layers via the FAPI protocol [11] (through the PCIe interface), which is then parsed, and the appropriate configuration packets are generated and transferred to the Programming Logic (PL) of the FPGA fabric via DMA. Since the parsing of these FAPI packets is highly sequential and dynamic, it is implemented separately on the ARM Cortex-A53 processor (part of the FPGA SoC). To ensure timely processing of the transmitter and receiver chain modules,

(1) The L1 Controller config generation and transfer procedure must complete within the first 100 μs of each slot. Hence, this controller is implemented as a bare-metal program to guarantee deterministic latency.
(2) A two-slot look-ahead notion, as explained in Section VI of [13], is followed.

2.2 Radio Unit (RU)

The Radio Unit (RU), shown in Fig. 3, is deployed at the antenna site and performs the lower PHY functions, including precoding. It converts digital baseband signals received from the BBU into analog RF signals for over-the-air (OTA) transmission, and vice versa for reception. The RU operates from an industry-standard -48V supply. Its protection circuitry is designed to withstand reverse polarity, over-current, over-voltage, and lightning surges. The full-scale power of the RU is 768W. The RU comprises the following FPGA-based units:

Data Aggregator (DA): The DA, depicted in Fig. 4, is a custom-designed Xilinx Zynq UltraScale+ MPSoC FPGA that integrates both Programmable Logic (PL) and a Processing System (PS). It

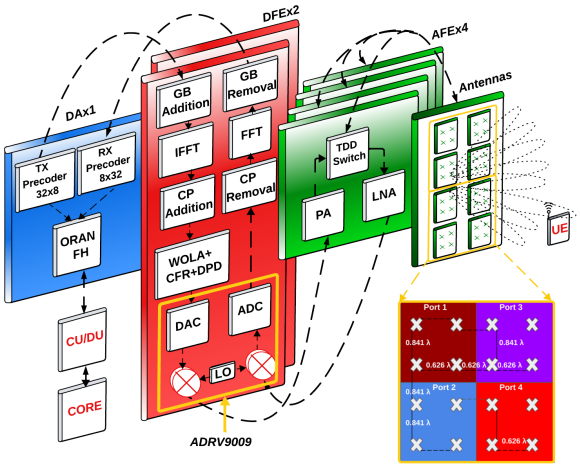


Figure 3: FPGA-based functional blocks for low-PHY processing in the RU: 1 Data Aggregator (DA) → 2 Digital Front Ends (DFEs) → 4 Analog Front Ends (AFEs) → 32 Antenna Elements

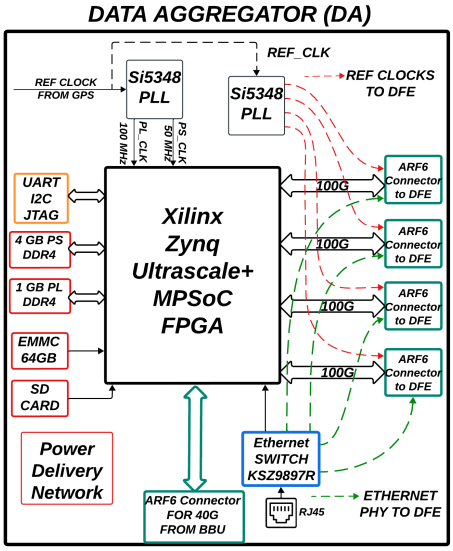


Figure 4: Hardware block design of custom DA card with an interface to a maximum of four DFEs

aggregates and distributes baseband data across multiple Digital Front Ends (DFEs), depending on the system configuration: a single DA interfaces with 1, 2, or 4 DFEs to make a 16, 32, or 64-channel system, respectively. The DA incorporates O-RAN fronthaul de-framers/framers for DL and UL, respectively, responsible for removing and padding headers before passing data and control information to the Tx precoder and from the Rx precoder. The precoders scale and steer beams towards intended users. In the current deployment, a 32-channel linear precoder [18] is implemented. Both Tx and Rx precoders comprise two main entities: (i) a memory module that stores real-time precoder coefficients and maps them to the RG with user data, and (ii) a multiplier that combines coefficients with user data to direct signals toward specific antennas. The precoder maps L resource elements to N_t transmit antenna ports using precoder coefficients derived from channel state information (CSI) feedback from the UE. The precoder matrix is generated according to the Precoder Matrix Indicator (PMI) and CSI codebook specified in [17]. The current system supports real-time update of up to 50 precoder coefficient matrices per slot. The efficient utilization of the precoder optimizes data rates by mitigating the adverse effects of wireless channel impairments.

Digital Front-End (DFE): The DFE, shown in Fig.5, is a high-speed digital PCB built around a Xilinx UltraScale+ MPSoC FPGA[19], which integrates both PL and a PS. The PS includes a quad-core 64-bit ARM Cortex-A53 Processing Unit (APU) running a Linux-based system controller. Each DFE incorporates ADRV9009 transceivers that provide digital-to-analog (DAC) and analog-to-digital (ADC) conversion, filtering, and RF up/down conversion with support for carrier frequencies from 75 MHz to 6 GHz. These transceivers also support variable gain control. A single DFE with 16 RF transmit and receive chains, performs baseband functions such as IFFT/FFT, guard band (GB) addition/removal, and cyclic prefix (CP) insertion/removal. Additional features include Weighted Overlap and Add (WOLA), Crest-Factor Reduction (CFR), followed by Digital Pre-Distortion (DPD), which collectively mitigate the high peak-to-average power ratio (PAPR) and prevent power amplifier saturation. Each DFE interfaces with two AFEs, yielding a total of four AFEs in the 32-channel radio system.

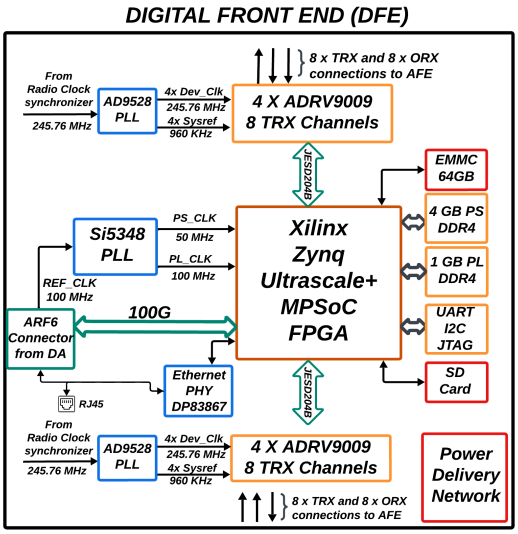


Figure 5: Hardware block design of custom DFE card with transceivers, PLLs, and other peripherals

Analog Front-End (AFE): The AFE, shown in Fig. 6, comprises 8 TRX and 8 ORX channels, with multistage amplification on both the Tx and Rx paths. On the Tx side, a driver power amplifier (DPA) followed by a Power Amplifier (PA) provides a cumulative gain of

50 dB. On the Rx side, a TDD switch and low-noise amplifier (LNA) stages contribute a combined gain of 55 dB. A TDD switch is used to facilitate efficient switching between downlink transmission and uplink reception paths. This enables the device to seamlessly transition from PA to LNA mode during the uplink slots. Each AFE is capable of delivering 33dBm (2W) per channel on the output power.

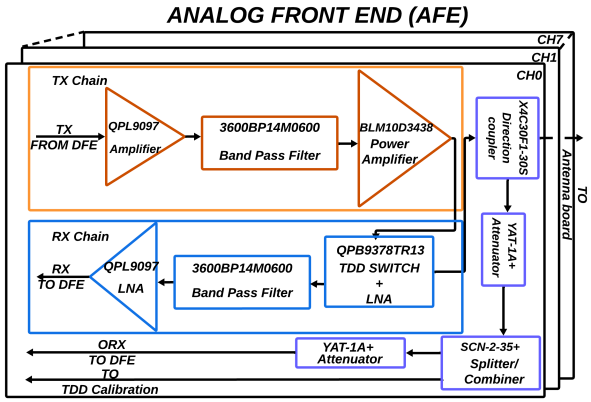


Figure 6: Hardware block design of custom AFE with amplifiers and the feedback path.

Antenna Elements: For the 32-channel RU, the antenna configuration follows $N_1 \times N_2 = 8 \times 4$ configuration, consistent with [17]. The antenna panel consists of 8 sub-panels, each containing 4 dual-polarized elements ($+45^\circ$ and -45°). The inter-element spacing in the proposed antenna array, as shown in Fig.3 is carefully chosen as 0.626λ and 0.841λ to achieve an optimal balance between mutual coupling suppression, spatial correlation reduction, and eliminating grating lobes. The vertical spacing between the elements is 69.2 mm, and the horizontal spacing is 51.5 mm. Each antenna PCB contains a 2×2 antenna array with a 4T4R configuration, which scales to a 32T32R system with an 8-panel configuration. The PCBs are fabricated on standard two-layer copper FR4 dielectric substrates.

3 Clocking and Antenna Calibration

This section discusses two critical aspects of the RAN implementation: the clocking structure and antenna calibration.

3.1 Clocking Structure

The clocking structure is fundamental to base station design, as it directly impacts RF performance such as Bit Error Rate (BER), Error Vector Magnitude (EVM), and overall signal quality. Any degradation in clock precision propagates to the system level Key Performance Indicators (KPI) [20]. To synchronize DU and RU, we employ the use of a GNSS receiver [21] as illustrated in Fig. 7. The GNSS receiver synchronizes with the available satellites and provides a reference clock of 9.592 MHz and 1 pulse per second (PPS). In the RU, DA houses the GNSS receiver.

We chose a Si5348 PLL (on the DA board) to generate FPGA clocks for PS, PL, and Gigabit Transceivers(GT) because it can deliver sub-100 fs RMS jitter clocks, which are essential for high-speed 40G and 100G SerDes interfaces. To ensure synchronization of the DFE, this PLL provides a reference clock source of 100 MHz to the DFEs' Si5348, which further generates all the required clocks in the DFE. The DFE contains ADRV9009 transceiver chips [22] for interfacing between the digital and analog domains. This transceiver chip requires two clocks: a device clock (245.76 MHz LVDS) and a SYSREF (960 KHz). The device clock is used to generate internal clocks for data converters, RF up-converters, and the serial interface. Whenever requested, SYSREF (a pulse train) is sent to the transceiver and is used to reset the internal clock dividers and PLLs to ensure multi-chip synchronization. The Si5348 in DA provides 122.88 MHz reference clock to the Radio clock synchronizer. The AD9528 PLL in the Radio clock synchronizer locks to this reference and generates the necessary device and SYSREF clocks to the AD9528 in the DFE board. This PLL will provide both device and SYSREF clocks to all the transceivers in the DFE board.

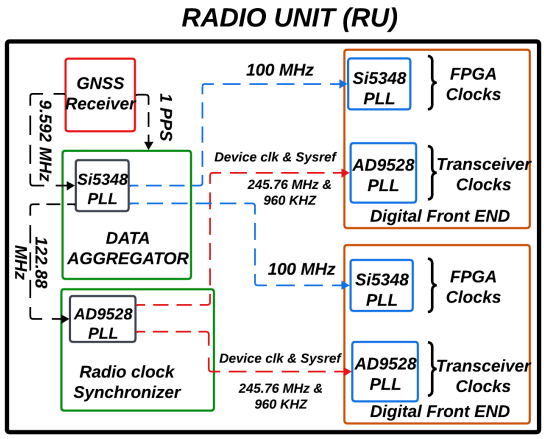


Figure 7: Clocking structure of RU

3.2 Antenna Calibration

The deployment of large antenna arrays in 5G RU facilitates the implementation of massive MIMO and beamforming. The performance of these techniques relies on the coherent combination of signals transmitted and received across multiple antenna elements. In practice, non-ideal characteristics in the RF front-end introduce phase, delay, and amplitude mismatches between RF paths. These irregularities must be meticulously calibrated to preserve the system's performance. For the calibration across multiple RF paths, we employ one of the receiver chains as a reference feedback path. This receiver continuously monitors the analog data transmitted to the antenna and feeds it back to the digital chain via the Rx or ORx path iteratively across all the transmit paths. The differences in amplitude, phase, and delay among the channels are estimated and compensated for subsequent transmissions to ensure accurate beam forming.

4 Testing Procedure and De-bugging Tools

The testing eco-system explained in Fig. 8 to evaluate the system is two-fold as explained in 4.1 and 4.2

4.1 Simulation level testing

Before building the hardware chain, all the modules/chains are simulated using MATLAB. This is followed by RTL design level simulations using the Xilinx Vivado platform. Each module/chain is validated against MATLAB-generated test cases and then integrated.

4.2 Hardware testing

The transmit and receive chains are first validated separately on hardware using Vector Signal Analyzers (VSAs) and Vector Signal Generators (VSGs), respectively, in conductive mode. Parameters such as chain latency, backpressure, and the continuous flow of data across multiple configurations are validated. Radio equipment with duly phase-calibrated antennas/channels is tested by collecting data across each channel and cross-referencing it with simulation data for benchmarking. For DL transmissions, the OTA waveform, bandwidth, spectrum, and modulation order of data, control, and synchronized allocations for various users are observed in the VSA. Once the hardware is field-proof and deployed, multiple tests are performed by connecting several UEs to the network. In case of any erroneous results, the data collected through various tools, illustrated in Fig. 8 as part of the testing ecosystem, is fed back to the simulation chains for further analysis.

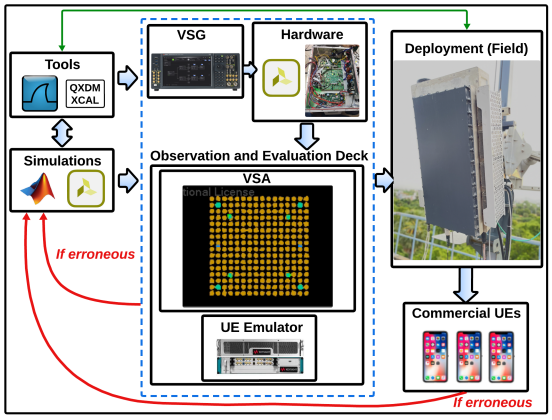


Figure 8: Testing eco-system of 5G NR base station

5 Results

An end-to-end 5G NR system with focus on a physical layer BBU and RU are designed for the specifications as mentioned in Table 1 with the best throughput values for different layers summarized in the Table 2. It should be noted that the throughput results pertain to a single layer on the UL, four layers on the DL (commercial UEs capability), and the choice of Coordinated Universal Time (UTC) synchronized TDD structure. Commercial UEs like Samsung S22, Samsung A23, Samsung M55, OnePlus 8T, Telit 5G Modem, and Quectel 5G Modem have all been successfully latched and tested with our gNB. Figure 9 depicts the DL and UL throughput values as a function of distance (in meters), with a single UE latched to our network.

Table 2: Testing Results with 1 UE in different scenarios

# Layers (L)	Test Scenario	Avg RSRP (dBm)	Best Throughput DL/UL (Mbps)	
			Observed	Theoretical
DL = 1 UL = 1	Lab	-83	308 / 58	371 / 65
	Field	-90	276 / 45	371 / 65
DL = 4 UL = 1	Lab	-84	1230 / 58	1281 / 65
	Field	-89	820 / 45	1281 / 65

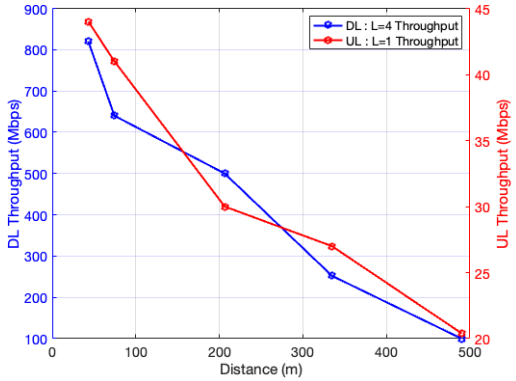


Figure 9: Throughput vs distance for 1 UE with 4 layers on the downlink and 1 layer on the uplink

6 Conclusion

In this paper, we present a framework for a 5G NR base station that adheres to stringent timing constraints, such as a bounded slot duration of 0.5 ms, on an FPGA. We provide an overview of the architecture, including the clock structure, and a more detailed overview of the 32-channel implementation. Our results demonstrate that this end-to-end integrated setup can successfully connect with 30+ commercial UEs, providing good throughputs and connectivity across all UEs. The modular implementation of this architecture, with its flexible TDD structure, bandwidth, and antenna configurations, makes it suitable for standard deployments and research. Future directions include developing a database to efficiently collect data from various nodes of the integrated chain, thereby enabling the training of AI/ML models for diverse applications.

7 Acknowledgments

This work was funded by the Ministry of Electronics and Information Technology (MeitY), Government of India, through the project, "Next Generation Wireless Research and Standardization on 5G and Beyond".

References

[1] Stefan Parkvall, Erik Dahlman, Anders Furuskar, and Mattias Frenne. 2017. NR: The New 5G Radio Access Technology. *IEEE Communications Standards Magazine*, 1, 4, 24–30. doi:10.1109/MCOMSTD.2017.1700042.

[2] Patrick Marsch, Icaro Da Silva, Omer Bulakci, Milos Tesanovic, Salah Eddine El Ayoubi, Thomas Rosowski, Alexandros Kaloxylos, and Mauro Boldi. 2016. 5G Radio Access Network Architecture: Design Guidelines and Key Considerations. *IEEE Communications Magazine*, 54, 11, 24–32. doi:10.1109/MCOM.2016.1600147CM.

[3] Open Air Interface. 2025. OAI. Retrieved Aug. 17, 2025 from https://openairinterface.org.

[4] SRS RAN. 2025. SRS. Retrieved Aug. 17, 2025 from https://www.srsran.com/5g.

[5] GNU Radio. 2025. GNU. Retrieved Aug. 17, 2025 from https://www.gnuradio.org.

[6] Sai Shruthi Nakkina, Sudhakar Balijepalli, and Chandra R. Murthy. 2021. Performance Benchmarking of the 5G NR PHY on the OAI Codebase and USRP Hardware. In *WSA 2021; 25th International ITG Workshop on Smart Antennas*, 1–6.

[7] Nadia Yoza Mitsuishi, Yao Ma, and Jason Coder. 2024. 5G NR and LTE Downlink Coexistence Measurements Using Software-Defined Radios. In *ICC 2024 - IEEE International Conference on Communications*, 1273–1279. doi:10.1109/ICC51166.2024.10622536.

[8] Tianyu Zhang, Jiachen Wang, Xiaobo Sharon Hu, and Song Han. 2023. Real-Time Flow Scheduling in Industrial 5G New Radio. In *2023 IEEE Real-Time Systems Symposium (RTSS)*, 371–384. doi:10.1109/RTSS59052.2023.00039.

[9] Ali Ahmed, Atef A. Aburas, Khalid Al-Mashouq, Akram Aburas, Mohammad Yaser Ammar, and Asad Ali Khan. 2024. Optimizing Coverage and Capacity in 5G ORAN Small Cells with srsRAN Test Bed. In *2024 IEEE Future Networks World Forum (FNWF)*, 462–469. doi:10.1109/FNWF63303.2024.11028796.

[10] Ali A Zaidi, Robert Baldemair, Hugo Tullberg, Hakan Bjorkegren, Lars Sundstrom, Jonas Medbo, Caner Kilinc, and Icaro Da Silva. 2016. Waveform and Numerology to support 5G Services and Requirements. *IEEE Communications Magazine*, 54, 11, 90–98.

[11] Clare Somerville. 2021. *5G FAPI A PHY API Specification*. Small Cell Forum.

[12] WG-4. 2023. O-RAN Working Group 4 (Open Fronthaul Interfaces WG) Control, User and Synchronization Plane Specification. Technical Specification (TS). O-RAN.WG4.CUS.0-v12.00. Open RAN Alliance.

[13] Jeeva Keshav S, K B S D Sai Praneeth, K B S M D S Sai Prasanth, Rohit Singh, and et.al. 2024. Physical Layer Design of a 5G NR Base Station. In *2024 National Conference on Communications (NCC)*, 1–6. doi:10.1109/NCC60321.2024.10485845.

[14] 3GPP. 2018. Physical Channels and Modulation. Technical Specification (TS) 38.211. Version 15.3.0. 3rd Generation Partnership Project (3GPP).

[15] 3GPP. 2021. Multiplexing and Channel Coding. Technical Specification (TS) 38.212. Version 16.5.0. 3rd Generation Partnership Project (3GPP).

[16] 3GPP. 2021. Physical Layer Procedures for Control. Technical Specification (TS) 38.213. Version 16.2.0. 3rd Generation Partnership Project (3GPP).

[17] 3GPP. 2020. Physical Layer Procedures for Data. Technical Specification (TS) 38.214. Version 16.2.0. 3rd Generation Partnership Project (3GPP).

[18] Kalyani Bhukya, Shahid Aamir Sheikh, and Radha Krishna Ganti. 2025. Precoder Implementation and Optimization in 5G NR Massive MIMO Radio. In *2025 National Conference on Communications (NCC)*, 1–6. doi:10.1109/NCC63735.2025.10983419.

[19] AMD. 2024. UltraScale Architecture PCB Design User Guide (UG583). Technical Information Report. Revision 1.28. AMD.

[20] 3GPP. 2021. Management and Orchestration; 5G End to End Key Performance Indicators (KPI). Technical Specification (TS) 28.554. Version 16.7.0. 3rd Generation Partnership Project (3GPP).

[21] Lihua Yang, Bing Li, and Yue Yin. 2021. GNSS Aided Precision Multi-clock Synchronization. In *2021 4th International Conference on Hot Information-Centric Networking (HotICN)*, 63–67. doi:10.1109/HotICN53262.2021.9680871.

[22] Analog Devices. 2019. ADRV 9009 DataSheet. Technical Report. Analog Devices.

Safety and Risk Pathways in Cooperative Generative Multi-Agent Systems: A Telecom Perspective

Zeinab Nezami, Shehr Bano, Abdelaziz Salama, Maryam Hafeez, Syed Ali Raza Zaidi
School of Electrical and Electronic Engineering, University of Leeds
Leeds, United Kingdom

Abstract

Generative multi-agent systems are rapidly emerging as transformative tools for scalable automation and adaptive decision-making in telecommunications. Despite their promise, these systems introduce novel risks that remain underexplored, particularly when agents operate asynchronously across layered architectures. This paper investigates key safety pathways in telecom-focused Generative Multi-Agent Systems (GMAS), emphasizing risks of miscoordination and semantic drift shaped by persona diversity. We propose a modular safety evaluation framework that integrates agent-level checks on code quality and compliance with system-level safety metrics. Using controlled simulations across 32 persona sets, five questions, and multiple iterative runs, we demonstrate progressive improvements in analyzer penalties and Allocator–Coder consistency, alongside persistent vulnerabilities such as policy drift and variability under specific persona combinations. Our findings provide the first domain-grounded evidence that persona design, coding style, and planning orientation directly influence the stability and safety of telecom GMAS, highlighting both promising mitigation strategies and open risks for future deployment.

CCS Concepts

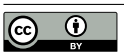
• **Computing methodologies** → **Multi-agent planning**; **Simulation evaluation**; • **Networks** → **Wireless access networks**.

Keywords

Generative Multi-Agent Systems, GMAS, Telecommunications, ORAN, Safety Evaluation, Miscoordination

1 Introduction

GMAS offer transformative potential for critical infrastructure by enabling scalable automation, adaptive decision-making, and collaborative intelligence [1]. In telecommunications, these systems promise self-evolving networks [2–4], where agents autonomously perceive, reason, and adjust network behavior in real time. However, GMAS introduce novel classes of risk that differ from those in single-agent or traditional automation frameworks [5], which remain largely unexplored. The scarcity of real-world deployments combined with unresolved safety challenges in even single-agent AI, highlights the urgent need to identify and mitigate emergent risks prior to widespread adoption [5].



OpenRan '25, November 4-8, 2025, Hong Kong, China

ACM ISBN 979-8-4007-1977-6/25/11
https://doi.org/10.1145/3737900.3770174

As wireless communication evolves toward self-evolving 6G networks, AI-driven systems will autonomously integrate telemetry, user intent, and environmental signals to refine internal policies, decision logic, and workflow across layers [2, 3, 6]. Open Radio Access Network (O-RAN) [7], together with large language model (LLM)-enhanced intelligence [8, 9], enables dynamic, programmable control, allowing networks to adapt to both operational and societal demands. Yet existing multi-agent evaluation frameworks [10–12] largely emphasize social interaction, persona adherence, or game-theoretic reasoning, while overlooking operational risks in high-stakes domains.

Despite their promise, GMAS are vulnerable to three primary failure modes [5]: miscoordination, conflict, and collusion. This paper focuses on miscoordination, which arises when agents act on incomplete or inconsistent information, leading to incompatible actions. For example, in an O-RAN deployment, one agent may reserve spectrum based on local telemetry while another follows a conflicting plan derived from a different state, causing resource contention or dropped connections. Such failures emerge naturally from asynchronous actions, divergent tool outputs, or mismatched knowledge contexts, without requiring adversarial intent. Information asymmetry is particularly pronounced in retrieval-augmented generation (RAG) setups, where agents query distinct document stores or maintain non-overlapping memory contexts. Given their layered architecture, real-time constraints, and safety-critical operations, telecom systems provide a compelling testbed for studying these dynamics.

This work proposes a GMAS for network orchestration and automation in O-RAN, and uses it as a testbed to develop and validate a safety evaluation framework for cooperative GMAS. Our framework evaluates agents that share a common objective while executing modular tasks through distributed roles with asymmetric context. Our contributions are as follows: (i) Safety evaluation framework: We design a domain-agnostic framework that combines agent-level checks with system-level safety metrics, and demonstrate its application in O-RAN and RAN Intelligent Controller (RIC) contexts. (ii) Knowledge–persona integration: We combine heterogeneous knowledge layers (RAG and graph-RAG) with systematic persona variation, and show how these dimensions jointly influence safety, alignment, and reproducibility in multi-agent systems. (iii) Proposed GMAS for telecom orchestration: We implement a cooperative GMAS for O-RAN automation, providing a concrete instantiation of our evaluation framework. (iv) Empirical insights: Through multi-run experiments, we uncover progressive improvements in GMAS performance, agent consistency, and embedding stability. (v) Open resources: We release a GitHub repository containing datasets and code to support reproducibility.

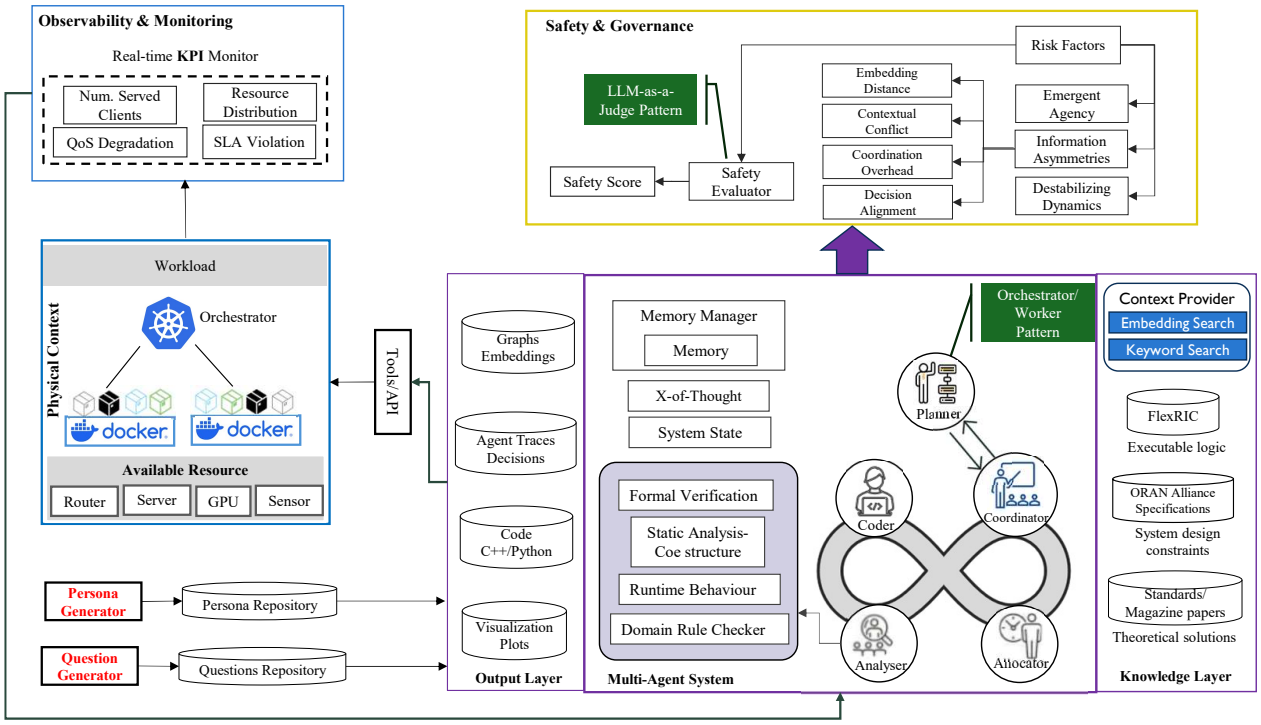


Figure 1: Architecture of safety framework for generative-based multi-agent systems in O-RAN setup

The paper is organised as follows: Section 2 reviews related work, Section 3 presents the GMAS and safety framework, Section 4 reports experiments and results, and Section 5 concludes with future directions.

2 Related Work

Generative AI (GenAI) has demonstrated strong potential in automating telecom and reducing human effort in tasks such as troubleshooting recommendations [13], code refactoring [14], and network configuration [15, 16]. While recent work has explored GMAS for telecom, the emphasis has largely been on benchmarking [17] and automation [18] rather than safety. To the best of our knowledge, there is no dedicated research addressing safety risks in GMAS for telecom. Related discussions have instead focused either on the safety of GenAI in other domains, or on artificial intelligence safety for telecom more generally. We review three strands of related work: (i) GMAS for telecom, (ii) safety in GenAI-based single-agent systems across domains, and (iii) safety evaluation for GMAS frameworks in telecom.

Panek et al. [19] propose a modular framework for cloud-native 5G networks that employs chain-of-thought reasoning before delegating tasks to cloud management tools. Zou et al. [20] introduce an edge-based multi-model approach that deploys LLM on wireless devices rather than a central server, reducing reliance on centralized infrastructure but increasing computational overhead. Similarly, Li et al. [21] design a multi-agent system for agile and autonomous telecom workflows, where specialized agents handle prompt engineering, fine-tuning, and resource optimization to enhance adaptability. These studies demonstrate the feasibility of GenAI-based multi-agent orchestration in telecom, but none systematically addresses safety risks. In telecom specifically, [22] highlight the need for system-level safety and runtime assurance in GenAI-based multi-agent systems, urging development of compositional explainability and robust assurances across heterogeneous models.

Freitas et al. [23] survey safety risks such as hallucination, misinformation, and disinformation, auditing commercial chatbots in a mental health context and exposing risky response patterns. Wang et al. [24] propose a proactive multi-agent reinforcement learning (MARL) framework integrated with GenAI to handle high dimensionality and non-stationarity, highlighting safety concerns unique to MARL environments. Bellay et al. [25] further outline risks for multi-agent systems, including instability, recursive behavior, and incomplete contextual awareness, and propose system-level strategies to increase control over the generative context. These studies illustrate growing recognition of GenAI safety risks, but they primarily focus on single-agent contexts or general MARL environments, rather than domain-specific, high-stakes infrastructures like telecom.

Wang et al. [11] propose a benchmark for assessing social interaction capabilities of generative agents through objective action-level metrics. Samuel et al. [10] introduce PersonaGym, a dynamic evaluation framework for agents, alongside an automatic, human-aligned metric for measuring persona adherence. Mao et al. [12] present a simulation framework leveraging LLM agents for game theory research in abstract socioeconomic scenarios. These frameworks

advance the evaluation of language-agent interactions, but their emphasis is on social dynamics and persona alignment rather than safety in mission-critical domains. In summary, prior studies establish the feasibility of GMAS in telecom and highlights safety challenges in other domains, but there remains a gap in domain-specific frameworks that systematically evaluate risks such as miscoordination and information asymmetry in telecom multi-agent settings. Our work addresses this gap by proposing an autonomous evaluation framework that integrates safety, technical correctness, and feasibility into the assessment of GMAS for telecom networks.

3 System Design

This section presents the proposed GMAS and the safety evaluation framework for safe agent governance and execution. The framework supports deployment across multiple telecom layers (e.g., edge, RIC, and core orchestration), where agents may operate with differential visibility or symmetric context. As shown in Fig. 1, the GMAS is organized into three functional subsystems and supporting modules. The system is triggered by a systematically generated question, framing a problem or request in the network, which is first received by the Coordinator Agent to initiate orchestration.

- **Orchestration**: Anchored by the Coordinator Agent, which manages run life cycle, selects solution paths proposed by the Planner Agent, and orchestrates downstream execution. The Planner and Coordinator leverage Tree-of-Thoughts (ToT) [26], which extends LLM reasoning by exploring multiple solution paths, self-evaluating intermediate steps, and selecting the most promising trajectory. ToT enables both correction of poor initial decisions and robustness through exploration, thereby improving reliability in telecom planning tasks and RAN optimization under uncertain patterns.
- **Execution**: Comprises the Resource Allocator Agent, which generates RIC resource allocation plans in pseudo-code, and the Coder Agent, which translates these plans into executable code.
- **Analysis**: This involves the Analyzer, which performs static analysis, policy compliance, runtime monitoring, and formal verification, with failed checks triggering targeted refinements. Complementing this, the Safety and Governance module tracks agent runs, interactions, and knowledge asymmetries to detect risks and miscoordination, ensuring safe operation at the system level.

Personas: It define the role and behavioral orientation of each agent in the GMAS ecosystem. Beyond user-facing interactions, personas also shape reasoning. prior work [10, 12] shows that role-specific personas can improve reasoning compared to zero-shot prompting, though poorly designed personas may introduce bias. In our system, personas are generated automatically using an LLM, guided by each agent's responsibilities. We evaluate combinations of personas across agents to study how diversity influences individual actions and overall system performance.

Knowledge Layer: RAG and GraphRAG In addition to persona-driven behavior, agent reasoning is refined through domain-grounded knowledge. We integrate both RAG [27] and GraphRAG via a context provider interface: RAG retrieves unstructured documents, while GraphRAG supplies structured relational context from a telecom-specific knowledge graph. The Planner leverages GraphRAG (standards, research papers) to generate multi-step solution paths, which the Coordinator selects for execution. Downstream, the Resource Allocator applies graph-derived O-RAN specifications for compliant strategies, and the Coder retrieves from the FlexRIC codebase to produce executable, context-aware code.

Memory and State Management: We employ **episodic memory**, which captures the past actions of each agent. This memory guides subsequent decisions by allowing agents to recall how earlier tasks were approached and refined, thereby reducing repetition and improving task efficiency. The *Memory Manager* maintains per-agent trajectories, including inputs, thought summaries, and refinement reasons. State artifacts—such as inputs, outputs, embeddings, metrics, and errors—are persisted as JSON and can be aggregated into CSVs for analysis or dashboards. This structure ensures that agents operate not only with domain knowledge but also with continuity of experience and traceable decision-making.

Analyzer and Safety Pipeline The Analyzer Agent performs multi-dimension validation across:(i) **Static checks**: e.g., Pylint, Bandit for Python; Clang-based tools for C++. (ii) **Policy compliance**: AST-based enforcement of constraints such as forbidden APIs, and action conflicts. (iii) **Runtime monitoring**: sandboxed execution and KPI tracking (e.g., throughput, latency). (iv) **Formallite verification**: lightweight guarantees via type hints, exception handling, and deterministic execution patterns. If the checks fail, the refinement loop triggers targeted re-execution from the relevant agent. The safety pipeline is responsible for analyzing and monitoring potential risk pathways to ensure safe operation of the GMAS. Distinct from the Analyser Agent, the pipeline traces all runs, actions, and interactions between agents, examining how system variables including personas and asymmetric knowledge may contribute to the miscoordination risks and unsafe behaviors. The safety pipeline operates across three stages: (i) *Design-time:* The alignment checks between the solution paths generated by the planer and the Coordinator selection of most promising plan, static and policy checks are applied before execution. (ii) *Execution-time:* Sandboxed runs and testbed simulations validate runtime behavior and operational compliance. (iii) *Post-execution:* Embedding-driven drift detection, and operational metrics (e.g., contextual conflict rate, coordination overhead) assess alignment, consistency, and overall system health.

Extensibility and Deployment: The framework is highly modular and extensible, allowing new domain agents, static or policy checkers, runtime harnesses, or evaluation modules to be easily integrated. Personas can be customized per agent to modulate behavior, while governance knobs allow thresholds for drift, alignment, and refinement depth to be configured. Additionally, the framework can operate autonomously: given a domain specification, it can automatically generate relevant questions or tasks for the agents to address, without manual intervention. This capability enables rapid adaptation to new telecom scenarios or other domains, supporting end-to-end evaluation and refinement cycles. Deployment options range from local sandboxed environments for testing, to lab or testbed setups for KPI-driven validation, and ultimately to

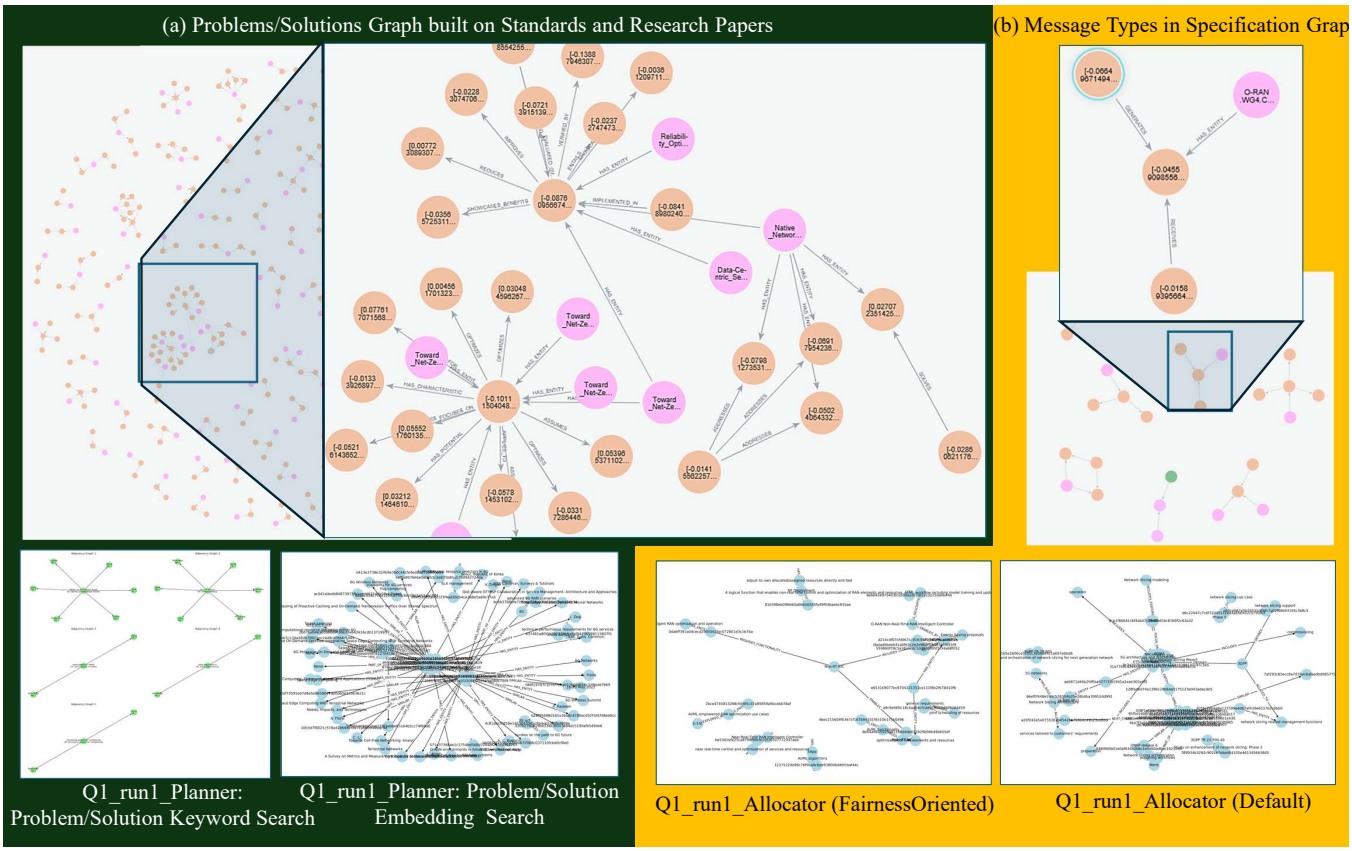


Figure 2: (a) Knowledge graph constructed from O-RAN standards and research papers, supporting Planner Agent in problem-solution searches via keyword and embedding retrieval. (b) Knowledge graph derived from O-RAN specifications, enabling the Allocator Agent to generate pseudocode through embedding-based retrieval.

production monitoring with strict policy enforcement and human-in-the-loop oversight.

4 Evaluation

This section evaluates the performance and safety of the proposed framework for GMAS in the telecom domain. Due to space constraints, we report a subset of the results, while the full dataset and additional analyses are available in our GitHub repository. The framework employs two complementary categories of metrics. *Analyzer Agent metrics* evaluate the technical quality and compliance of agents output across the dimensions described in Section 3. These dimensions are aggregated into an overall *Analyzer Penalty*. In parallel, *Safety Risk metrics* capture system-level risks across interacting agents. Agents consistency measures the consistency between Resource Allocator and Coder intent and execution, and cross-run embedding distance quantifies semantic drift across runs. Together, these metrics provide a multi-dimensional view of both agent-level reliability and system-level safety. We evaluate the framework on **five questions** across **32 persona sets** (2^5 combinations), using input data from the **O-RAN domain**. The system is powered by **GPT-3.5-turbo** as the language model and **all-MiniLM-L6-v2** as the embedding model. We generated the evaluation question set using an LLM-based generator (gpt-3.5-turbo, temperature 0.5) to produce realistic, engineering-oriented questions in the O-RAN domain, focusing on resource allocation and network optimization.

4.1 Analyzer Penalty

Fig. 3 shows analyzer penalty score across five runs for 32 persona sets. Lower scores indicate more severe issues detected by the Analyzer. Performance improves markedly after the first run: Run 1 starts weak, with a mean score of 10.17 and median of 1.0, suggesting wide-spread alignment and quality issues. From Run 2 onward, the mean rises to around 46–48, and the median stabilizes at 40, indicating that most question-persona combinations achieve moderate-to-high alignment. A small number of persona sets reach scores near 100 suggesting that certain persona sets can yield highly reliavble outputs. In Run 1, 45 out of 160 combinations had the lowest score of 5, concentrated within nine persona sets (28% of all persona sets). These largely involved "StrictAssessor" persona for the Analyzer or "Default"/"Minimalist" for other agents, indicating elevated safety concerns. Conversely, four persona sets

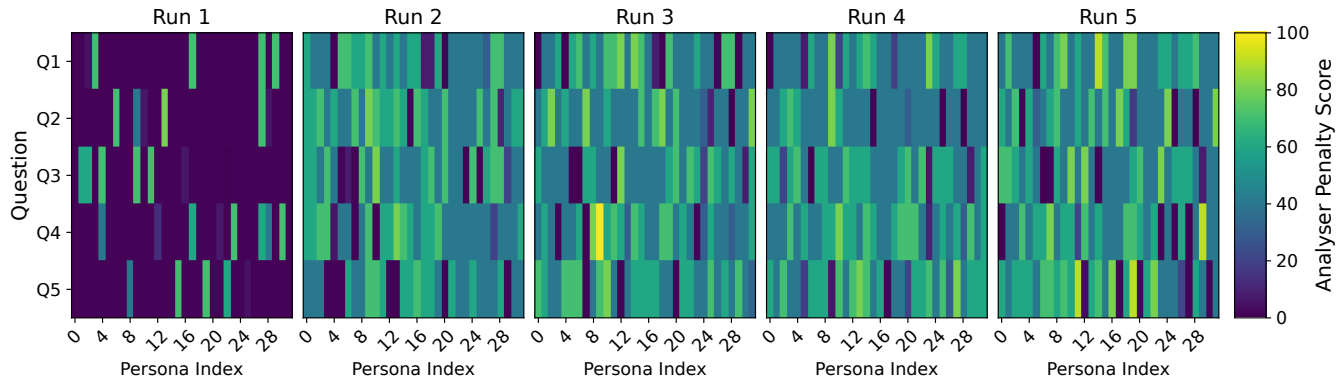


Figure 3: Analyzer Agent penalty score across five runs

(12.5%) achieved the highest scores (above 70), often featuring "CreativeThinker" and "FairnessOriented", which appear to promote more robust and safer outputs.

By Run 5, performance consolidates: the three persona sets with the highest averages (60–63) combine "CreativeThinker" and "FairnessOriented" roles in Planner and Resource Allocator with stricter Analyzer settings, yielding the fewest and least severe issues. In contrast, the lowest-scoring sets (28–32) clustered around "StrictAssessor" in Analyzer and "Minimalist" in Coder, suggesting that overly rigid or underspecified personas hindered improvement. Overall, these results highlight substantial variability across personas but also demonstrate the framework's capacity to surface persona-driven risks, trace their impact on system safety, and identify configurations that foster safer coordination.

4.2 Agents Consistency

Fig. 4 shows consistency scores between pseudo-code generated by the Resource Allocator and the executable code produced by the CodeAgent. Coding style emerges as the dominant factor: Minimalist Coders consistently achieve stronger alignment, averaging 81–85 with standard deviations of 10–17, while Default Coders exhibit wider variability (73–87, with deviations up to 23). For instance, the configuration *Analyzer:Default, Coder:Minimalist, Coordinator:Default, Planner:CreativeThinker, Allocator:FairnessOriented* achieved 85.6, compared to 73.8 for its Default Coder counterpart. Planner personas such as CreativeThinker improve alignment marginally (2–5 points), but Allocator–Coder consistency is primarily driven by coding style. These findings underscore that persona design in the coding role directly influences system stability: Minimalist Coders yield more predictable Allocator–Coder translations, while Default configurations introduce higher risk of divergence. The framework thus surfaces how localized persona choices propagate into system-level coordination, an essential property for ensuring safe execution in telecom workflows.

4.3 Cross-Run Embedding Distance

Fig. 5 shows the average embedding distance between consecutive runs for the Coder, averaged across all questions. On average, distances decrease from run1 to run2 (0.226) to run4 to run5

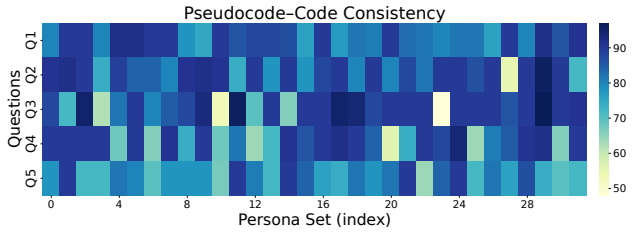


Figure 4: Consistency scores between pseudo-code from Allocator and executable code from Coder.

(0.150), indicating that outputs become more consistent over successive runs. Standard deviations remain relatively high compared to the means, reflecting variability among specific outputs in each transition. Distances vary substantially by persona. Combinations such as Minimalist Coder, Default Coordinator, CreativeThinker Planer, and Default Resource Allocator yield high average distances (0.38), reflecting greater variability, while others, such as Minimalist Coder, Default Coordinator, Default Planer, FairnessOriented Resource Allocator achieve very low averages (0.06), indicating stability. Overall, coding style, planning approach, and reasoning orientation strongly influence output consistency of agent behavior, with some combinations producing more reproducible results and others yielding more diverse or exploratory outputs. The Minimalist Coder leads to the highest variability (up to 0.37665), whereas FairnessOriented Allocator reduces distances to as low as 0.05951. Default Coder show moderate variability (0.1457 to 0.2506), while strategic Coordinators slightly lower variability. Overall, Coder and Resource Allocator choices have the largest impact on embedding consistency, with Planner and Coordinator exerting secondary but still noticeable effects in modulating cross-run differences. Embedding distance analysis shows progressive convergence across runs, but variability is highly sensitive to Coder and Resource Allocator personas, making them the most critical levers for stability.

5 Conclusion

This paper introduced a GMAS for safety evaluation in the telecom domain, with a focus on O-RAN and RIC settings. The framework integrates agent-level checks and system-level safety metrics,

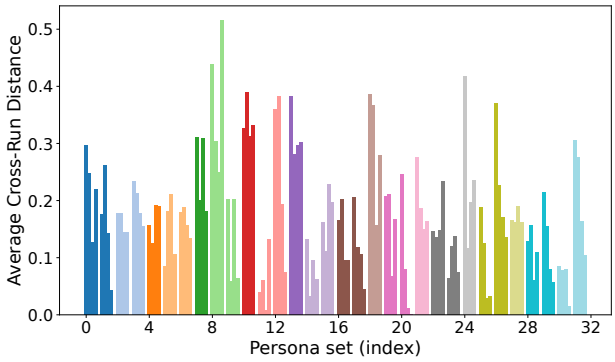


Figure 5: Aggregate cross-run distances for each persona group (Coder Agent). Bars show the mean distance between runs across all questions, with each color representing a persona group.

enabling fine-grained evaluation of technical correctness, policy compliance, runtime behavior, and formal guarantees, alongside system-oriented measures such as decision alignment, semantic drift, and coordination load. Through experiments across multiple persona configurations and refining runs, we observed that agent outputs converge over time, with analyzer penalties and embedding distances demonstrating progressive improvements in safety and consistency. The results also highlighted the strong influence of persona design on output stability and reliability. By bridging the gap between domain-specific deployment prospects and responsible AI evaluation, this work contributes a structured and extensible methodology for auditing and governing GMAS in telecom. Future work will extend the framework to larger-scale testbeds, integrate human-in-the-loop safety governance, and explore adaptive policies for dynamic persona adjustment under real-world operating conditions.

6 Acknowledgment

This work was supported by EPSRC grants EP/Y037421/1 (CHEDDAR) and UKRI851 on AI-agent cooperation for telecom safety and governance.

References

[1] Zeinab Nezami, Syed Ali Raza Zaidi, Maryam Hafeez, Jie Xu, and Karim Djemame. Toward standardization of genai-driven agentic architectures for radio access networks. *Frontiers in Artificial Intelligence*, Volume 8 - 2025, 2025.

[2] Abdelaali Chaoub, Aarne Mammel, Pedro Martinez-Julia, Ranganai Chaparadza, Muslim Elkotob, Lyndon Ong, Dilip Krishnaswamy, Antti Anttonen, and Ashutosh Dutta. Hybrid self-organizing networks: Evolution, standardization trends, and a 6g architecture vision: submitted (is under review) to ieee magazine on standards for evolving and future networks. *May 31st*, 2022.

[3] Liangxin Qian, Ping Yang, Gang Wu, Wanbin Tang, and Ze Chen. Self-evolving wireless communications: A novel intelligence trend for 6g and beyond. In *2024 International Conference on Future Communications and Networks (FCN)*, pages 1–6. IEEE, 2024.

[4] Zeinab Nezami, Syed Danial Ali Shah, Maryam Hafeez, Karim Djemame, and Syed Ali Raza Zaidi. From connectivity to autonomy: the dawn of self-evolving communication systems. *Frontiers in Communications and Networks*, Volume 6 - 2025, 2025.

[5] Lewis Hammond, Alan Chan, Jesse Clifton, Jason Hoelscher-Obermaier, Akbir Khan, Euan McLean, Chandler Smith, Wolfram Barfuss, Jakob Foerster, Tomáš Gavenčiak, et al. Multi-agent risks from advanced ai. *arXiv preprint arXiv:2502.14143*, 2025.

[6] Jacky Liang, Wenlong Huang, Fei Xia, Peng Xu, Karol Hausman, Brian Ichter, Pete Florence, and Andy Zeng. Code as policies: Language model programs for embodied control. In *2023 IEEE International Conference on Robotics and Automation (ICRA)*, pages 9493–9500. IEEE, 2023.

[7] AI-RAN. Ai-ran: Telecom infrastructure for the age of ai. https://ai-ran.org/, 2025. Accessed: Aug. 2025.

[8] Rajdeep Mondal, Alan Tang, Ryan Beckett, Todd Millstein, and George Varghese. What do llms need to synthesize correct router configurations? In *Proceedings of the 22nd ACM Workshop on Hot Topics in Networks*, pages 189–195, 2023.

[9] Lina Bariah, Qiyang Zhao, Hang Zou, Yu Tian, Faouzi Bader, and Merouane Debbah. Large generative ai models for telecom: The next big thing? *IEEE Communications Magazine*, 2024.

[10] Vinay Samuel, Henry Peng Zou, Yue Zhou, Shreyas Chaudhari, Ashwin Kalyan, Tanmay Rajpurohit, Ameet Deshpande, Karthik Narasimhan, and Vishvak Murahari. Personagym: Evaluating persona agents and llms. *arXiv preprint arXiv:2407.18416*, 2024.

[11] Chenxu Wang, Bin Dai, Huaping Liu, and Baoyuan Wang. Towards objectively benchmarking social intelligence for language agents at action level. *arXiv preprint arXiv:2404.05337*, 2024.

[12] Shaoguang Mao, Yuzhe Cai, Yan Xia, Wenshan Wu, Xun Wang, Fengyi Wang, Tao Ge, and Furu Wei. Alympics: Language agents meet game theory. *arXiv preprint arXiv:2311.03220*, 2023.

[13] Nathan Bosch. Integrating telecommunications-specific language models into a trouble report retrieval approach, 2022.

[14] Yuyang Du, Hongyu Deng, Soung Chang Liew, Kexin Chen, Yulin Shao, and He Chen. The power of large language models for wireless communication system development: A case study on fpga platforms. *arXiv preprint arXiv:2307.07319*, 2023.

[15] Feibo Jiang, Li Dong, Yubo Peng, Kezhi Wang, Kun Yang, Cunhua Pan, Dusit Niyato, and Octavia A Dobre. Large language model enhanced multi-agent systems for 6g communications. *arXiv preprint arXiv:2312.07850*, 2023.

[16] Yifei Xu, Yuning Chen, Xumiao Zhang, Xianshang Lin, Pan Hu, Yunfei Ma, Songwu Lu, Wan Du, Zhuoqing Mao, Ennan Zhai, et al. Cloudeval-yaml: A practical benchmark for cloud configuration generation. *Proceedings of Machine Learning and Systems*, 6:173–195, 2024.

[17] Zeinab Nezami, Maryam Hafeez, Karim Djemame, Syed Ali Raza Zaidi, and Jie Xu. Descriptor: Benchmark dataset for generative ai on edge devices (bedged). *IEEE Data Descriptions*, 2025.

[18] Xiaoqi Qin, Mengying Sun, Jincheng Dai, Peixuan Ma, Yuecheng Cao, Jingjing Zhang, Jiacheng Wang, Xiaodong Xu, Ping Zhang, and Dusit Niyato. Generative ai meets wireless networking: An interactive paradigm for intent-driven communications. *IEEE Transactions on Cognitive Communications and Networking*, 2025.

[19] Grzegorz Panek, Piotr Matysiak, Marcin Ziółkowski, Ilhem Fajjari, Cyril Auboin, and Iwona Wojan. Taia: Telco generative ai-powered multi-agent assistant for managing cloud-native networks. In *2025 IEEE International Conference on Communications*. Institute of Electrical and Electronics Engineers, 2025.

[20] Hang Zou, Qiyang Zhao, Lina Bariah, Mehdi Bennis, and Merouane Debbah. Wireless multi-agent generative ai: From connected intelligence to collective intelligence. *arXiv preprint arXiv:2307.02757*, 2023.

[21] Xu Li, Weisen Shi, Hang Zhang, Chenghui Peng, Shaoyun Wu, and Wen Tong. The agentic-ai core: An ai-empowered, mission-oriented core network for next-generation mobile telecommunications. *Engineering*, 2025.

[22] Amadeu Do Nascimento Júnior, Alexandros Palaios, Klaus Raizer, Swarup Kumar Mohalik, and Rafia Inam. Safe and explainable ai in 6g telecommunication systems: The way forward. pages 167–171, 2025.

[23] Julian De Freitas, Ahmet Kaan Uğuralp, Zeliha Oğuz-Uğuralp, and Stefano Puntoni. Chatbots and mental health: Insights into the safety of generative ai. *Journal of Consumer Psychology*, 34(3):481–491, 2024.

[24] Hang Wang and Junshan Zhang. Genai-based multi-agent reinforcement learning towards distributed agent intelligence: A generative-rl agent perspective. *arXiv preprint arXiv:2507.09495*, 2025.

[25] Jeremy Bellay, J Timothy Balint, Stephen A Boxwell, and Jeffrey Geppert. Safe systems with unsafe agents: Challenges and opportunities. In *Proceedings of the 24th International Conference on Autonomous Agents and Multiagent Systems*, pages 2849–2853, 2025.

[26] Shunyu Yao, Dian Yu, Jeffrey Zhao, Izhak Shafran, Tom Griffiths, Yuan Cao, and Karthik Narasimhan. Tree of thoughts: Deliberate problem solving with large language models. *Advances in neural information processing systems*, 36:11809–11822, 2023.

[27] Patrick Lewis, Ethan Perez, Aleksandra Piktus, Fabio Petroni, Vladimir Karpukhin, Naman Goyal, Heinrich Küttler, Mike Lewis, Wen-tau Yih, Tim Rocktäschel, et al. Retrieval-augmented generation for knowledge-intensive nlp tasks. *Advances in neural information processing systems*, 33:9459–9474, 2020.

Poster: Traffic-Aware Energy Efficiency Optimization in Practical Open RAN 5G Networks

E. Ramagiddaiah[‡], GV Preethi[‡], SVR Anand[‡], Chandra R. Murthy[‡], Ryan Husbands[†]
[‡]Indian Institute of Science, India, [†]British Telecom, UK
ramagiddaiah@fsid-iisc.in,{preethigv,anandsvr,cmurthy}@iisc.ac.in,ryan.husbands@bt.com

Abstract

Energy Efficiency (EE) is a crucial design goal for 5G networks. This paper focuses on energy savings in the Radio Access Network (RAN), specifically targeting the computationally intensive Distribution Unit (DU) and Centralized Unit (CU) of a 5G base station (gNB). We present a real-time power control mechanism that dynamically selects the number of active CPU cores and system clock frequencies based on the network load. The optimal selection is determined by measuring the energy followed by an inverse-mapping procedure. We experimentally validate our solution on an OpenAirInterface (OAI) based 5G testbed. We also present a software implementation of a power saving application, developed under the O-RAN's Non-RT RIC framework, enabling remote control of CU, DU and Radio Unit (RU) modes of operation. Our approach achieves at least ~16% power savings without compromising on the performance, for data rates up to 240 Mbps. Thus, operating system level power control mechanisms are very useful in practice.

CCS Concepts

• **Networks** → **Network measurement**; *Network performance analysis*; **Network experimentation**;

Keywords

5G, gNB, EE, OAI, O-RAN, SMO, paES

ACM Reference Format:
E. Ramagiddaiah[‡], GV Preethi[‡], SVR Anand[‡], Chandra R. Murthy[‡], Ryan Husbands[†]. 2025. Poster: Traffic-Aware Energy Efficiency Optimization in Practical Open RAN 5G Networks. In *2nd ACM Workshop on Open and AI RAN (OpenRan '25), November 4–8, 2025, Hong Kong, China.* ACM, Hong Kong, China, 3 pages. https://doi.org/10.1145/3737900.3770166

Financial assistance for this work from British Telecom, UK is gratefully acknowledged.

OpenRan '25, November 4-8, 2025, Hong Kong, China

ACM ISBN 979-8-4007-1977-6/25/11
https://doi.org/10.1145/3737900.3770166

1 INTRODUCTION

Energy efficiency (EE) is a key concern with the recent growth of fifth-generation (5G) mobile networks. As the throughput and latency requirements increase, the energy consumption of base stations also rises significantly. According to the GSMA Mobile Economy Report 2025, energy accounts for about 20% of the operational expenses of a Telecom service provider [1]. The importance of network energy saving use cases has been highlighted as "critical" by the O-RAN Alliance [2], 3GPP, and NGMN Alliance.

The O-RAN Alliance has proposed strategies to improve the EE of open and disaggregated RAN components. The O-RAN architecture [3] facilitates the control of RAN components through rApps deployed on the Non-RT RIC in the service management and orchestration (SMO) framework. The O1 interface is used to manage RAN functions and set RAN parameters, as well as handle import and export commands between the SMO and network nodes. Most base stations today operate as software on edge servers and white box hardware, making power optimization is mainly a problem of managing software execution and server settings.

1.1 Related work

Various techniques have been proposed to improve EE in 5G networks, including sleep mode management, computational resource optimization, and machine learning-based approaches, with some of them being in compliance with 3GPP and O-RAN specifications. Energy-saving techniques such as carrier shutdown, deep sleep modes, and AI-based solutions are discussed in [4].

In [5], software power monitoring tools have been used to assess the power consumption of 5G testbed components, including time-series based energy monitoring frameworks, namely, Power Distribution Units (PDUs), Kepler, and Scaphandre, which provide processor-level power tracking. In our work, we use Scaphandre for its ability to provide real-time processor-level power metrics, making it well suited for analyzing the energy behavior of RAN components under varying compute loads. The O-RAN WG1 report [2] defines energy-saving strategies like "Carrier and Cell On/Off" which dynamically deactivate carriers or cells during low traffic via the non-RT or near-RT RIC, saving 25–30% power

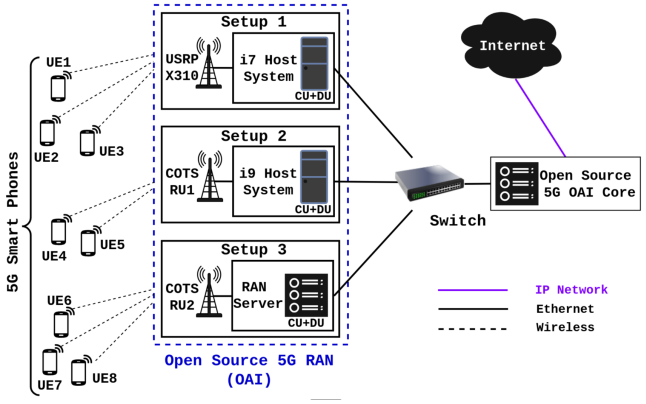


Figure 1: 5G Testbed Architecture

in dense deployments while ensuring on-demand reactivation. We also used a smart energy meter to evaluate power consumption while implementing cell on/off, CPU active core management, and clock frequency settings.

2 5G TESTBED ARCHITECTURE

In this section, we discuss the functional overview and preliminary work of our 5G testbed.

2.1 Functional Overview

The end-to-end 5G testbed deployed in our lab is shown in Fig. 1. It provides a fully functional 5G gNB, connecting commercial 5G user equipment (UE). The OpenAirInterface (OAI) 5G RAN and core [6] is used to build the network. The core manages UE authentication, session control, and data routing to external networks. The gNB handles over-the-air communication, radio resource management, and connects UEs to the core network. The connection between the OAI 5G Core and RAN is achieved via standardized interfaces. As shown in Fig. 1, our testbed is based on the Intel platform running the Ubuntu 22.04 OS.

The three setups are as follows: In **Setup 1**, the CU/DU runs on an Intel platform connected to a USRP X310 software-defined radio (SDR) unit. The 5G RAN is configured with 40 MHz channel bandwidth, 30 kHz subcarrier spacing (SCS), band n78, and employs 106 Physical Resource Blocks (PRB). This RU enables power consumption monitoring, and control of its modes of operation. **Setup 2** employs a COTS RU (VVDN[1]), shown as COTS RU1 in the figure. The 5G RAN is configured with 100 MHz bandwidth, 30 kHz SCS, 273 PRBs, band n78. In this, the CU/DU system runs on 24 CPUs at a base clock frequency of 3.2 GHz, with the CPU frequency adjustable between 1100 and 3200 MHz and the number of active cores ranging from 6 to 24.[2] **Setup 3** uses Intel Xeon Gold 6544Y server system (CU/DU), connected to an RU of a different make, shown as COTS RU2. These different setups are aimed at conducting power consumption studies on different commercial radios and CU/DU platforms.

2.2 Preliminary Work

Using our 5G testbed, we evaluated how various network and system-level parameters influence power consumption. Key observations are summarized below.

The gNB power consumption was studied when the number of UEs increased. *Scaphandre* power telemetry tool was integrated with *Prometheus* and *Grafana* for visualization, and recorded an increase of only about 90 mW in the gNB's power consumption for each additional UE.

For PRB utilization analysis, we used a system configured for 40 MHz channel bandwidth in SISO mode. As throughput increased from 5 to 120 Mbps, power consumption of the task_gtp_v1 thread rose accordingly. PowerTOP measurements showed that total stack thread power remained steady at 2.1 W, independent of PRB utilization.

Next, we varied the distance between the UE and the gNB, at 1 m, 3 m, and 10 m. The received signal reference power (RSRP) varied approximately -78 dBm at 1 m, -80 dBm at 3 m, and -92 dBm at 10 m, but the power consumption and throughput remained nearly constant. During uplink transmissions, RU power consumption was stable at approximately 37 W for data rates from 10 to 50 Mbps, regardless of distance. Similarly, in the downlink, the RU power consumption remained consistent at 38 W for data rates of up to 300 Mbps. We also observed that the CU/DU (Setup2 in Fig. 1) power consumption increased from 115 to 125 W for MCS values ranging from 9 to 27 (when UE was connected but idle to the case when the UE achieved maximum DL throughput), in line with the results reported in [7]. These observations suggest that power consumption and throughput are not significantly affected by UE location, although the signal quality degrades with distance. Of course, at larger distances (beyond 10 m), the UE throughput degrades.

3 O-RAN ENERGY-SAVING STRATEGIES

In this section, we present O-RAN-based power control measures and the associated numerical results.

3.1 O-RAN-based Power Control

To demonstrate cell on/off feature, we implemented a power control framework in our 5G testbed, featuring an SMO, a performance-aware energy saver (PaES) rApp using REST interface, an O1 adapter, and the OAI stack as shown in Fig. 2.

(1) RU Power Control: We experimented with both COTS and software-defined radios. Since COTS RUs provided limited programmability, we employed a USRP X310 to evaluate dynamic power states. During idle or low-traffic periods, the RU was switched to a low-power state via the SMO through the O1 interface. The RU was successfully reactivated when traffic resumed, in line with O-RAN's carrier and cell on/off policy.

[1] https://www.vvdntech.com/adaptive-compute-and-comms/enterprise-radio-unit

[2] gNB fails below 1100 MHz CPU clock frequency or with fewer than 6 active cores.

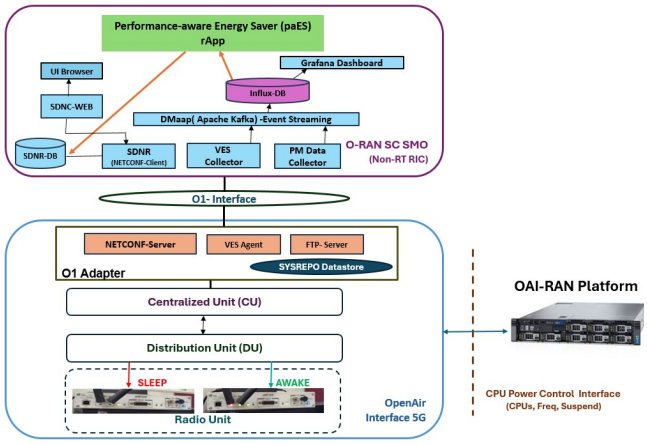


Figure 2: End-to-End RAN Power Control in O-RAN Framework via O1 Interface

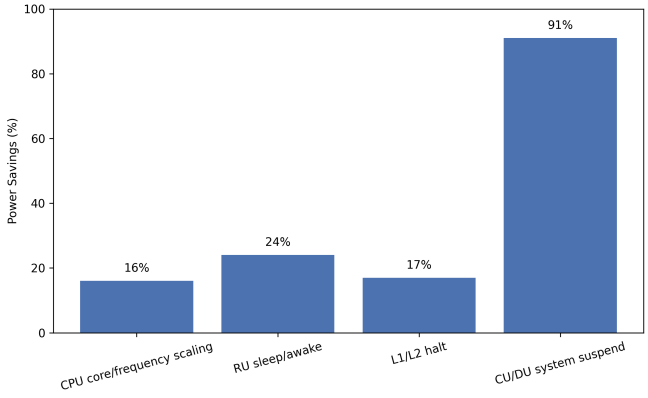


Figure 3: Power savings (%) achieved by different power optimization methods.

(2) CU/DU Power Control: On the CU/DU system (Setup 1, Fig. 1), a gNB shutdown was triggered via the SMO, effectively halting Layer 1 and Layer 2 operations during prolonged idle states. In the current implementation, shutdown occurs regardless of UE attachment, assuming no active traffic. Consequently, idle UEs temporarily lose connectivity. While acceptable for low-traffic tests, practical deployments will require UE cell reselection before shutdown to ensure service continuity. Further, enabling full system suspend during extended inactivity led to a dramatic reduction in system-level power usage. Upon wake-up, the base station was seamlessly re-initialized through the O1 interface.

In Setup2 (Fig. 1), experiments varied core counts in steps of 3 and all available frequencies. Power consumption was optimized by dynamically adjusting CPU frequency and active cores based on user demand, significantly reducing energy use over maximum settings. In practice, the gNB tracks data rate requirements for each flow at DRB setup and updates DRBs as traffic fluctuates. While data demands change every 100s of milliseconds to seconds, CPU frequency adjustments occur within milliseconds, causing only minimal delay during DRB setup with no other processing impact.

Thus, under low traffic conditions, RU and CU/DU power states can be dynamically managed via the O1 interface to save energy, with transparent reactivation to maintain QoS. Additionally, dynamic control of active CPU core and clock frequencies further contributes to energy savings. The implementation details and code for the RAN power control mechanism are available in the GitHub repository [8].

3.2 Experimental Results

We briefly summarize our experimental results. As shown in Fig. 3, significant power savings are achieved through the implementation of power optimization methods, highlighting the effectiveness of integrating component-level techniques such as CPU core frequency scaling of the CU/DU during normal traffic fluctuations, L1/L2 cache halting of the CU/DU and RU sleep/awake transitions during low traffic conditions, and enabling suspend mode at the CU/DU side during complete inactivity. Together, these strategies contribute to substantial reductions in overall energy consumption.

4 CONCLUSION AND FUTURE WORK

In this work, we developed power consumption minimization strategies for gNB operation by dynamically adjusting the number of active CPU cores and system clock frequencies, achieving at least 16% power savings for data rates up to 240 Mbps while meeting user data requirements.

RU-level power savings of ~24% were achieved by turning off the RU and 17% saved by L1/L2 shutdown during low traffic, and further enabling system suspend mode resulted in up to 91% power savings. These savings can be achieved without compromising system performance, provided that data demand is handled via UE or network requests, thereby being available at the gNB at least a few tens of ms in advance.

Future work includes extending to MIMO configurations and developing better algorithms to adapt to dynamically varying data demand to optimize system parameters and reduce power consumption.

References

[1] GSMA. 2025. *The Mobile Economy 2025.* https://www.gsma.com/solutions-and-impact/connectivity-for-good/mobile-economy/.
[2] O-RAN Alliance, Sustainability Focus Group. Potential Energy Savings Features in O-RAN. White Paper. O-RAN Alliance, Jan 2025.
[3] O-RAN Alliance. 2025. *O-RAN Architecture Description v13.0.* Feb 2025. https://specifications.o-ran.org/specifications.
[4] R. Tan, Y. Shi, Y. Fan, et al. 2022. Energy Saving Technologies and Best Practices for 5G Radio Access Network. *IEEE Access.*
[5] N. K. Shankaranarayanan, Z. Li, I. Seskar, et al. POET: A Platform for O-RAN Energy Efficiency Testing. In *Proceedings of the IEEE 100th Vehicular Technology Conference (VTC2024-Fall)*, 2024.
[6] N. Nikaein, M. K. Marina, S. Manickam, et al. OpenAirInterface: A Flexible Platform for 5G Research. *SIGCOMM Rev.*, Oct 2014, NY, USA.
[7] U. Pawar, B. R. Tamma, F. A. Antony. Traffic-Aware Compute Resource Tuning for Energy Efficient Cloud RANs. In *IEEE GLOBECOM*, 2021.
[8] paES rApp. Power control mechanism for Open RAN over 5G Networks. https://github.com/Energy-optimization/Energy-optimization, 2025.

Poster: Agentic AI meets Neural Architecture Search: Proactive Traffic Prediction for AI-RAN

Abdelaziz Salama, Mohammed M. H. Qazzaz, Zeinab Nezami, Maryam Hafeez, Syed Ali Raza Zaidi
School of Electrical and Electronic Engineering, University of Leeds
Leeds, United Kingdom

Abstract

Next-generation wireless networks require intelligent traffic prediction to enable autonomous resource management and handle diverse, dynamic service demands. The Open Radio Access Network (O-RAN) framework provides a promising foundation for embedding machine learning intelligence through its disaggregated architecture and programmable interfaces. This work applies a Neural Architecture Search (NAS)-based framework that dynamically selects and orchestrates efficient Long Short-Term Memory (LSTM) architectures for traffic prediction in O-RAN environments. Our approach leverages the O-RAN paradigm by separating architecture optimisation (via non-RT RIC rApps) from real-time inference (via near-RT RIC xApps), enabling adaptive model deployment based on traffic conditions and resource constraints. Experimental evaluation across six LSTM architectures demonstrates that lightweight models achieve $R^2 \approx 0.91$–0.93 with high efficiency for regular traffic, while complex models reach near-perfect accuracy ($R^2 = 0.989$–0.996) during critical scenarios. Our NAS-based orchestration achieves a 70-75% reduction in computational complexity compared to static high-performance models, while maintaining high prediction accuracy when required, thereby enabling scalable deployment in real-world edge environments.

CCS Concepts

• **Networks** → **Network performance modeling**; • **Information systems** → Mobile information processing systems.

Keywords

AI-RAN, Agentic AI, Network Optimisation, Traffic Prediction

1 Introduction

The evolution towards next-generation wireless networks demands intelligent and autonomous network management capabilities that extend beyond traditional performance optimisation approaches. Modern telecommunication infrastructures require predictive analytics and proactive resource allocation to handle increasingly complex traffic patterns and diverse service requirements. Within this context, the Open Radio Access Network (O-RAN) framework emerges as a pivotal architecture that enables intelligent network control through disaggregated components and programmable interfaces [1]. The integration of agent AI models, particularly Long Short-Term Memory (LSTM) agents, within O-RAN's RAN Intelligent Controllers (RICs) presents significant opportunities for real-time traffic prediction and autonomous network optimisation, as we have shown in our earlier work [2].

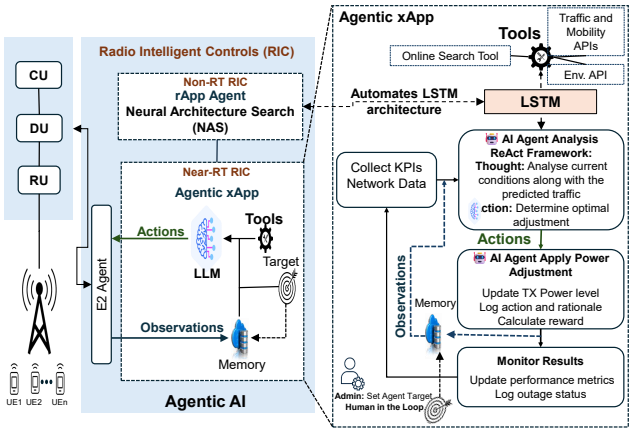


Figure 1: Balanced complexity agentic AI framework using NAS-based LSTM for traffic prediction.

The deployment of Agentic AI and LSTM-based prediction agents in O-RAN environments faces a fundamental trade-off between predictive accuracy and computational efficiency. Large and complex models deliver high accuracy but are often impractical at the edge due to constraints in computation, memory, and energy, leading to high operational costs and limited scalability [3]. Conversely, overly lightweight models may fail to capture the temporal dynamics of network traffic, resulting in suboptimal prediction performance.

This challenge necessitates an adaptive framework that dynamically selects optimal architectures to balance accuracy and efficiency based on operational context. Leveraging O-RAN's modular architecture, we implement Neural Architecture Search (NAS) [4, 5] as an rApp to identify optimal LSTM configurations for time-series traffic prediction, while xApps execute the selected models in real-time. This approach maintains computational efficiency during normal operations while ensuring high accuracy during critical network events.

2 System Model

The proposed framework operates within the Open Radio Access Network (O-RAN) architecture to predict in advance the network traffic load and enable proactive optimisation in telecom networks. The architecture employs a two-tier approach that separates model optimisation from real-time inference execution.

OpenRan '25, November 4-8, 2025, Hong Kong, China

ACM ISBN 979-8-4007-1977-6/25/11
https://doi.org/10.1145/3737900.3770175

At the architectural optimisation level, a NAS module is deployed as a non-real-time RAN Intelligent Controller (non-RT RIC) rApp, as shown in Fig 1. This component periodically explores and evaluates different LSTM configurations to determine the optimal architecture for traffic forecasting based on current network conditions and performance requirements. The rApp operates with relaxed timing constraints, allowing for comprehensive architectural exploration and evaluation.

Once the optimal configuration is identified, it is instantiated as an xApp in the near-real-time RIC, functioning as a low-latency inference agent for real-time traffic prediction. This separation of concerns ensures that the computationally intensive architecture search process does not interfere with the time-critical prediction tasks, while enabling continuous model optimisation as network conditions evolve.

The prediction model leverages historical network data through a timestep input sequence to forecast the next traffic load value using the selected LSTM architecture, ensuring sufficient context while preserving computational efficiency for real-time operation.

The input features combine network KPIs and temporal parameters. Network KPIs capture physical layer conditions that directly influence traffic patterns, while temporal features (hour, day of week, weekend/peak flags) capture cyclical traffic behaviours in telecom networks.

An LSTM cell regulates information flow through three key gates: the input gate (i_t), forget gate (f_t), and output gate (o_t), which control the cell state and hidden state transitions [6]. The computational complexity of an LSTM layer with input dimension d_x and hidden dimension d_h is quantified by its parameter count [3]:

$$C_{\text{LSTM}} = 4 \times (d_x d_h + d_h^2 + d_h), \tag{1}$$

where the factor 4 accounts for the input, forget, output, and candidate cell gates. This formulation enables precise complexity assessment for architecture comparison.

The NAS component evaluates six candidate LSTM architectures, systematically exploring the trade-space between predictive accuracy and computational cost. The search space ranges from lightweight single-layer models with 32–64 units, suitable for resource-constrained deployments, to multi-layer configurations with up to 256 units in the first layer, designed for high-accuracy applications.

To enable objective architecture comparison, we define a normalised efficiency metric that balances predictive performance P with computational complexity:

$$E = \frac{P}{C_{\text{norm}}}, \quad C_{\text{norm}} = \frac{C_{\text{LSTM}}}{\max(C_{\text{LSTM}})}. \tag{2}$$

This formulation allows direct comparison of architectures across varying sizes, explicitly quantifying the trade-off between prediction accuracy and computational cost.

Consequently, the NAS framework evaluates six distinct LSTM architectures designed to explore the trade-off between computational complexity and predictive performance. The *Lightweight* architectures (Lightweight-32 and Lightweight-64) employ single-layer configurations with 32 and 64 hidden units, respectively, utilising a reduced feature set of 6 input parameters to minimise computational overhead for edge deployment scenarios. The *Balanced* architectures (Balanced-Small and Balanced-Medium) implement two-layer LSTM configurations with 64×32 and 100×50 hidden unit arrangements, processing the full 8-feature input set to achieve optimal performance-efficiency trade-offs suitable for production environments. The *Deep-Performance* architecture employs a three-layer configuration (128×100×64 units) with an expanded 10-feature input set, targeting high-accuracy applications where computational resources are less constrained. Finally, the *Ultra-Performance* architecture employs a three-layer design with significantly larger hidden dimensions (512×256×128 units) and a comprehensive 16-feature input set, representing the upper bound of model capacity for scenarios demanding maximum predictive accuracy regardless of computational cost.

Table 1: NAS-based LSTM architecture comparison for O-RAN traffic prediction.

Arch	Params	Size	MAE	RMSE	MAPE	R^2 (Reg)	R^2 (Crit)	R^2 (Overall)	Eff
Light-32	25K	0.02	0.006	0.018	1.95	0.976	0.860	0.934	0.597
Light-64	38K	0.07	0.004	0.015	1.63	0.980	0.895	0.914	0.496
Bal-Small	44K	0.17	0.007	0.016	2.53	0.981	0.910	0.949	0.903
Bal-Med	74K	0.28	0.008	0.017	4.29	0.986	0.965	0.975	0.950
Deep-Perf	205K	0.78	0.009	0.019	4.91	0.990	0.970	0.989	0.901
Ultra-Perf	1.08M	4.13	0.008	0.019	4.03	0.996	0.982	0.996	0.279

3 Results and Discussion

The system employs a multi-dimensional evaluation approach that considers both predictive accuracy and computational efficiency. Accuracy is assessed using standard regression metrics, including Mean Absolute Error (MAE), Root Mean Squared Error (RMSE), Coefficient of Determination (R^2), and Mean Absolute Percentage Error (MAPE). Efficiency is evaluated in terms of model size, parameter count, and the performance-to-complexity ratio, highlighting architectures that satisfy network requirements while minimising computational overhead.

Within the proposed NAS-based LSTM framework, each LSTM model functions as an xApp agent in the near-real-time RIC, tasked with mobile network traffic prediction. These models process data from two primary sources: (i) historical time series of traffic load and network parameters, and (ii) dynamic contextual information obtained through external APIs. API integration ensures context-awareness by incorporating real-time updates, such as user mobility, service demand, or unexpected network events. Consequently, predictions are not solely based on past patterns but adapt to evolving operational conditions, enhancing robustness under critical scenarios.

The evaluation of our NAS-based LSTM architecture reveals a trade-off between model complexity, computational efficiency, and predictive performance (Table 1). Lightweight models (e.g., Lightweight-3 and Lightweight-6) achieve low error values (MAE $\approx$ 0.004–0.006) with overall R^2 scores around 0.91–0.93, while maintaining high efficiency in regular traffic scenarios and minimal parameter counts. Balanced architectures, such as Balanced-Small and Balanced-Medium, improve prediction accuracy (R^2 = 0.949 and 0.975, respectively) with moderate complexity increases. Larger models, including Deep-Performance and Ultra-Performance, attain

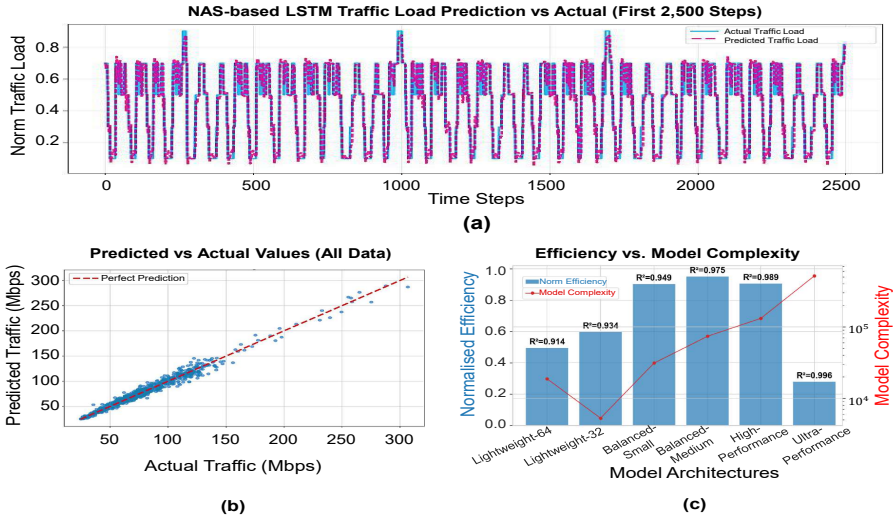


Figure 2: NAS-based LSTM traffic prediction results: (a) time series prediction over 2,500 steps, (b) predicted vs actual traffic correlation, and (c) architecture efficiency analysis across model complexities.

near-perfect R^2 values (0.989 and 0.996), particularly under critical traffic conditions, outperforming smaller models ($R^2 = 0.97$ and 0.982). These gains, however, come at the cost of higher computational demand and reduced efficiency. Notably, under regular traffic, all models perform comparably well ($R^2 > 0.97$), indicating that the benefit of complex models is most pronounced during critical scenarios.

The proposed NAS-based agent dynamically orchestrates multiple LSTM architectures, from lightweight to ultra-performance designs, each instantiated as an xApp variant tailored to operator requirements. Lightweight models are optimised for efficiency and minimal resource use, making them suitable for regular traffic or resource-constrained edge environments. Intermediate architectures leverage broader features from APIs and historical data to enhance prediction accuracy with moderate cost. Larger architectures exploit extended temporal dependencies and richer context to maximise accuracy during sudden traffic surges or anomalies.

In practice, our framework predominantly adopts the Balanced-Medium model for regular scenarios. This choice keeps the system proactive while reducing computational complexity by approximately 70–75% compared to Deep- and Ultra-Performance models, which are deployed selectively for high-impact events. This hierarchical and adaptive orchestration enables the O-RAN framework to flexibly assign the most appropriate xApp based on network conditions and resource availability, ensuring lightweight and intermediate models perform reliably under normal conditions, while larger models handle extreme scenarios efficiently.

4 Conclusion

This work proposes a NAS-based LSTM framework for O-RAN traffic prediction that balances predictive accuracy and computational efficiency in edge environments. It dynamically orchestrates LSTM models ranging from lightweight (25K params, $R^2 \approx 0.91$–0.93) to ultra-performance (1.08M params, $R^2 = 0.996$). A key innovation is separating architecture search (rApps) from real-time inference (xApps), allowing adaptive model selection based on network context and resource availability.

The system achieves optimal efficiency by primarily deploying Balanced-Medium models ($R^2 = 0.975$) during normal traffic, while activating heavier models for critical events. This reduces computational load by 70–75% compared to static high-performance approaches, with no significant loss in accuracy. Additionally, the integration of API-driven context awareness further enhances model robustness across diverse operational conditions.

Acknowledgment

This research was funded by EPSRC CHEDDAR (EP/X040518/1, EP/Y037421/1), UKRI Grant EP/X039161/1, and MSCA Horizon EU Grant 101086218. Additionally, this research was supported by the UKRI Funding Service under Award UKRI851: Strategic Decision-Making and Cooperation among AI Agents: Exploring Safety and Governance in Telecom.

References

[1] Michele Polese, Leonardo Bonati, Salvatore D'oro, Stefano Basagni, and Tommaso Melodia. Understanding o-ran: Architecture, interfaces, algorithms, security, and research challenges. *IEEE Communications Surveys & Tutorials*, pages 1376–1411, 2023.

[2] Abdelaziz Salama, Zeinab Nezami, Mohammed MH Qazzaz, Maryam Hafeez, and Syed Ali Raza Zaidi. Edge agentic ai framework for autonomous network optimisation in o-ran. *arXiv preprint arXiv:2507.21696*, 2025.

[3] Hui Xie, Meng Liu, Ashley Cusack, Guangxian Li, Andrew P Longstaff, Songling Ding, and Wencheng Pan. Analysis of the performance of lstm-dnn models with the consideration of signal complexity in milling processes. *Journal of Intelligent Manufacturing*, pages 1–44, 2025.

[4] Maher Dissem, Manar Amayri, and Nizar Bouguila. Neural architecture search for anomaly detection in time-series data of smart buildings: A reinforcement learning approach for optimal autoencoder design. *IEEE Internet of Things Journal*, 11(10):18059–18073, 2024.

[5] Thomas Elsken, Jan Hendrik Metzen, and Frank Hutter. Neural architecture search: A survey. *Journal of Machine Learning Research*, 20(55):1–21, 2019.

[6] Sepp Hochreiter and Jürgen Schmidhuber. Long short-term memory. *Neural computation*, 9(8):1735–1780, 1997.

Towards AI-Native RAN: Architecture and Key Technologies

Jingyi Wang
China Telecom Research Institute
Beijing, China
wangjy74@chinatelecom.cn

Zexu Li
China Telecom Research Institute
Beijing, China
lizx28@chinatelecom.cn

Ang Li
China Telecom Research Institute
Beijing, China
lia15@chinatelecom.cn

Abstract

Empowered by artificial intelligence (AI) techniques, the sixth generation (6G) mobile networks are expected to evolve as an AI-native platform. Although recent advances have promoted the design of AI-native radio access network (RAN), the integration of cross-domain AI collaboration with AI-native RAN poses challenges. To mitigate this gap, we propose edge AI layer as a unified orchestration platform across AI-native RAN and edge cloud. It provides AI-native RAN operation management and edge cloud resource management, and employs an edge AI orchestrator to enable cross-domain collaboration. By holistically orchestrating on the AI-related resources and network functions, edge AI layer has shown the feasibility towards 6G AI-native network.

CCS Concepts

• **Networks** → **Mobile networks**; **Network design principles**.

Keywords

6G, AI-native RAN, edge AI layer, orchestrator.

ACM Reference Format:
Jingyi Wang, Zexu Li, and Ang Li. 2025. Towards AI-Native RAN: Architecture and Key Technologies. In *2nd ACM Workshop on Open and AI RAN (OpenRan '25), November 4–8, 2025, Hong Kong, China.* ACM, New York, NY, USA, 3 pages. https://doi.org/10.1145/3737900.3770169

1 Introduction

Driven by the proliferation of artificial i ntelligence (AI)-empowered applications, mobile networks are evolving to the sixth generation (6G) networks to deliver low-latency and ubiquitous services [1]. It requires deep integration of AI techniques into signal processing, network management and resource allocation of 6G, rather than externally enhance learning models and inference results [2]. This aligns with the principles of AI-native network design [3], which highlights a unified platform and deeply integrates communication, computing, and sensing capabilities. The 3rd Generation Partnership Project (3GPP) defined Network Data Analytics Function (NWDAF) to facilitate AI-enabled analysis in core network [4]. The O-RAN Alliance has introduced RAN Intelligent Controller (RIC) and standardized interfaces for multi-vendor deployment and network control [5]. The AI-RAN Alliance aims to improve RAN performance by leveraging AI techniques and explore the converged infrastructure to run AI and RAN workloads [6].

On the design of AI-native RAN architecture, current works mainly focus on single-domain AI. However, in the emerging new use cases such as agentic AI and embodied robots, the resources may be constrained for on-device computation-intensive tasks [7]. Moreover, driven by the distributed learning and reinforcement learning paradigm, multi-modal data flows from multiple nodes concurrently, with vertical (north-south) and horizontal (east-west) traffic interwinded [8]. Therefore, how to enable cloud-edge-device AI collaboration has become a critical issue.

Integrating cross-domain AI collaboration with AI-native RAN architecture needs to address the challenges of service orchestration and resource management. Firstly, current architecture has not specified cross-domain service orchestration mechanism for AI-related data, algorithms and workflows. The network functions deployed on AI-native RAN and edge cloud should be equipped with AI capability and adaptively configured based on the operation of AI tasks. Moreover, the computing and storage resources on the heterogeneous infrastructure need to be scalably managed and allocated for cross-domain AI collaboration. Motivated by these challenges, we propose an edge AI layer to integrate cross-domain AI collaboration in AI-native RAN, and illustrate the architecture in details.

2 Architecture of Edge AI Layer

2.1 Overview

To provide unified management and orchestration on AI-native RAN and edge cloud, we propose edge AI layer, a

OpenRan '25, November 4-8, 2025, Hong Kong, China

ACM ISBN 979-8-4007-1977-6/25/11
https://doi.org/10.1145/3737900.3770169

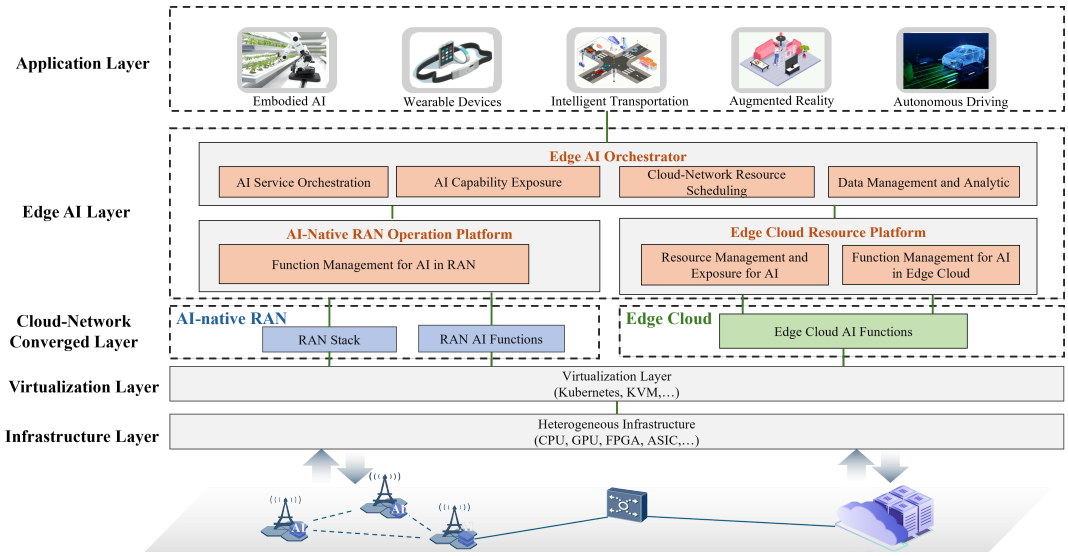


Figure 1: An architectural illustration of edge AI layer

management function and interface set across AI-native RAN and edge cloud domain. As depicted in Figure 1, the edge AI layer consists of three main components. The AI-native RAN operation platform manages the operation of AI algorithms and models on AI-native RAN, while the edge cloud resource platform manages computing and storage resources on edge cloud infrastructure. The edge AI orchestrator uniformly manages cross-domain resources, data, and network functions based on heterogeneous infrastructure.

2.2 AI-Native RAN Operation Platform

The platform enhances the monitoring, configuration and management functions for the deployment and operation process of AI algorithms or models on AI-native RAN. It also triggers the Uu process to control UE-side intelligence.

2.2.1 Function Management for AI in RAN (FMAIR). The FMAIR function holistically manages the network functions related to AI algorithms or models deployed on E2 nodes, RICs and UE side. FMAIR reports operation data to the edge AI orchestrator, and receives the update strategy, based on which FMAIR can reconfigure AI-native RAN functions and thus enhance AI algorithm efficiency. The operation data is correlated with the operation of AI algorithms or models on AI-native RAN, which encompasses the performance management (PM) data, configuration management (CM) data and fault management (FM) data through O1 interface. FMAIR also supports deployment of network functions for AI model training and inference on AI-native RAN.

2.3 Edge Cloud Resource Platform

The platform integrates management functions of computing and storage resources for the deployment and storage of AI algorithms or models, and provides configuration of AI network functions running on edge cloud.

2.3.1 Resource Management and Exposure for AI (RMEAI). The RMEAI function manages and exposes the AI-related computing and storage resources deployed on edge cloud infrastructure. The services include computing/storage resource perception, registration, exposure, activation, deactivation and configuration. RMEAI reports the resource-related management data to the edge AI orchestrator for deployment and storage of AI algorithms or models. RMEAI reconfigures the resources on the edge cloud based on feedback from the edge AI orchestrator.

2.3.2 Function Management for AI in Edge Cloud (FMAIEC). The FMAIEC function manages the deployed AI-related network functions on edge cloud, which support flexible allocation, recycling and orchestration of computing and storage resources. FMAIEC collects PM, CM and FM data correlated with edge cloud AI network functions via O2 interface, and configures the management functions on computing resources or storage resources. FMAIEC supports deployment of edge cloud AI model training and inference functions.

2.4 Edge AI Orchestrator

The edge AI orchestrator establishes a control plane across AI-native RAN and edge cloud, and jointly manages the resources and network functions. The orchestrator works on

the resource, operation and service layers, which manifests the capability of cross-domain collaboration.

2.4.1 AI Service Orchestration (AISO). Based on the operation and management data reported from both AI-native RAN and edge cloud, the AISO function generates strategy for resource scheduling, data management, and model optimization to improve overall quality of service (QoS). AISO may identify the priority for various AI services, and determine the workflow of multi-task processing. For instance, the large-scale training tasks can be deployed on edge cloud, and then transferred lightweight inference tasks to AI-native RAN for low-latency service provisioning.

2.4.2 Cloud-Network Resource Scheduling (CNRS). The CNRS function provides the capability of scheduling computing and storage resources on AI-native RAN and edge cloud. Through the virtualization layer, the heterogeneous resources are abstracted into transferable computing and storage containers, which can be scalably allocated to the cloud-network functions. CNRS interacts with RMEAI to obtain the infrastructure information of deployed resources and delivers configuration on cloud-network resources to RMEAI.

2.4.3 Data Management and Analytic (DMA). The management data from both AI-native RAN and edge cloud are registered in DMA function, which can be subscribed by external or internal data consumers for analytics based on service requirements. For instance, the orchestrator can subscribe AI-native RAN data including the performance monitoring of RAN functions or the indicator for AI model deployment, and generate reconfiguration strategy for FMAIR.

3 Capability of Edge AI Layer

3.1 Cloud-Network Convergence

The AI-related operation and resource management metadata from both AI-native RAN and edge cloud are exposed to edge AI orchestrator for unified management. By integrating intelligence into connectivity and computing, the services are orchestrated based on cloud-network convergence, and thus the task efficiency can be improved. The Application Programming Interface (API) gateway on the edge AI layer may enforce authorization policies for data governance.

3.2 Integrated Abstraction

Built on the technical components including Kubernetes, kernel-based virtual machine and other lightweight platforms, the edge AI layer can abstract the capability of unified application orchestration, resource scheduling, and data management. Through these integrated abstractions, it can decouple from underlying hardware heterogeneity, and provide a capability interface for the application layer to realize AI workflow management and wireless control.

3.3 Operation on Scalable Infrastructure

The heterogeneous infrastructure comprises both communication resources and computation resources, in which Central Processing Unit (CPU) and Application Specific Integrated Circuit (ASIC) are employed for signal processing and networking protocols, while Graphical Processing Unit (GPU) handles AI-driven processing and computing tasks. Based on the scalable infrastructure layer via virtualization, the edge AI layer can allocate on-demand resources for AI-native RAN and edge cloud functions, respectively.

3.4 Co-management of AI and RAN Traffic

By leveraging general-purpose hardware resources and streamlined task processing pipelines, the edge AI layer can support the coexistence of AI and RAN workloads through enhanced traffic management capabilities. Furthermore, for the latency-sensitive traffic, the edge AI layer can intelligently identify service types based on traffic pattern and UE reported QoS, and allocate dedicated resources consequently.

4 Conclusion

In this paper, an edge AI layer is proposed to integrate cross-domain AI collaboration with AI-native RAN, which employs edge AI orchestrator to collaboratively manage the AI workflow, data and resources on AI-native RAN nodes and edge cloud. The future work will focus on the development of AI-native RAN prototype and use case study to provide insights for 6G AI-native network evolution.

References

[1] Zhi Zhou, Xu Chen, En Li, Liekang Zeng, Ke Luo, and Junshan Zhang. 2019. Edge intelligence: Paving the last mile of artificial intelligence with edge computing. *Proc. IEEE* 107, 8 (2019), 1738–1762.

[2] Khaled B Letaief, Yuanming Shi, Jianmin Lu, and Jianhua Lu. 2021. Edge artificial intelligence for 6G: Vision, enabling technologies, and applications. *IEEE journal on selected areas in communications* 40, 1 (2021), 5–36.

[3] Nan Li, Qi Sun, Lehan Wang, Xiaofei Xu, Jinri Huang, Chunhui Liu, Jing Gao, Yuhong Huang, et al. 2025. Towards AI-Native RAN: An Operator's Perspective of 6G Day 1 Standardization. *arXiv preprint arXiv:2507.08403* (2025).

[4] 3GPP TS 23.288. v19.2.0. *Architecture enhancements for 5G System (5GS) to support network data analytics services.* Technical Specification. 3GPP.

[5] O-RAN. WG2. v07.00. *Non-RT RIC: Architecture.* Technical Specification. ORAN.

[6] Lopamudra Kundu, Xingqin Lin, Rajesh Gadiyar, Jean-Francois Lacasse, and Shuvo Chowdhury. 2025. AI-RAN: Transforming RAN with AI-driven Computing Infrastructure. *arXiv preprint arXiv:2501.09007* (2025).

[7] Youbing Hu, Zhijun Li, Yongrui Chen, Yun Cheng, Zhiqiang Cao, and Jie Liu. 2023. Content-aware adaptive device–cloud collaborative inference for object detection. *IEEE Internet of Things Journal* 10, 21 (2023), 19087–19101.

[8] Jiawei Shao and Xuelong Li. 2025. AI flow at the network edge. *IEEE Network* (2025).

Demo: A Drift-Handling Automation in AI/ML Framework Integrated O-RAN

Venkatesh Gani
IIT Dharwad, India
cs25dp001@iitdh.ac.in

Dhruv
IIT Dharwad, India
cs25dp004@iitdh.ac.in

Yaswanth Kumar L.S.
IIT Hyderabad, India
cs24resch11007@iith.ac.in

Ashit Subudhi
IIT Dharwad, India
ashit.subudhi.21@iitdh.ac.in

Majid K
IIT Dharwad, India
cs24dp006@iitdh.ac.in

Abhishek Bhattacharyya
IIT Dharwad, India
CS21MS001.alum24@iitdh.ac.in

Venkateswarlu Gudepu
IIT Dharwad, India
CS21DP003.alum24@iitdh.ac.in

Bheemarjuna Reddy Tamma
IIT Hyderabad, India
tbr@cse.iith.ac.in

Koteswararao Kondepu
IIT Dharwad, India
k.kondepu@iitdh.ac.in

Abstract

Beyond 5G (B5G) networks increasingly leverage open and disaggregated Radio Access Networks (O-RAN) to enable high-speed, low-latency, and flexible services across diverse use cases. However, the dynamic nature of user traffic and network conditions causes Artificial Intelligence (AI)/ Machine Learning (ML) models in RAN Intelligent Controllers (RICs) to experience performance drift – leading to inefficient radio resource allocation and potential service level agreements (SLA) violations. This work demonstrates a real-time *drift detection*, *analysis*, and *adaptation* by leveraging an AI/ML framework integrated O-RAN. This is achieved with real-time data collection from open interfaces which would be used by intelligent controllers (i.e., Non-RT RIC) for training and inference purposes of AI/ML models. The proposed drift-handling automation monitors network performance metrics to identify *model drift* with minimal data and computation overheads and then adapts to current network conditions by either *retraining* the model or *replacing* it with an available pre-trained model. This approach improves reliability, agility, and automation in multi-vendor B5G deployments by means of drift detection and resolution.

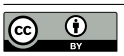
CCS Concepts

• **Networks** → **Network measurement**; **Network experimentation**.

Keywords

Open RAN, Beyond 5G Networks, AI/ML Models, Network Traffic Monitoring, Drift-Handling

ACM Reference Format:
Venkatesh Gani, Dhruv, Yaswanth Kumar L.S., Ashit Subudhi, Majid K, Abhishek Bhattacharyya, Venkateswarlu Gudepu, Bheemarjuna Reddy Tamma, and Koteswararao Kondepu. 2025. Demo: A Drift-Handling Automation in AI/ML Framework Integrated O-RAN. In *2nd ACM Workshop on Open and AI RAN (Open & AI RAN 25), November 4–8, 2025, Hong Kong, China.* ACM, New York, NY, USA, 3 pages. https://doi.org/10.1145/3737900.3770176



OpenRan '25, November 4-8, 2025, Hong Kong, China

ACM ISBN 979-8-4007-1977-6/25/11
https://doi.org/10.1145/3737900.3770176

1 Introduction

In Beyond 5G (B5G) networks, mobile operators aim to support a broader range of services that require higher data rates, ultra-low latency, and greater reliability for billions of connected devices, while simultaneously reducing capital and operational expenditures (CAPEX/OPEX) [7]. Open and disaggregated RAN (O-RAN) plays a central role in this vision by enabling interoperability, programmability, and cost efficiency through open interfaces, virtualization, and AI/ML-driven network automation.

AI/ML models offer a promising approach in O-RAN, as they can effectively handle complex network protocols and make intelligent decisions for network optimization and management [3]. Traditionally, these models are trained on historical or observed data, making their performance highly sensitive to the characteristics of the training dataset. However, in real-world deployments, network traffic evolves continuously due to addition of new devices and roll out of new applications and services. This mismatch between the training data and the current traffic often leads to AI/ML model's performance degradation, a phenomenon commonly referred to as *model or concept drift*.

In [1], model drift is identified when AI/ML model's performance metrics like *accuracy*, *F1-score*, *recall*, and *precision* exceed a predefined threshold. Various drift detection techniques, including Drift Detection Method (DDM) and Early Drift Detection Method (EDDM) are reviewed in [6]. In [8], the fisher score—a statistical measure capturing variation in specific features—is used for drift detection instead of relying on the model's online error rate. In addition, [6] detects drift by monitoring shifts in the incoming user traffic using descriptive statistics such as data range, mean, mode, median, standard deviation, and variance. The authors of [5] introduced a hybrid approach that tracks both error rate as well as changes in the incoming data. All these methods require large training dataset and high computational resources, and also prone to false positives/negatives causing over- or under-provisioning of resources and thereby leading to SLA violations.

This demonstration presents an advanced drift-handling framework – *drift detection*, *drift analysis*, and *drift adaptation* – designed

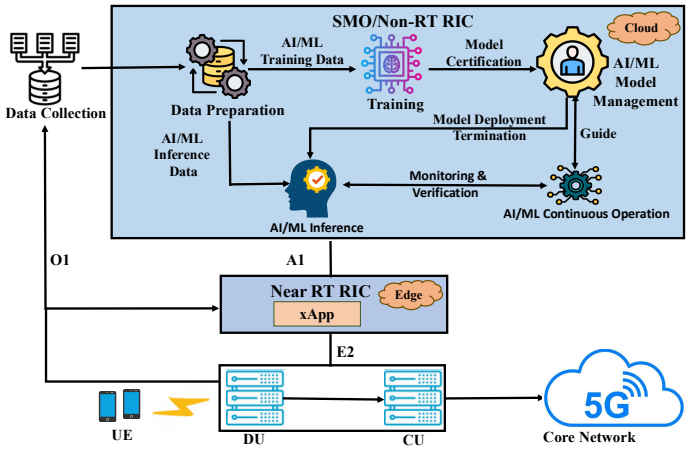


Figure 1: A Drift-Handling Framework in O-RAN

for O-RAN environments. Leveraging automated real-time data collection from open interfaces and AI/ML model training and inference at the RAN intelligent controller (Non-RT RIC), the proposed framework swiftly detects the model drift. Once drift is detected, the framework dynamically selects and implements the most appropriate adaptation strategy: rapidly retrains the model if it is the case of *sudden drift* occurrence or automatically replaces the trained model with an alternative model if it is the case of *incremental drift*. Through this dynamic approach, we demonstrate how the integration of real-time drift detection along with automated adaptation can significantly improve the resilience and efficiency of O-RAN, thereby helping the mobile operators to maintain optimal performance despite evolving user traffic and dynamic network conditions.

2 Drift-Handling Framework

Figure 1 shows the proposed draft-handling framework in O-RAN. RICs provide more granular control and optimization of resource allocation at various operational levels by managing the RAN through Non-RT Applications (*rApps*) and Near-RT Applications (*xApps*). The Non-RT RIC, part of the Service Management and Orchestration (SMO) framework, is responsible for the AI/ML model life-cycle management: data collection, data preparation, model training, model management, model inference, and continuous operation [2], as well as continuous monitoring of *x/rApps* for detecting model drift. The Near-RT RIC provides near-real-time control, optimization and monitoring of O-CU and O-DU nodes via *xApps* [7].

Data collection is a process by which telemetry data (e.g., Uplink/Downlink user throughput) is gathered from various O-RAN entities through open interfaces (E2, O1/O2) and stored inside a time-series database like influxDB. *Data preparation* occurs in an ETL (extract-transform-load) process which comprises of extracting data sources stored in storage repositories such as buckets, volumes, and data lakes, transforming it by cleaning, structuring, and loading the data into target environments.

During the model training, the chosen ML model (e.g., throughput prediction model) is trained, evaluated, and validated to guarantee accuracy and reliability. This itself can be enhanced by optimizing the hyper-parameters of the model. *Model management* is a service where trained models are stored in a centralized model catalog and used for inference purposes. The models are packaged with the appropriate artifacts, metadata, and all supporting information needed to deploy, replicate, and update in the future.

The *AI/ML continuous operation* component facilitates the continuous enhancement of AI/ML models over the AI/ML workflow life-cycle. It is the component that monitors the performance of all other workflow components by continuously monitoring on the processes and the models' performance. It gathers feedback of the system and performs the necessary steps including re-training and replacement of models on the basis of the drift observations, thus facilitating the adaptive and efficient incorporation of AI/ML models in O-RAN.

3 Live Demonstration

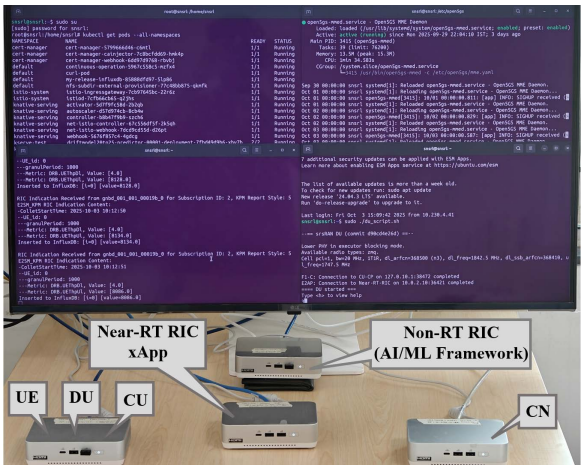


Figure 2: Testbed Setup

Figure 2 shows a snapshot of the testbed setup. This live demo incorporates an Open5GS instance (the core network) and an srsRAN deployment which includes one instance of srsCU, srsDU, and srsUE. The O-RAN Software Community (SC) Near-RT RIC (*i release*)

is deployed as a multi-container app following the O-RAN Alliance specs. The srsCU connects to the core network, and the srsDU interfaces with both the srsCU and the Near-RT RIC. The srsUE talks to the srsDU over-the-air using ZeroMQ (ZMQ), to emulate RF but without actual hardware. A continuous *iperf* session is used to generate the user traffic and the RIC makes use of the *KPM-MON xApp* to gather real-time performance related stats (i.e., downlink and uplink throughput) on a per-second interval over the E2 interface.

```
root@snsrl:/home/snsrl# kubectl get pods
NAME                                          READY   STATUS
continous-operation-bb6475c78-b7h68           1/1     Running
```

Figure 3: Status of Continuous Operation

The Non-RT RIC uses Kubernetes (K8s) for the AI/ML workflow, whose deployment was carried out on an Ubuntu 24.04 mini PC having 22 CPU cores, 64 GB RAM, and enough storage to handle the datasets used for training AI/ML models. It supports an AI/ML framework (*L release*) that allows managing the lifecycle of AI/ML models: training models, retraining, data ingestion, and drift detection. The data collected through O1/E2 interfaces are pre-processed and used to train an LSTM model for throughput prediction which is suited for sequential and time-series data. Once trained and validated, the model and its artifacts are packaged within the model management component, which stores it in a versioned, cataloged, and packaged format to be deployed or retrained in future.

3.1 Drift Detection Methodology

Once the setup is established, KPIs are continuously fed from *KPI-MON xApp* to the influxDB database pod. Inside the Non-RT RIC, the pre-trained deployed model processes the data streams and forecasts the future values, such as UL and DL *user throughputs*.

The *continuous operation pod* continuously monitors incoming KPIs and compares them against predicted values to detect drift in the AI/ML model employed. The main responsibilities of this pod include *drift detection*, *drift analysis*, and *drift adaptability*. Figure 3 represents the status of the continuous operation pod. For drift detection, this pod employs a Root Mean Square Error (RMSE)-based technique. The pod divides the real-time data into sliding windows and calculates RMSE for each window by comparing the model's predicted outputs to the corresponding real-time values reported through KPIs. When consecutive RMSE values exceed a dynamically tuned threshold – calibrated via Particle Swarm Optimization (PSO) to reduce false positives and maximize sensitivity – the pod flags the occurrence of model drift [4].

The key algorithmic parameters used for the drift detection are as follows: the window size, the total number of windows, $RMSE_{th}$, and $flag_{vth}$ (flag violation threshold — a variable to indicate whether the window is exceeding $RMSE_{th}$ RMSE threshold) are set to 5,5,4.5,0.31, respectively. To optimizing these parameters, Particle Swarm Optimization (PSO) is employed with a swarm size of 20 particles iterating over 50 generations. The inertia weight is set to 0.9 to balance exploration and exploitation, while the cognitive and social coefficients are both set to 2.0, ensuring particles are influenced equally by their own best position and the swarm's global best. Convergence is declared when the improvement in the global best fitness value falls below 0.001 for five consecutive iterations, indicating that a near-optimal set of parameters has been found.

3.2 Adaptation Workflow

When consecutive RMSE violations are detected and the RMSE values are observed to be in a strictly increasing sequence, the proposed framework performs a model replacement using a pre-trained model from the catalog. If the RMSE values change abruptly without an increasing trend, this signals sudden drift and triggers model retraining. The model catalog contains pre-trained models that can be swapped at the granularity of 500 ms. The duration required for retraining depends on the volume of new data collected and the batch size used for training.

3.3 Performance Results

During the live demo, we will show four phases : (i) initial operation – where constant-rate iperf traffic yields stable model predictions; (ii) incremental drift – by changing iperf traffic from constant-rate to strictly increasing pattern; (iii) automatic adaptation – where the model swap happens with a pre-trained model; (iv) sudden drift – by changing iperf traffic from constant-rate to a burst pattern where the model automatically goes for the retraining for adapting to the new traffic pattern.

The proposed framework performs well by combining real-time drift monitoring with automatic adaptation. The considered AI/ML model achieves *precision* (0.9785), *recall* (1.0000), and *F1-score* (0.9891), thanks to the dynamic drift-handling mechanism.

Acknowledgments

This work has been supported by TTDF funded "SMART-RIC6G: Smart Drift-Handling Enabler for RAN Intelligent Controllers in 6G Networks" and DST funded "Synergy Taking Openness to the Next Level in 6G Networks" projects.

References

[1] Abu Hena Al Muktadir and Ved P Kafle. 2020. Prediction and dynamic adjustment of resources for latency-sensitive virtual network functions. In *IEEE Conference on Innovation in Clouds, Internet and Networks and Workshops (ICIN)*. 235–242.

[2] O-RAN Alliance. 2021. *O-RAN Working Group 2 AI/ML Workflow Description and Requirements.* Technical Report ORAN-WG2.AIML-v01.03. O-RAN Alliance.

[3] Ioannis A Bartsiokas, Panagiotis K Gkonis, Dimitra I Kaklamani, and Iakovos S Venieris. 2022. ML-based radio resource management in 5G and beyond networks: A survey. *IEEE Access* 10 (2022), 83507–83528.

[4] Venkateswarlu Gudepu, Venkatarami Reddy Chintapalli, Piero Castoldi, Luca Valcarenghi, Bheemarjuna Reddy Tamma, and Koteswararao Kondepu. 2024. The drift handling framework for open radio access networks: An experimental evaluation. *Computer Networks* 243 (2024), 110290.

[5] Meenal Jain, Gagandeep Kaur, and Vikas Saxena. 2022. A K-Means clustering and SVM based hybrid concept drift detection technique for network anomaly detection. *Expert Systems with Applications* 193 (2022), 116510.

[6] Jie Lu, Anjin Liu, Fan Dong, Feng Gu, Joao Gama, and Guangquan Zhang. 2018. Learning under concept drift: A review. *IEEE transactions on knowledge and data engineering* 31, 12 (2018), 2346–2363.

[7] Michele Polese, Leonardo Bonati, Salvatore D'oro, Stefano Basagni, and Tommaso Melodia. 2023. Understanding O-RAN: Architecture, interfaces, algorithms, security, and research challenges. *IEEE Communications Surveys & Tutorials* 25, 2 (2023), 1376–1411.

[8] Kungang Zhang, Anh T Bui, and Daniel W Apley. 2023. Concept drift monitoring and diagnostics of supervised learning models via score vectors. *Technometrics* 65, 2 (2023), 137–149.